

BREDA UNIVERSITY OF APPLIED SCIENCES

Academy of Games and Media | Data Science & Applied Artificial Intelligence



Bachelor's Thesis

**Classical machine learning versus large language models for the severity classification of
cloud-infrastructure incidents**

June 6, 2026

Submitted by

Victoria Vilyanova Vicheva

Student ID: 233182

233182@buas.nl

victoria.v.vicheva@gmail.com

Supervised by

Shival Indermun

Tsegaye Tashu

Breda University of Applied Sciences

Contents

1	Introduction	5
1.1	Context and Relevance	5
1.2	CloudPulse: System Overview and Business Proposition	5
1.3	Problem Statement	6
1.4	Gap in Knowledge	6
1.5	Research Question	7
1.6	Approach Overview	7
1.7	Contributions	7
2	Related Work	9
2.1	AIOps and Automated Incident Management	9
2.2	Classical Machine Learning for Text Classification	9
2.3	Large Language Models and Prompt Engineering	10
2.4	Recommendation Systems for Incident Management	11
2.5	Gap in the Literature	11
2.6	Competitive Landscape and Commercial Alternatives	11
2.6.1	Observability Platforms	12
2.6.2	Incident Response Platforms	12
2.6.3	Status-Page Aggregators	12
2.6.4	Emerging LLM-Based Incident Assistants	13
2.6.5	Positioning of CloudPulse	13
2.7	Summary	14
3	System Design	15
3.1	Architectural Overview	15
3.2	The FastAPI Service and the Integration Contract	16
3.3	Artefact Loading at Startup	17
3.4	Preprocessing Symmetry Between Training and Inference	17
3.5	The Classification Pipeline	17
3.6	Per-Prediction Explainability	18
3.7	Similarity Search	18
3.8	Anomaly Detection	19
3.9	Remediation Generation	19
3.10	Weekly Intelligence Pass	20
3.11	Static Metrics and Health	20
3.12	Failure Modes and Graceful Degradation	20
3.13	Tech Stack	21
3.14	React Dashboard	21
3.15	Summary	25
4	Methodology	26
4.1	Dataset	26
4.2	Preprocessing and Anonymisation	26
4.3	Data Management, Privacy, and Ethics	27
4.4	Architectural Decision: Three-Class Classifier with a Deterministic Rule	27

4.5	Feature Engineering	28
4.6	Classical Machine-Learning Models	29
4.7	Transformer Fine-Tunes	29
4.8	Large Language Model Prompting Baselines	30
4.9	Cross-Validation	31
4.10	Statistical Testing with McNemar's Test	31
4.11	Calibration and Explainability	32
4.12	Anomaly Detection	32
4.13	Likert Evaluation of Generated Remediations	32
4.14	Stress Testing	34
4.15	Reproducibility and Source of Truth	34
4.16	Data Leakage Audit	34
4.17	Summary	35
5	Results	36
5.1	Cross-Model Leaderboard	36
5.2	Per-Class Breakdown for the Production XGBoost Classifier	38
5.3	Cross-Validation Results	39
5.4	McNemar Pairwise Testing	40
5.5	Binary Severe-vs-Not-Severe Task	41
5.6	Transformer Fine-Tunes	41
5.7	LLM as Classifier	43
5.8	Calibration Analysis	45
5.9	Error Analysis	47
5.10	SHAP Global Feature Importance	49
5.11	Likert Evaluation of the Remediation Generator	51
5.12	Automated Stress Test Reproduction	51
5.13	Summary	52
6	Discussion	53
6.1	Classical Machine Learning Versus Transformer Fine-Tuning at Small Scale	53
6.2	The Gap Between Supervised Classifiers and Off-The-Shelf LLM Prompting	54
6.3	The Cross-Component Failure Mode	55
6.4	Industry Evaluation of the Deployed System	56
6.5	The Integration With the Data Engineering Track	57
6.6	Limitations	58
6.6.1	High-Class Misclassification Rate	58
6.6.2	Critical-Class Unlearnability and Rule Dependence	58
6.6.3	Test Set Size and Statistical Power	58
6.6.4	Single-Annotator Likert Evaluation	59
6.6.5	Likert Sample Size	59
6.6.6	Single-Seed Transformer Fine-Tuning	59
6.6.7	Calibration Developed Offline But Not Deployed	59
6.6.8	Anomaly Detector Inference-Time Feature Asymmetry	60
6.6.9	Templated Description Duplicates Across Splits	60
6.6.10	English-Only Operation	61

6.6.11 Stylistic-Register Learning

6.6.12 Provider Distribution Skew

6.6.13 Single-Vendor LLM Dependence and Model Version Anchoring

6.6.14 Skew in the Dataset

6.6.15 Static Models and No Continuous Training

6.7 Future Work

6.8 Reflection

6.9 Summary

7 Conclusion

7.1 Answer to the Research Question

7.2 Supporting Architectural Properties

7.3 The Cross-Component Finding

7.4 Qualifications

7.5 Contribution

List of Abbreviations

Appendix A: ClassifyRequest Schema Fields

Appendix B: ClassifyResponse Schema Fields

Appendix C: Model Artefacts Loaded at Startup

Declaration on the Use of Generative AI

61

61

61

62

62

63

64

66

66

66

66

67

67

67

74

75

75

76

77

3

Abstract

CloudPulse is an automated cloud infrastructure monitoring system that classifies the severity of publicly reported incidents and generates natural language remediation recommendations for cloud operations teams. This study investigates whether classical machine learning models, trained on domain-specific incident data, can compare or surpass large language models in the automatic severity classification of cloud infrastructure incidents. A dataset of 909 incidents was collected from the public status pages of major cloud providers, including AWS, Azure, and GCP, labelled across four severity levels: critical, high, medium, and low. Three classical models were evaluated, namely Logistic Regression, Random Forest, and Linear SVM, using TF-IDF, sentence embeddings, and metadata as feature representations. Due to severe class imbalance in the critical category, a binary reformulation was introduced, collapsing severity into severe and non-severe classes, which yielded more reliable results. Large language model classification was subsequently explored using zero-shot, few-shot, and chain-of-thought prompting strategies via the Claude API, without any task-specific training. Model comparisons were conducted using Macro F1 as the primary evaluation metric and McNemar's test for statistical significance. Results indicate that TF-IDF features consistently outperformed richer combined feature sets across all classical models, and that no single approach demonstrated statistically significant superiority given the dataset size. The system further generates natural language remediation recommendations for classified incidents. Findings suggest that while large language models offer competitive performance without task-specific training, classical models remain a strong and interpretable baseline for structured incident data.

Keywords: Incident Classification, Severity Prediction, AIOps, Large Language Models, Prompt Engineering, Machine Learning, Cloud Infrastructure

1 Introduction

1.1 Context and Relevance

Cloud infrastructure incidents, ranging from minor service degradations to complete regional outages, represent one of the most operationally sensitive challenges facing modern technology organisations. When an incident occurs, the accuracy and speed of its severity assessment directly determines the urgency of the engineering response, the allocation of resources, and ultimately the extent of user impact. At scale, across providers such as Amazon Web Services (AWS), Microsoft Azure, and Google Cloud Platform (GCP), incidents are reported continuously and in high volumes, making manual triage both impractical and inconsistent (Remil et al., 2024; Dang et al., 2019). As cloud infrastructure continues to grow in complexity, the demand for automated and reliable incident triage systems has become a critical operational priority for technology organisations worldwide.

1.2 CloudPulse: System Overview and Business Proposition

CloudPulse is an automated multi-cloud incident intelligence platform developed as a joint graduation project at Breda University of Applied Sciences. The platform addresses an operational gap that affects every organisation running production workloads on public cloud infrastructure: the time and effort required to detect, triage, and respond to incidents published by cloud providers on their public status pages. CloudPulse ingests these incidents in real time across approximately twenty providers, classifies their severity automatically, generates actionable remediation recommendations, and surfaces the result to an on-call engineer through a web dashboard. The product is designed to reduce the cognitive load of incident triage during the first minutes of an outage, when manual review of multiple status pages is slowest and most error-prone, and to give engineering teams a single pane of glass across the cloud and SaaS providers their systems depend on.

The platform is built across two tracks. The data engineering track, owned by the collaborating data engineer Madalina Carcea, is responsible for ingesting incidents from public StatusPage feeds at sub-45-second latency, normalising them into a unified schema, persisting them to a TimescaleDB hypertable on Azure, orchestrating the ingestion pipeline through Apache Airflow, and deploying the AI engineering side's API and dashboard to the shared cloud environment (Carcea, 2026). The AI engineering track, which is the subject of this thesis, is responsible for three artefacts: the trained models that classify each incident's severity and detect anomalies, the FastAPI service that exposes those models together with a Claude Haiku-based remediation generator through five typed HTTP endpoints, and the React dashboard through which a human reviewer consumes the service's output. The two tracks meet at a single integration contract: the typed Pydantic request and response schemas of the FastAPI service.

The business proposition rests on three properties. The first is operational coverage. Existing commercial incident intelligence platforms such as Datadog, PagerDuty, and Atlassian Statuspage either operate on internal customer telemetry rather than provider-side status feeds, or require manual subscription to individual provider status pages without unified classification. CloudPulse occupies the gap between these approaches: it pulls public provider data automatically, applies classification and remediation across providers, and presents the result through a single product surface. The second property is interpretability by design. Every severity prediction returned by the deployed service is accompanied by per-prediction SHAP attribution showing the top five token-level contributions, a model confidence score, and an explicit escalation reason when the deterministic safety rule fires. This makes the service auditable in a way that opaque LLM-based classification cannot match, which matters for the operations engineers who must justify their triage decisions to incident commanders and to post-mortem reviewers. The third property is cost efficiency at production scale. The deployed classifier is a gradient-boosted tree ensemble that runs in tens

of milliseconds per incident on commodity hardware, with the LLM-based remediation generator confined to the genuinely generative portion of the response. This architectural separation, supervised learning where labelled data is available and LLM generation only where it is the right tool, keeps the per-incident inference cost an order of magnitude below an LLM-everywhere design while preserving the response quality that matters to the end user.

The target customer is the operations or platform engineering team at a mid-sized technology organisation whose production systems depend on multiple cloud and SaaS providers, and whose on-call rotation currently includes manual review of public status pages as part of incident response. The deployed Cloud-Pulse service is operational at the time of writing, with the API and dashboard running in the shared cloud environment maintained by the data engineering track and consuming live incidents from the upstream pipeline.

1.3 Problem Statement

Despite the operational importance of incident severity classification, current approaches in industry rely heavily on manual assessment by on-call engineers, a process that is inherently subjective, time-consuming, and difficult to scale (Luo et al., 2025). When incidents are misclassified or delayed in triage, the consequences can range from degraded user experience to prolonged outages affecting millions of users across global services. At the same time, the volume of incidents generated across major cloud providers continues to grow, driven by the increasing complexity of distributed systems and microservice architectures (Dragoni et al., 2017). This creates a clear and urgent need for automated severity classification systems that are both accurate and interpretable, capable of operating reliably on the short, unstructured natural language descriptions that characterise publicly reported cloud incidents. The central challenge is not only whether automation is possible, but which approach, classical machine learning trained on historical data or large language models operating without task-specific training, is better suited to this task in practice.

1.4 Gap in Knowledge

Research in Artificial Intelligence for IT Operations (AIOps) has made considerable progress in automating aspects of incident management, including anomaly detection, root cause analysis, and log classification (Remil et al., 2024). Commercial platforms such as Datadog apply machine learning to severity scoring, yet these systems operate on internal telemetry data, including metrics, traces, and logs, rather than on the natural language incident descriptions that cloud providers publish on public status pages (Dang et al., 2019). The academic literature reflects a similar gap. Prior work by S. Ahmed et al. (2023) demonstrated that classical machine learning models can achieve competitive performance on IT incident severity prediction, yet their study relied on large proprietary datasets of up to 500,000 records collected from internal enterprise systems. This condition does not reflect the data-scarce, publicly available setting that characterises cloud provider status pages, where incident descriptions are short, operator-assigned, and span multiple providers without a unified labelling standard. Furthermore, despite the growing role of large language models in AIOps tasks (Zhang et al., 2024) and recent comparisons of prompting strategies for incident classification in security operations (Almeida et al., 2025), a direct empirical comparison between trained classical machine learning models and LLM prompting strategies on structured incident severity classification from public status page data remains absent from the literature.

1.5 Research Question

This study seeks to address this gap by investigating the following research question: to what extent can classical machine learning models, trained on domain-specific incident data, outperform large language models in the automatic severity classification of cloud infrastructure incidents? This question is practically motivated. Classical models are efficient, interpretable, and deployable at low cost, but require labelled training data and may struggle with rare severity classes. Large language models require no training data but introduce latency, external API dependency, and reduced transparency in decision-making. Understanding which approach performs better on this task has direct implications for the design of automated triage systems in production cloud environments.

1.6 Approach Overview

To answer this research question, a dataset of 909 cloud-incident records was collected from the public status pages of approximately twenty cloud providers, spanning the project's data collection window. Three classical models, together with a gradient-boosted ensemble, were trained and evaluated across multiple feature representations, including TF-IDF, sentence embeddings, and structured metadata features. An LLM-based classifier was subsequently assessed using zero-shot, few-shot, and chain-of-thought prompting strategies (Brown et al., 2020; Wei et al., 2022), without any task-specific training. Macro F1 served as the primary evaluation metric, with McNemar's test applied for statistical comparison between the best-performing approaches.

The classifier comparison is complemented by a structured remediation generator, also driven by the same large language model, which produces an actionable JSON object for each classified incident. The two components, classification and remediation, are evaluated independently and then jointly: classification through macro F1 and per-class error analysis, remediation through a Likert rubric defined a priori, and the two together through a cross-component error analysis that connects classifier failure modes to remediation quality.

1.7 Contributions

CloudPulse is an automated cloud infrastructure monitoring system that ingests publicly reported incidents from cloud provider status pages, classifies their severity, and generates natural language remediation recommendations for operations teams. The system is built across two tracks: a data engineering track responsible for ingestion, storage, and the monitoring dashboard, and an AI engineering track, which is the subject of this thesis, responsible for the classification and remediation components. This thesis makes three contributions, all of which are part of the deployed CloudPulse AI service described in Chapter 3.

The first contribution is an empirical comparison of classical machine learning, transformer fine-tuning, and large language model prompting on cloud-incident severity classification, conducted on a single 909-incident corpus collected from publicly available status pages. The comparison fills the gap identified in Section 4.4: prior work has not run this comparison on public multi-provider data at this scale (S. Ahmed et al., 2023; M. Jin et al., 2026).

The second contribution is a remediation generator and its evaluation. The generator uses Claude Haiku to produce a JSON object with five fields (summary, root-cause hypothesis, immediate mitigation, investigation, prevention) for every classified incident. The fields are validated against a Pydantic schema at the service boundary. The generator is evaluated through a Likert rubric on five dimensions (specificity, correctness, completeness, actionability, prioritisation), each on a 1-5 scale, with anchors set before any

score was assigned and an acceptance threshold of 3.50 set before scoring began (Amidei et al., 2019). The composite score on a random sample of 20 generations is 4.18, above the threshold.

The remediation generator is a contribution, not a side feature, for three reasons. First, the language-generation task is one the supervised classifier cannot perform. The two-component design routes each problem to the technique that fits: classification to the trained model, language generation to the LLM. Second, the Likert rubric and the a priori threshold are part of the contribution. The rubric was set before scoring began, so the 4.18 composite is a test of an acceptance criterion, not a post-hoc summary. Third, the remediation evaluation surfaces a finding that classification alone would miss. The medium-severity slice scores lower (3.95) than either the high or low slice (4.60 in both), and the same upstream description property that drives 40.7% of the classifier's misclassifications, descriptions dominated by resolution-status language rather than active-failure language, also drives the medium-class Likert drop. The remediation evaluation is what makes this cross-component connection visible.

The third contribution is the deployed FastAPI service and React dashboard described in Chapter 3. The service exposes the trained models, the SHAP explainability layer (Lundberg & Lee, 2017), the calibration wrapper, the deterministic escalation rule, the similarity search, the anomaly detector, and the remediation generator through five typed HTTP endpoints. The dashboard, designed and built as part of this AI engineering track and deployed by the data engineering track, renders the service's output for human review.

2 Related Work

2.1 AIOps and Automated Incident Management

The term Artificial Intelligence for IT Operations (AIOps) was introduced by Gartner in 2017 to describe the application of machine learning and big data analytics to automate and enhance IT operations (Dang et al., 2019). Since then, the field has grown considerably, with research spanning anomaly detection, root cause analysis, alert correlation, and automated remediation. Notaro et al. (2021) provide the first comprehensive taxonomy of AIOps methods for failure management, reviewing over 100 papers and establishing a structured five-category framework covering failure detection, prediction, diagnosis, remediation, and adaptation. Their analysis demonstrates that traditional manual and rule-based approaches cannot scale to the data volumes and alert rates generated by modern distributed systems.

The cloud-specific dimension of AIOps has received increasing attention. Cheng et al. (2023) review AIOps on cloud platforms from the perspective of major platform providers, categorising key tasks into incident detection, failure prediction, root cause analysis, and automated action. Their work explicitly discusses minimising mean time to triage (MTTT) and mean time to resolve (MTTR) as primary operational objectives, framing automated severity assessment as a prerequisite for efficient incident response. Ghosh et al. (2022) provide empirical grounding for this concern through a large-scale study of production incidents at Microsoft Teams, finding that approximately 29% of incidents are first detected by external users rather than automated monitoring systems, and that mis-triage frequently adds substantial delays to resolution.

The operational consequences of cloud outages have been extensively documented. Gunawi et al. (2016) analyse 1,247 outage reports across 32 major cloud services over seven years, finding that 31% of services failed to achieve 99% uptime. This scale of disruption establishes the practical motivation for automated triage: at high volumes, human-driven severity assessment is both too slow and too inconsistent to serve as the sole classification mechanism. More recent surveys position AIOps within the broader MLOps and operational research landscape (Diaz-de-Arcaya et al., 2023), while Remil et al. (2024) provide a comprehensive technical review of AIOps frameworks for incident management, proposing a structured incident management procedure and highlighting the lack of standardised conventions across the field. A comprehensive survey of AIOps approaches in the era of large language models (Zhang et al., 2024) further documents the transition from classical ML-based AIOps towards LLM-augmented approaches, covering 183 studies published between 2020 and 2024.

2.2 Classical Machine Learning for Text Classification

Classical machine learning approaches to text classification rely on explicit feature representations derived from document content. The most widely used representation is term frequency-inverse document frequency (TF-IDF), a bag-of-words weighting scheme that captures the relative importance of terms within and across documents. Li et al. (2022) survey text classification from traditional to deep learning methods, demonstrating that TF-IDF combined with linear classifiers such as logistic regression and support vector machines remains highly competitive on short, domain-specific texts, particularly when training data is limited. This finding is relevant to the present study, where incident descriptions are short, technically specialised, and available in modest quantities.

The relative performance of classical versus neural approaches on small imbalanced datasets has been examined directly by Shyrokykh et al. (2023), who compare logistic regression, random forest, and SVM with TF-IDF features against convolutional neural networks and BERT on a dataset of 5,750 short texts with class imbalance. Their results show that classical models perform comparably to deep learning while requiring significantly less computation, a finding that motivates the use of classical baselines in the present

study. Galke and Scherp (2022) reach a similar conclusion, demonstrating that bag-of-words representations with wide multilayer perceptrons approach fine-tuned BERT performance on multiple text classification benchmarks, questioning the assumption that graph-based or sequence-based neural models are necessary.

In the domain of IT operations specifically, Revina et al. (2020) demonstrate that simple TF-IDF-based representations with classical classifiers achieve high-quality predictions for IT ticket complexity classification, arguing that simpler is often better for structured operational text. This work is directly analogous to the incident severity classification task in the present study, as both involve short operator-authored natural language descriptions with a small number of discrete target classes. Where class imbalance is present, standard resampling techniques such as SMOTE (Chawla et al., 2002) provide a principled mechanism for mitigating majority-class dominance during training, and are applied in the present study.

Dense sentence embeddings represent an alternative to TF-IDF. The Sentence-BERT architecture introduced by Reimers and Gurevych (2019) uses siamese BERT networks to produce fixed-size semantic embeddings that capture contextual meaning beyond term co-occurrence. The all-MiniLM-L6-v2 model used in the present study is a distilled variant of this architecture. However, as discussed in the feature engineering literature, the superiority of dense embeddings over TF-IDF is not guaranteed on domain-specific short text classification tasks (Galke & Scherp, 2022), making the empirical comparison of both feature sets a meaningful research contribution. The production classifier in the present study is XGBoost (T. Chen & Guestrin, 2016), a gradient-boosted tree ensemble that has demonstrated strong performance on tabular and sparse feature inputs across a broad range of classification tasks.

2.3 Large Language Models and Prompt Engineering

Large language models demonstrated the capacity for zero-shot and few-shot classification without gradient updates through the seminal GPT-3 work of Brown et al. (2020), which showed that scaling to 175 billion parameters enables strong task-agnostic performance across diverse NLP tasks through in-context learning alone. This established the paradigm that motivates LLM-based classification in this study: rather than training a task-specific model on labelled incident data, an LLM can be prompted with a task description and optionally a small number of examples to perform severity classification without any parameter updates.

The design of effective prompts is itself a research area. Wei et al. (2022) demonstrate that chain-of-thought prompting, in which models are prompted to produce intermediate reasoning steps before committing to a final answer, significantly improves performance on classification and reasoning tasks compared to direct answer prompting. Their experiments show that chain-of-thought reasoning emerges as a property of model scale and is not present in smaller models. This motivates the inclusion of chain-of-thought as one of the three prompting strategies evaluated in the present study, alongside zero-shot and few-shot approaches.

Sahoo et al. (2024) provide a systematic survey of prompt engineering techniques, covering zero-shot, few-shot, chain-of-thought, self-consistency, and tree-of-thought prompting across diverse tasks and model families. Their taxonomy provides the conceptual framework for the prompting strategies evaluated in the present study. The application of LLMs specifically to IT operations has been explored through OWL (Guo et al., 2024), the first domain-specialised large language model for IT operations, trained on a curated instruction dataset and evaluated across multiple AIOps tasks. OWL demonstrates that domain-specialised LLMs outperform general-purpose models on IT operations tasks, suggesting that the general-purpose Claude Haiku model evaluated in the present study represents a conservative lower bound on LLM performance for this task.

2.4 Recommendation Systems for Incident Management

Beyond classification, the generation of natural language remediation recommendations represents a growing area of research at the intersection of LLMs and incident management. T. Ahmed et al. (2023) conduct the first large-scale study of LLM-generated root cause and mitigation recommendations for cloud incidents, evaluating fine-tuned GPT-3.5 on over 40,000 incidents from more than 1,000 Microsoft cloud services. Their results show a 45.5% improvement in lexical similarity for root cause generation, with over 70% of on-call engineers rating generated recommendations at 3 or above on a 5-point Likert scale (Joshi et al., 2015) for production usefulness. This work directly motivates the remediation recommendation component of the present study and provides a methodological precedent for human-rated Likert-scale evaluation.

Y. Chen et al. (2024) introduce RCACopilot, a production-deployed system at Microsoft that uses LLMs with chain-of-thought prompting and in-context learning to predict root cause categories and generate explanatory recommendations for cloud incidents. The system achieves root cause prediction accuracy of up to 0.766 and has been deployed in active production use. Jiang et al. (2024) extend this paradigm to diagnostic query recommendations, using LLMs to automatically generate KQL queries that guide on-call engineers during incident investigation. P. Jin et al. (2023) present Oasis, an LLM-based system that generates human-readable outage impact summaries and remediation guidance, demonstrated across 18 real-world cloud systems. Most directly relevant to the present study, Sarda et al. (2024) demonstrate that LLMs can generate executable Ansible playbooks for automatically remediating microservice incidents, covering network failures, resource constraints, and configuration errors.

2.5 Gap in the Literature

The literature reviewed above establishes several important findings. First, classical machine learning with TF-IDF features remains competitive with neural approaches on short, domain-specific, imbalanced text classification tasks. Second, LLMs are capable of zero-shot and few-shot classification without labelled training data, and chain-of-thought prompting can improve their performance. Third, LLMs have been successfully applied to root cause analysis, outage summarisation, and remediation recommendation generation for cloud incidents. However, a direct empirical comparison between trained classical machine learning models and LLM prompting strategies on the specific task of severity classification from publicly available, multi-provider cloud incident descriptions has not been conducted. Prior severity classification work such as S. Ahmed et al. (2023) relied on large proprietary enterprise datasets of up to 500,000 records, which do not reflect the data-scarce, publicly available setting that characterises cloud provider status pages. Commercial platforms such as Datadog apply machine learning to severity scoring, but operate on internal telemetry data rather than natural language incident descriptions. This study addresses the gap by conducting a controlled empirical comparison of both approaches on a publicly collected, multi-provider dataset, providing evidence to guide practitioners designing automated triage systems for cloud infrastructure.

2.6 Competitive Landscape and Commercial Alternatives

The academic literature reviewed above, establishes the research gap that this thesis addresses. A parallel gap exists in the commercial landscape, where multiple categories of product touch the cloud-incident response workflow but none combine the specific properties that CloudPulse will be delivering. This subsection positions CloudPulse against four categories of existing tool: observability platforms, incident response platforms, status-page aggregators, and emerging LLM-based incident assistants. Multi-cloud strategies have become the dominant operational pattern across enterprise organisations, with industry surveys reporting

that the majority of organisations now operate across two or more cloud providers (Flexera, 2024; HashiCorp, 2023), which makes the cross-provider incident-monitoring problem that CloudPulse addresses a structural feature of the contemporary cloud landscape rather than an edge case. The purpose of this subsection is to ground the thesis's contribution in the operational reality that practitioners face when they consider whether automated severity classification from public status-page text is a problem worth solving with the architecture proposed here.

2.6.1 Observability Platforms

Datadog, New Relic, Splunk Observability Cloud, and Dynatrace dominate the commercial observability market (Byrne et al., 2024). These platforms ingest internal telemetry from a customer's own infrastructure (metrics, traces, logs, real-user monitoring) and apply machine learning to anomaly detection, alert correlation, and increasingly to severity scoring (Dang et al., 2019; Datadog Inc., 2024; New Relic Inc., 2024). Datadog's Watchdog and New Relic's Applied Intelligence both surface severity-ranked anomalies to on-call engineers (Datadog Inc., 2024; New Relic Inc., 2024), and recent product additions have introduced LLM-based incident summarisation features. However, all four platforms operate exclusively on the customer's own internal telemetry. They do not ingest, classify, or correlate incidents from external cloud-provider status pages, which is the upstream data source that drives CloudPulse. When a major cloud provider experiences an outage, the observability platform reports the downstream symptoms in the customer's environment (failed requests, elevated latencies, error spikes) but does not surface the upstream cause unless the customer's engineering team manually correlates the symptoms against the provider's status page. This is precisely the manual triage step that CloudPulse automates.

2.6.2 Incident Response Platforms

PagerDuty, Opsgenie, FireHydrant, and incident.io occupy the incident response category (Atlassian Corporation, 2024a; incident.io Ltd, 2024; PagerDuty Inc., 2024), providing on-call scheduling, alert routing, incident declaration workflows, and post-mortem tooling. PagerDuty's AIOps tier adds machine-learning-driven alert grouping and noise reduction (PagerDuty Inc., 2024), and PagerDuty has announced LLM-based features for incident summarisation and runbook generation. These tools operate downstream of detection: they assume an incident has already been declared and focus on coordinating the response. The severity classification step itself is either manual (an engineer declares the incident at a chosen severity tier) or driven by simple threshold rules over alert payloads. None of the major incident response platforms ingests public cloud-provider status-page text and produces a severity classification from it.

2.6.3 Status-Page Aggregators

A small set of products targets the specific problem of aggregating status across multiple cloud and SaaS providers. StatusGator, IsDown, and Atlassian Statuspage aggregate public status feeds from hundreds of providers into unified dashboards (Atlassian Corporation, 2024b; IsDown, 2024; Nimble Industries Inc., 2024). StatusGator and IsDown are the closest commercial analogues to CloudPulse's upstream ingestion layer, while Atlassian Statuspage is the provider-facing platform that publishes the feeds CloudPulse consumes. However, the aggregator products operate as passive consumers: they surface the provider's own self-reported severity tag and display the raw incident text without applying further classification, explainability, or remediation generation. A user of StatusGator who sees an incident reported as "minor" by the provider must still read the incident description, assess whether the self-reported severity matches the description's actual content, and decide what action to take. The classification re-evaluation that CloudPulse performs (often catching

incidents that the provider tagged conservatively but whose text describes severe impact) and the remediation generation are both absent from this product category. The absence is consistent with the data engineering literature's observation that public-status-page ingestion has received limited treatment compared with the downstream AIOps tasks it could feed (Carcea, 2026).

2.6.4 Emerging LLM-Based Incident Assistants

The most recent category of commercial activity is LLM-based incident assistants that integrate with existing observability or incident-response platforms. Examples include Rootly's AI, FireHydrant's Runbook AI, and incident.io's AI features (FireHydrant Inc., 2024; incident.io Ltd, 2024; Rootly Inc., 2024). These products use general-purpose LLMs to summarise incidents, suggest mitigations, and draft post-mortems, typically inside the workflow of a paid incident-response platform. The research equivalents of this commercial trend are documented in the academic literature reviewed above, with Y. Chen et al. (2024) and P. Jin et al. (2023) as the most directly comparable systems. The architectural assumption of this category is that LLMs are the correct tool for both the structured classification task and the generative remediation task. The empirical results of the present thesis challenge that assumption for the classification half of the workflow: the same Claude Haiku model that scores 4.18 out of 5 on the Likert composite for remediation generation collapses to the majority class on the severity classification task and produces accuracy below the trivial always-predict-medium baseline. The implication for the LLM-assistant category is that the architectural choice to route every task through an LLM call leaves measurable accuracy on the table when a trained classifier could do the structured-prediction work more reliably.

2.6.5 Positioning of CloudPulse

CloudPulse occupies a specific gap that none of the four commercial categories above currently fills. Relative to observability platforms, CloudPulse adds the upstream public-provider data source that observability tools do not ingest. Relative to incident response platforms, CloudPulse adds the severity classification step that those tools assume is already complete. Relative to status-page aggregators, CloudPulse adds the trained classifier, the SHAP-based explainability, the deterministic safety rule, and the structured remediation generator that aggregators do not provide. Relative to emerging LLM-based incident assistants, CloudPulse adopts the architectural position that the empirical results of this thesis support: a trained classifier where structured prediction is the task, and an LLM only where the task is genuinely generative.

The competitive positioning is consistent with the academic gap identified in Section 2.5. The absence of a direct empirical comparison between classical machine learning and LLM prompting on public multi-provider incident classification is not only a literature gap, it is also the architectural question that the emerging LLM-assistant product category has been answering by assumption rather than by measurement.

2.7 Summary

This chapter has reviewed the literature across four areas relevant to the present study. AIOps research has established automated incident management as a practical necessity, with manual severity assessment identified as a bottleneck that does not scale to the volumes and rates of modern cloud operations. Classical machine learning with TF-IDF features has been shown to remain competitive with neural approaches on short, domain-specific, and imbalanced text classification tasks, motivating its use as the primary modelling approach in this study. Large language models have demonstrated strong zero-shot and few-shot classification capability through in-context learning, with chain-of-thought prompting offering further improvements on structured reasoning tasks. Finally, LLMs have been successfully applied to remediation recommendation generation in production cloud environments, providing both a methodological precedent and an evaluation framework for the remediation component of this study. The gap that unifies these threads is the absence of a controlled empirical comparison between trained classical models and LLM prompting strategies on publicly available, multi-provider cloud incident data, which the present study addresses.

3 System Design

This chapter describes the deployed CloudPulse AI service: a FastAPI application that integrates the trained classifier, explainability, anomaly detection, remediation generation, and weekly intelligence into a single endpoint surface consumed through a React dashboard. Methodology will describe what was trained and how it was evaluated. This chapter describes what runs in production: how artefacts are loaded at service startup, how a single incident moves through the classification pipeline, what each component returns to the caller, how the React dashboard renders that response for human review, and how the service degrades gracefully when external dependencies fail. The end-to-end design earns its product defensibility through three properties: a typed integration contract, identical preprocessing at training and inference time, and component-level fault isolation.

3.1 Architectural Overview

CloudPulse is a two-track project that jointly delivers a customer-facing product. The data engineering track, owned by the collaborating data engineer Madalina Carcea, is responsible for acquiring incidents from public StatusPage feeds, normalising them into a unified schema, persisting them to the upstream Azure database, orchestrating the ingestion pipeline through Apache Airflow, and deploying the AI engineering side's FastAPI service and React dashboard to the shared cloud environment (Carcea, 2026). The AI engineering track, which is the subject of this thesis, is responsible for the trained models, the FastAPI service that exposes them, and the React dashboard through which a human reviewer consumes the service's output. The two tracks meet at exactly one place: the FastAPI service described in this chapter.

The AI engineering track therefore comprises three artefacts that this thesis claims as its contribution: the trained models and their evaluation (Chapters 4 and 5), the FastAPI service that exposes them through a typed contract, and the React dashboard that consumes that contract for human review (Section 3.14). The cloud deployment of both the API and the dashboard to the shared environment is the data engineering track's responsibility, the design and implementation of the API and the dashboard, the trained models they expose, and the typed contract that connects them belong to the AI engineering work reported here.

The deployed FastAPI service is a single Python 3.13 application served by `uvicorn`. It exposes five HTTP endpoints (Section 3.2), loads its model artefacts once at process startup (Section 3.3), and serves incidents one at a time. Every endpoint is fully typed at both the request and response boundaries through Pydantic schemas defined in `ml/api/schemas.py`, and every classifier output that the service emits has a deterministic component layered on top before the response is returned. The service is stateless across requests, all per-incident reasoning is computed on demand from the loaded artefacts and the incoming request body. The service has two consumers: the upstream pipeline maintained by Madalina Carcea, which calls `/classify` per incident as part of the data engineering flow, and the React dashboard, which calls every endpoint to render incidents for human review.

The architectural pattern is a deliberate inversion of the LLM-everywhere design pattern that has become common in recent incident-management prototypes (Guo et al., 2024; M. Jin et al., 2026). Severity classification, explainability, similarity, and anomaly detection use trained models that produce deterministic, fast, and inspectable outputs. The two LLM-based components, remediation generation and the weekly summary, are confined to genuinely generative tasks where a trained classifier has nothing to offer, which is consistent with the way recent work uses LLMs as remediation recommenders rather than as classifiers (T. Ahmed et al., 2023). The result is that the majority of the response payload for a single `/classify` call (severity, confidence, SHAP, anomaly, similarity if requested) is produced by trained models in tens of milliseconds, and the remaining LLM-based remediation block is generated through a single Anthropic API call that takes

one to two seconds and is allowed to fail without taking the rest of the response with it.

3.2 The FastAPI Service and the Integration Contract

The service exposes five endpoints, summarised in Table 1:

Table 1: FastAPI endpoints exposed by the CloudPulse AI service.

Endpoint	Method	Purpose
/classify	POST	Classifies one incident end-to-end: severity (3-class and binary), model confidence, escalation rule result, SHAP explanation, anomaly score, and optional Claude Haiku remediation.
/similar	POST	Returns the top-k most similar past incidents from the historical corpus using MiniLM sentence embeddings and cosine similarity.
/weekly	POST	Consumes a batch of already-classified incidents over a date range and returns a deterministic aggregation (hotspots, severity counts, week-on-week deltas) plus a Claude Haiku natural-language intelligence report.
/metrics	GET	Returns the static evaluation results (test-set macro F1, per-class F1, best model per task) computed at training time.
/health	GET	Readiness probe that reports which model artefacts loaded successfully at startup.

The request schema, `ClassifyRequest`, accepts seven fields, documented in Appendix 7.5.

In addition, the URL accepts one query parameter, `include_remediation` (bool, default true). When set to false, the service skips the Claude Haiku call and the response omits the remediation block, reducing latency from approximately 1.5 seconds to under 100 milliseconds. The flag lives in the URL rather than the request body because it is a caller control rather than incident data.

Two design choices in the request schema are worth highlighting. First, `provider` and `service` are accepted in the request body even though they are not used by the classifier. The classifier is trained on anonymised text where provider names are replaced with the sentinel token `PROVIDER` (Section 3.4), and the anonymisation function is applied identically at inference time. The fields are accepted because the upstream pipeline already has them and because forwarding them keeps the request schema symmetric with the data engineering side's data model (Carcea, 2026). Second, every metadata field except `incident_id` and `description` is optional, and the service degrades gracefully when fields are absent. If `original_impact` is missing, the escalation rule simply does not fire and the model prediction stands. If `service_impact` or `affected_services_count` is missing, the anomaly detector falls back to default values. This is a deliberate robustness property: the contract is permissive on input so that the data engineering pipeline can evolve without requiring breaking changes to the AI service.

The response schema, `ClassifyResponse`, returns twelve fields covering the classification output, escalation result, explainability, anomaly detection, and optional remediation, documented in Appendix 7.5.

The `explanation` and `anomaly` fields are typed as `dict` rather than as nested `BaseModel` classes. This is a deliberate flexibility trade-off: the wire format is stable but not contractually enforced, which allows the data engineering side and the dashboard to extract or ignore fields without breaking on Pydantic validation. The `remediation` field, by contrast, is a strict nested model because its consumers (the dashboard, future automation) rely on the five-field structure for layout.

3.3 Artefact Loading at Startup

All model artefacts are loaded once at service startup through a FastAPI `lifespan` context manager that calls `load_artefacts()` from `ml/api/artefacts.py`. The artefacts directory is organised per notebook so the boot sequence mirrors the training pipeline:

The artefacts directory is organised per notebook so the boot sequence mirrors the training pipeline, with six artefact files loaded in total (see Appendix 7.5).

Each path is checked at boot. If a file is missing, the service fails loudly during startup rather than at the first request that needs it. This is the canonical fail-fast pattern for serving ML artefacts: production traffic is never the place to discover that an artefact did not deploy. The `Artefacts` dataclass collects all loaded objects into a single container that the request handlers reference through a module-level reference, which keeps the lifespan logic isolated and the request handlers free of file-system or deserialisation code.

3.4 Preprocessing Symmetry Between Training and Inference

The deployed service shares one preprocessing function, with the training pipeline. The function replaces URLs with the token `URL` and provider names with the token `PROVIDER`, then collapses repeated whitespace. The provider list is hard-coded as 33 entries spanning cloud-infrastructure name variants (`amazon`, `aws`, `azure`, `microsoft azure`, `google cloud`, `gcp`), the SaaS providers whose `StatusPage` feeds are represented in the dataset, and a small number of additional infrastructure and observability names that providers reference inside incident text. The list is sorted by length descending so that multi-word names such as “google cloud” are matched before the substring “google”. This identical implementation prevents the most common deployment failure mode for text classifiers, where the training pipeline and the serving pipeline preprocess text slightly differently and the served model drifts away from its evaluation distribution. The Methodology chapter describes the rationale, this chapter notes the production consequence, which is that the automated stress test described in Chapter 5 reproduces the offline test-set macro F1 to four decimal places.

3.5 The Classification Pipeline

A `/classify` request flows through six stages inside the service, in this order:

1. **Anonymisation.** The description string is passed through `anonymise()`. If the result is empty (for example, the entire description was a URL), the service returns HTTP 422.
2. **TF-IDF transformation.** The anonymised text is transformed into a sparse feature vector by the loaded vectoriser.
3. **Three-class prediction.** The vector is passed to the 3-class XGBoost classifier, which returns class probabilities through its softmax output. The predicted label is the `argmax`, the softmax probability of the predicted class becomes the `confidence_score`. A sigmoid-calibrated wrapper was developed and evaluated offline as part of the work reported in Chapters 4 and 5, but deployment testing showed that wiring it into the API degraded test-set macro F1, so the deployed service serves uncalibrated softmax probabilities. The constraint is discussed in Section 6.6.
4. **Binary prediction.** The same vector is passed to the binary classifier, which returns `severe` or `not_severe`. This is a secondary signal for the dashboard, not a confidence gate on the 3-class prediction.

5. **Escalation rule.** The 3-class prediction and the `original_impact` field are passed to `apply_escalation_rule()`. If the prediction is high and `original_impact == "critical"`, the final label is upgraded to `critical` and `escalation_reason` is set to `"provider_marked_critical"`. Otherwise the model prediction stands and `escalation_reason` is null. The rule is asymmetric by design: it can promote, never demote, and it operates only on the structured metadata field, never on the free-text description.
6. **Explanation, anomaly, remediation.** The SHAP explainer is queried for the top-five contributions toward the predicted class. The Isolation Forest scores the structured metadata. If `include_remediation=true` (the default), Claude Haiku is called once to generate the structured remediation block.

The response is assembled with all six results and the wall-clock duration in milliseconds. End-to-end latency is dominated by the Anthropic API call: with the remediation block enabled, a typical `/classify` response takes one to two seconds, with the flag disabled, latency drops to tens of milliseconds.

3.6 Per-Prediction Explainability

The SHAP explainability layer is implemented as a thin wrapper around a SHAP `TreeExplainer` loaded from `joblib` (Lundberg & Lee, 2017; Lundberg et al., 2020). The wrapper exposes a single method, `explain()`, which takes the sparse TF-IDF vector for the current incident, the predicted class label, and the desired number of top contributions. The method uses SHAP's `shap_values()` interface, which returns a list of arrays indexed by class for tree models. The wrapper selects the slice corresponding to the predicted class, masks out features that are zero in the input (only tokens that actually appear in the description can contribute), ranks the remaining tokens by absolute SHAP magnitude, and returns the top-k with their TF-IDF weight, SHAP value, and direction (`toward_high` or `away_from_high`, for example).

One design choice in the wrapper is worth flagging. When no tokens are present in the input after anonymisation (an edge case where the description consisted entirely of URLs or provider names), the explanation falls back to an empty list and returns the expected value for the predicted class. The endpoint does not fail, the dashboard simply receives an empty top-k list.

3.7 Similarity Search

The `/similar` endpoint serves dense-embedding similarity search over the historical corpus. At startup, the service loads a precomputed matrix of L2-normalised MiniLM embeddings (`minilm_embeddings.npy`, shape 909×384). At request time, the incoming description is anonymised, encoded through the same `all-MiniLM-L6-v2` model (Reimers & Gurevych, 2019), and L2-normalised. Cosine similarity reduces to a dot product against the precomputed matrix, which is a single matrix-vector multiplication, and the top-k indices are returned with their similarity scores and the matching incident metadata.

The endpoint exists for two reasons. First, dense-embedding similarity is the native strength of MiniLM, even when MiniLM features did not outperform TF-IDF for severity classification (a finding reported in Chapter 5). Second, the response from `/similar` gives the consuming dashboard a way to surface past incidents that resemble the current one, which is useful both for human reviewers (“we have seen this kind of failure before”) and as input to a future similarity-aware automation. The corpus contains all 909 incidents from the deduplicated dataset, including those used for training, validation, and test. This reflects the production deployment, where every classified incident is a candidate neighbour for subsequent incidents. The similarity component is not evaluated as a supervised model, it is a retrieval index over historical incidents, and its quality is not bounded by the train/test split. The endpoint is fast: the matrix-vector multiplication completes in under 50 milliseconds for the 909-incident corpus, and most of the latency in the automated stress test reproduction is HTTP round-trip and JSON serialisation overhead.

3.8 Anomaly Detection

The anomaly detector is an Isolation Forest (F. T. Liu et al., 2008) loaded at startup as a bundle of `{model, scaler, feature_columns}`. The trained feature set comprises eight columns: description length, resolution time in minutes, affected services count, and five one-hot encoded service-impact categories (`ai`, `compute`, `network`, `platform`, `storage`). At inference time, the metadata fields from the `/classify` request are assembled into a one-row `DataFrame`, scaled, and passed to the model. The endpoint returns both a Boolean `is_anomaly` flag (true when the model predicts -1) and the continuous decision-function score, which is more informative than the flag alone and which the dashboard can threshold differently.

The anomaly layer is not a classifier in its own right, it is a complementary signal that flags incidents whose structured metadata falls outside the typical training distribution. The component is computationally trivial, a fraction of a millisecond per request, and adds no latency relative to the classification pipeline.

One of the eight trained features, `resolution_time_minutes`, is not part of the `ClassifyRequest` schema and is therefore not available at inference. The deployed handler substitutes the value zero for this column when assembling the inference-time feature vector. The severity classifier explicitly excluded `resolution_time_minutes` on data-leakage grounds (Kapoor & Narayanan, 2023), as Methodology describes, because the field is set only after an incident is resolved and is therefore unavailable when the incident is first classified. The anomaly detector was nonetheless trained with the column included, because the anomaly detector is unsupervised and the column contributes to the training distribution. The consequence is a known asymmetry between the trained anomaly distribution (which has real resolution times) and the served anomaly scores (which always carry the substituted zero). The anomaly scores reported by the deployed service are therefore systematically biased toward the lower-resolution-time region of the training distribution, and this asymmetry is treated as a limitation in Section 6.6.

3.9 Remediation Generation

The remediation generator is the only LLM-based component in the `/classify` response. The generator calls Claude Haiku 4.5 (`claude-haiku-4-5-20251001`) with a fixed system prompt that defines a strict JSON schema (five fields: summary, root cause hypothesis, immediate mitigation, investigation, prevention) and a user message containing the severity label and the raw description. The response is parsed as JSON, stripped of optional markdown fences, validated through the `RemediationBlock` Pydantic model, and returned to the caller. This pattern of using an LLM as a remediation recommender rather than as a primary classifier follows recent work on LLM-based remediation for cloud incidents (T. Ahmed et al., 2023; Y. Chen et al., 2024).

Two design choices in the generator are worth discussing. First, the prompt instructs the model to “be concrete” and explicitly bans “hedging language” and “general SRE advice.” This is the language used in the Likert evaluation reported in Chapter 5. The prompt was iteratively refined during development based on inspection of early generations that scored low on specificity, and the rated batch reported in Chapter 5 was generated with the final version of the prompt. Second, the function catches every exception and returns `None` rather than re-raising. The intent is that an Anthropic API failure or a malformed JSON response should not fail the entire `/classify` request: the dashboard receives the classification result with a null remediation field and renders a placeholder. This is the central fault-isolation property of the deployed service.

The remediation generator deliberately does not see the classifier’s confidence score, the SHAP explanation, or the binary head’s output. It receives only the severity label and the original description. This isolation matches the inputs available in production. The Likert composite reported in Chapter 5 therefore represents the deployed system as it stands, not an idealised richer context.

3.10 Weekly Intelligence Pass

The `/weekly` endpoint accepts a batch of already-classified incidents over a date range plus an optional previous-week batch for trend deltas. The handler runs in two stages.

The first stage is deterministic. A pure-Python `aggregate()` function over the incident list computes severity counts, provider-by-severity and service-by-severity matrices, top hotspots (providers and services ranked by a severe score that double-weights critical incidents), the notable-incidents list (critical and high incidents, truncated at ten), and the week-on-week severity trend (with per-class deltas and percent changes) when a previous-week batch is supplied. The output is a typed `WeeklyAggregation` dataclass that can be serialised to a dict.

The second stage is LLM-based. The aggregation dict is passed to Claude Haiku as structured input, together with a fixed system prompt that defines four sections (executive summary, top hotspots, trend, what to watch next week) and asks for a CTO-style briefing. The LLM consumes the aggregation, not the raw incident descriptions, which keeps the prompt size bounded and keeps the generated summary grounded in counts and hotspots rather than in free-form description text. This two-stage design is the most distinctive architectural choice in the AI service: hotspot detection and trend computation are deterministic and auditable, language generation is delegated to the LLM only where its native strength applies (P. Jin et al., 2023).

3.11 Static Metrics and Health

The `/metrics` endpoint serves the static cross-model evaluation results from training time, loaded once at startup from the results CSVs. It is intended for the dashboard to display the deployed model's performance characteristics to a human operator, it does not run any model. The `/health` endpoint returns a status flag (ok or degraded) and a dict of which artefacts loaded successfully at startup. The status is ok only when all required artefacts loaded. The endpoint is consumed by the upstream pipeline maintained by Carcea as a readiness probe, by orchestration tooling for restart logic, and by the dashboard for the degraded-state banner.

3.12 Failure Modes and Graceful Degradation

The deployed service has three documented failure modes, each with a corresponding degradation path:

- **Anthropic API failure.** The remediation generator catches all exceptions and returns `None`. The `/classify` response is returned with a null remediation field. The dashboard renders a placeholder. The classification, explanation, anomaly, and confidence results are unaffected.
- **Missing optional metadata.** If `original_impact` is null, the escalation rule does not fire and the model prediction stands. If `service_impact` or `affected_services_count` is null, the anomaly detector substitutes defaults. The classification pipeline always completes.
- **Missing artefact at boot.** The `load_artefacts()` function raises `FileNotFoundError` at startup, preventing the service from accepting traffic. The `/health` endpoint, if reached during a partial-degraded state, returns the per-artefact status dict so the upstream pipeline and the dashboard can route around the service.

Of the three degradation paths above, the Anthropic API failure case is the most consequential for the deployed system. The remediation generator is the only component that introduces an external runtime dependency: every other part of the `/classify` response is computed locally from artefacts loaded at startup.

Isolating that dependency so that its failure cannot propagate to the classification, explanation, anomaly, or confidence outputs is a deliberate architectural decision. A reviewer receiving a null remediation field still has a severity label, a confidence band, an escalation flag, and a SHAP explanation, enough to triage the incident. The degraded response is therefore useful, not broken.

The pattern across all three is the same: failures are contained to the component that caused them, and the contract with the caller does not silently change. A null remediation field is observable in the response, a missing escalation_reason is observable, and a degraded /health status is observable. Silent fallbacks are avoided because they would erode the dashboard's ability to distinguish a successful classification from a partially failed one.

3.13 Tech Stack

The service runs on Python 3.13. The HTTP layer is FastAPI with uvicorn as the ASGI server, and the data contract is enforced by Pydantic v2. The model artefacts are serialised with joblib (for scikit-learn and XGBoost objects) and numpy (for the MiniLM embedding matrix). The numerical libraries are scikit-learn, xgboost (T. Chen & Guestrin, 2016), shap, and sentence-transformers (loading all-MiniLM-L6-v2 at boot for query encoding). The Anthropic SDK provides the Claude Haiku client used by the remediation generator and the weekly summary. Configuration is loaded from environment variables via python-dotenv, the only required variable is ANTHROPIC_API_KEY. CORS is enabled to support the dashboard during development and is configured for the production origins when the service runs in the shared cloud environment.

The service is stateless and horizontally scalable in principle: each instance loads the same artefacts at startup and serves requests independently. The deployment of the service to the shared cloud environment is the data engineering track's contribution, the service runs in production alongside the React dashboard described in Section 3.14, and both are reachable through the upstream pipeline's existing infrastructure (Carcea, 2026).

3.14 React Dashboard

The React dashboard is the customer-facing surface of the CloudPulse product and the human-facing consumer of the FastAPI service. It was designed and implemented as part of the AI engineering work reported in this thesis and is deployed by the data engineering track to the shared cloud environment alongside the API. The dashboard's role in the deployed system is to render the API's output in a form that an on-call engineer or incident reviewer can read at a glance, and to support the human-in-the-loop review that the high-class misclassification rate (a finding reported in Chapter 5 and discussed in Section 6.6) makes necessary.

The dashboard consumes every endpoint of the API. The /classify response populates the per-incident view, which displays the final severity label, the confidence band, the escalation reason when the rule has fired, the top five SHAP contributions with their direction toward or away from the predicted class, the anomaly flag, and the structured remediation block. Figure 1 shows the per-incident view for a representative Cloudflare incident, and Figure 2 shows the SHAP contribution panel in detail. The remediation block for the same incident is shown in Figure 3.

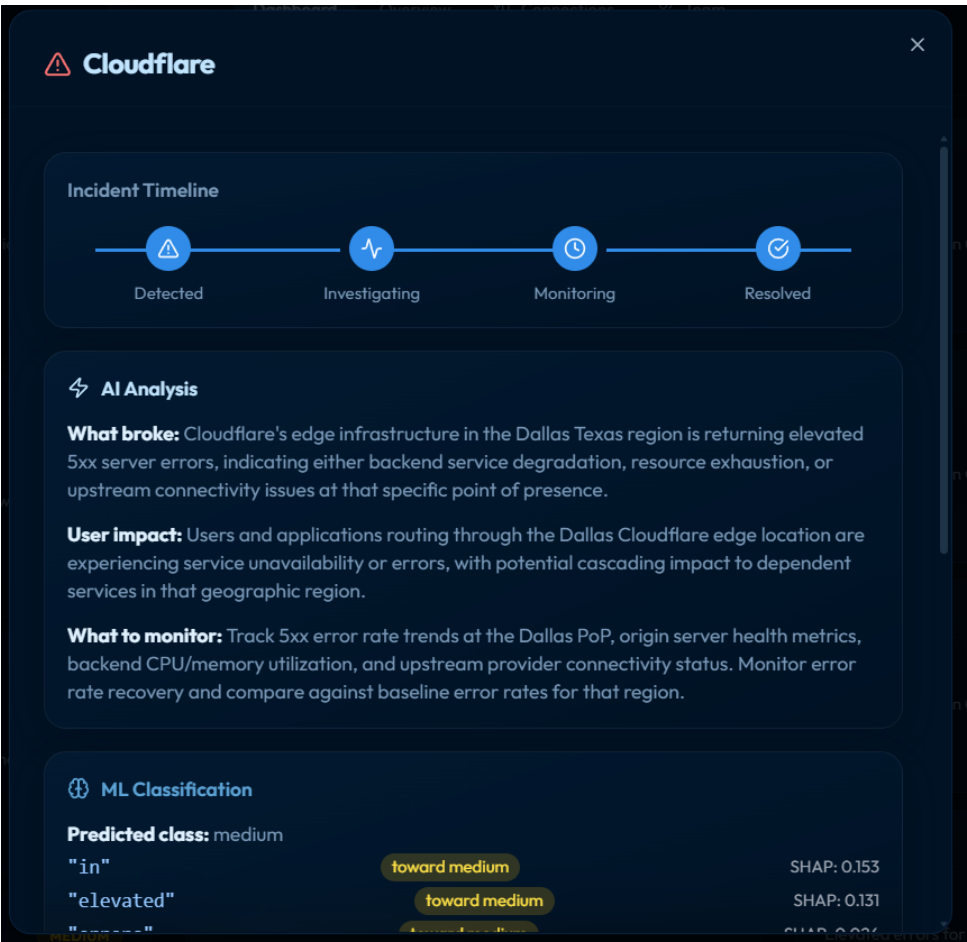


Figure 1: CloudPulse per-incident view showing the incident timeline and ML classification panel with SHAP contributions for a Cloudflare medium-severity incident.



Figure 2: SHAP contribution panel showing the top five tokens and their directional influence on the predicted medium-severity class.

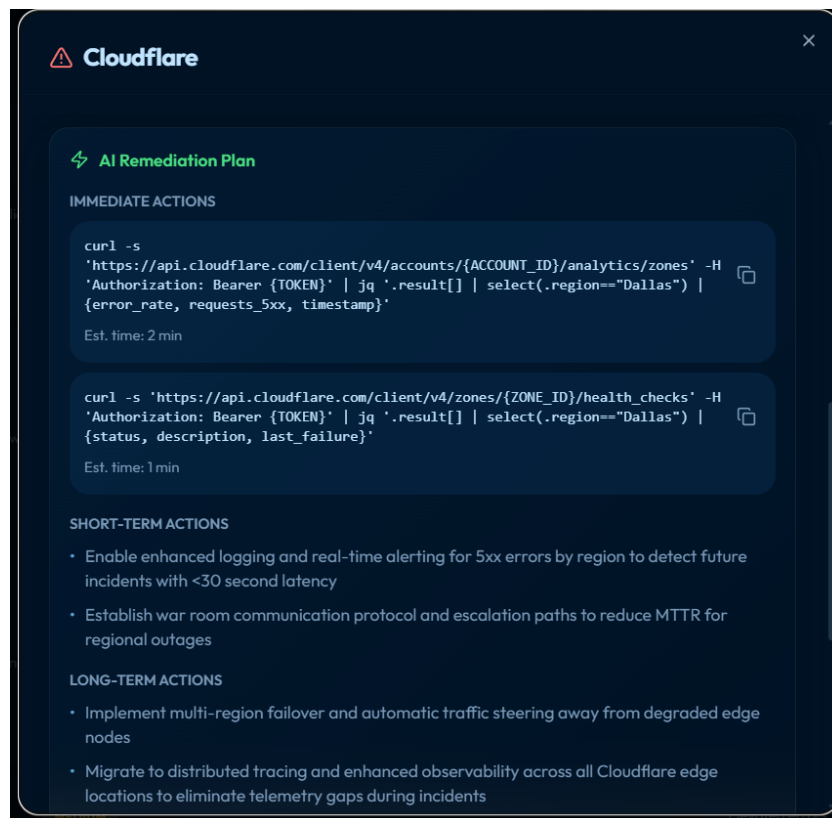


Figure 3: AI remediation plan rendered for a Cloudflare incident, structured into immediate actions with executable commands, short-term actions, and long-term actions.

The `/similar` endpoint backs a sidebar that surfaces past incidents with comparable descriptions, so a reviewer looking at a new incident sees three precedents on the same screen (Figure 4). The `/weekly` endpoint feeds a separate weekly intelligence view that displays the aggregation matrices (hotspots by provider and service, severity counts, week-on-week trend deltas) above the LLM-generated natural-language summary (Figure 5). The `/metrics` endpoint backs a static model-performance panel that exposes the deployed classifier's macro F1, per-class F1, and dataset sizes for operator awareness. The `/health` endpoint is polled in the background and surfaces a degraded-state banner if any required artefact failed to load.

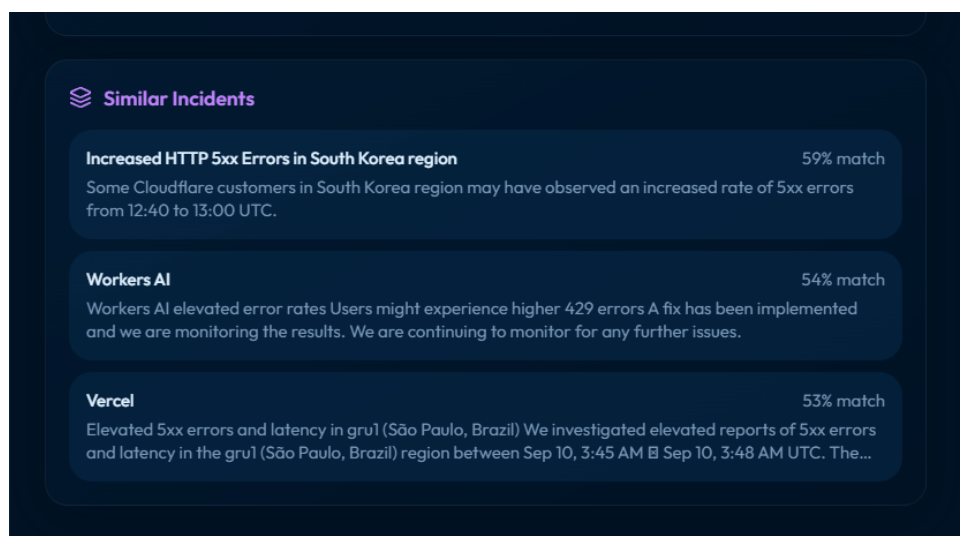


Figure 4: Similar incidents panel showing three past incidents retrieved by MiniLM similarity, with match percentages.

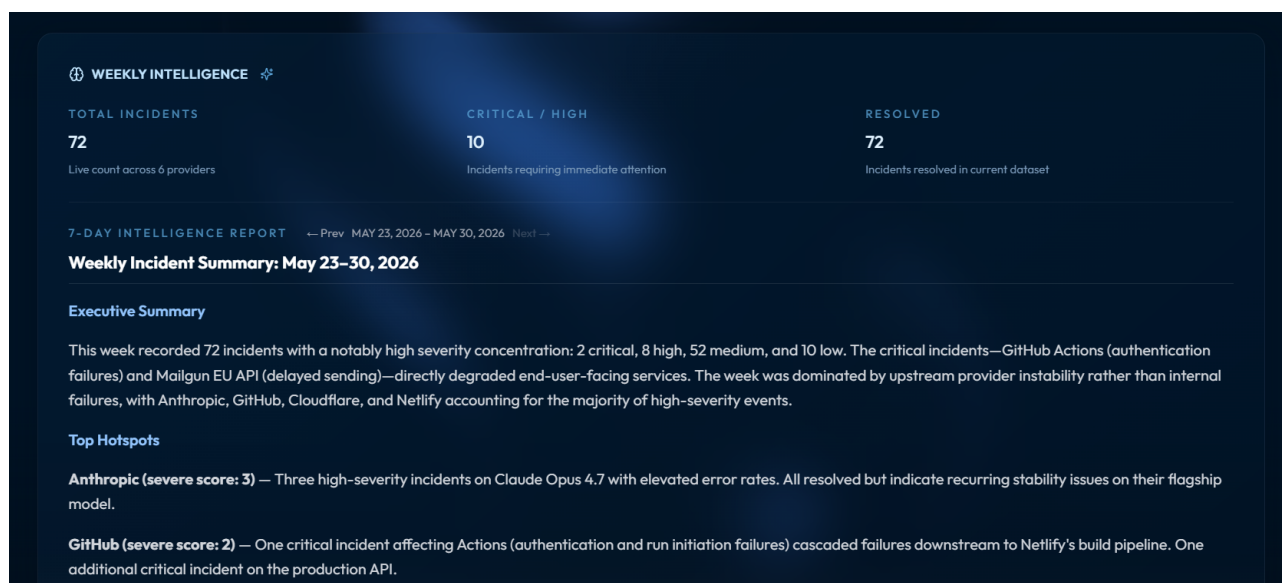


Figure 5: CloudPulse weekly intelligence view showing total incident counts, critical/high breakdown, and the LLM-generated seven-day intelligence report for 23–30 May 2026.

Three design choices in the dashboard are worth highlighting. First, SHAP contributions are rendered with their sign and direction visible (“toward high” or “away from high” for each token), not as unsigned importance scores. This matches the shape of the explanation field returned by the API and gives the reviewer the qualitative information needed to assess whether the classifier’s reasoning is plausible for the current incident. Second, the `escalation_reason` field is rendered as a distinct visual element when present, so an incident that was promoted to critical by the rule is visually separable from an incident that the classifier predicted as critical directly (the latter cannot occur in the deployed architecture, but the visual distinction reinforces the design property). Third, the remediation block is rendered as five collapsible sections rather than as a single text blob, which matches the five-field Pydantic schema returned by the API and allows the reviewer to read each section independently. When the remediation field is null (the documented degradation path described in Section 3.12), the dashboard renders an explicit placeholder rather than silently omitting the section, so the reviewer can distinguish a missing remediation from a successfully generated one.

The dashboard does not perform any inference of its own. All prediction, explanation, anomaly scoring, and remediation generation happen inside the FastAPI service the dashboard is a typed React client of the API contract. This separation of concerns is the same property that allows the upstream pipeline maintained by Madalina Carcea to consume the API identically: any client that conforms to the Pydantic schemas receives the same response shape, and the dashboard is one such client among others. The dashboard’s deployment to the shared cloud environment is handled by the data engineering track, which controls the infrastructure on which both the API and the dashboard run, the dashboard’s design, implementation, and contract with the API belong to the AI engineering track.

3.15 Summary

This chapter has described the deployed CloudPulse AI service, which is a FastAPI application that exposes five endpoints, loads its model artefacts once at startup, shares its preprocessing function with the training pipeline, layers a deterministic escalation rule on top of the classifier output, degrades gracefully when its LLM dependency fails, and is consumed by a React dashboard for human review alongside the upstream data engineering pipeline. The AI engineering track's contribution is the trio of artefacts described in this chapter: the trained models exposed by the service, the FastAPI service itself, and the React dashboard that consumes it. The data engineering track deploys the API and the dashboard to the shared cloud environment, where they run in production (Carcea, 2026). The integration contract between the two tracks is a typed Pydantic schema with permissive optionality on metadata fields, which gives the upstream pipeline room to evolve without breaking the AI service. The two-stage weekly intelligence pass separates deterministic aggregation from LLM-generated language, which keeps the summary grounded in pre-computed counts and hotspots rather than in free-form text. The React dashboard renders the same typed contract for human review, with explicit visual treatment for the escalation reason, the SHAP direction, and the degradation placeholders. Across all components, the design prefers explicit failure modes and observable degradation over silent fallbacks. Two known asymmetries between the trained artifacts and the deployed system, the uncalibrated softmax probabilities and the inference-time substitution in the anomaly feature vector, are highlighted in their respective sections and revisited in detail in Section 6.6.

Figure 6 illustrates how the AI engineering track integrates with the data engineering track through the typed `/classify` endpoint contract.

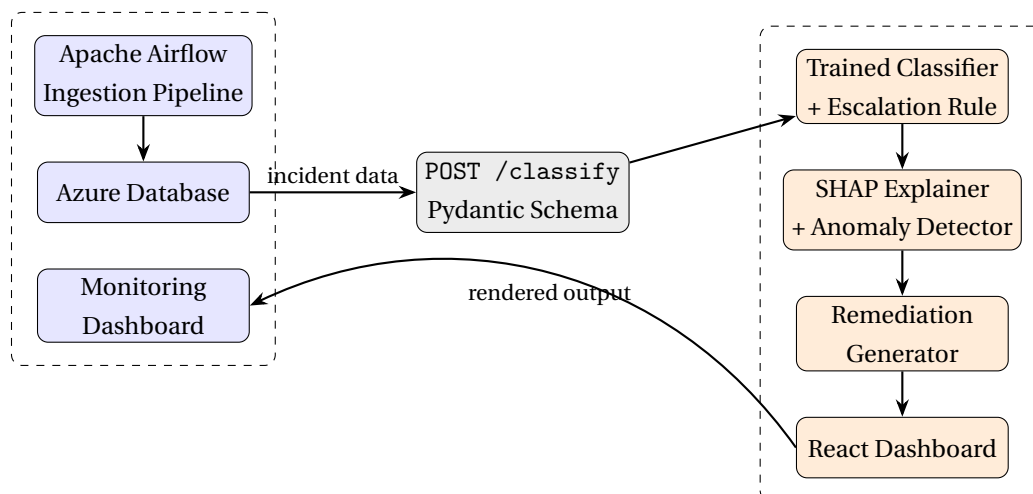


Figure 6: Integration between the data engineering track and the AI engineering track. The typed `POST /classify` Pydantic schema is the single integration boundary between the two tracks.

The next chapter discusses the methodology behind the trained models exposed by this service: feature engineering, model selection, evaluation protocol, and the calibration and explainability work that informs the deployed components described above.

4 Methodology

This chapter describes the empirical setup behind the CloudPulse severity classifier, the remediation generator, and the evaluation harness used to compare them. The structure follows the workflow of the project: the dataset and its preparation, the architectural decision that shaped every downstream model, the four classical learners and two transformer fine-tunes used for comparison, the LLM baselines, the evaluation machinery (cross-validation, statistical testing, calibration, explainability, expert rating, and stress testing), and the data leakage audit performed at the end of the project. Every methodological choice is justified against either the project’s empirical evidence or the prior literature.

4.1 Dataset

The dataset consists of 909 cloud-incident records collected from approximately twenty public StatusPage feeds operated by software-as-a-service providers, including Cloudflare, Twilio, GitHub, Vercel, Zoom, MongoDB, SendGrid, Datadog, OpenAI, Stripe, Discord, Atlassian, Typeform, HubSpot, New Relic, CircleCI, Shopify, DigitalOcean, Linear, Intercom, and Spotify. The three major hyperscalers (AWS, GCP, and Azure) are not providers in this dataset, they appear instead as values of a separate underlying-infrastructure attribute recorded for each incident, indicating the cloud platform on which the provider’s affected service was hosted at the time of the event.

Each record includes a free-text description written by the provider, several structured fields (`original_impact`, `service_impact`, `affected_services_count`), and a severity label (`low`, `medium`, `high`, or `critical`). The data collection and preparation was handled by the collaborating data engineering track, led by Madalina Carcea (Carcea, 2026). This thesis picked up from a cleaned, deduplicated dataset ready for use.

The class distribution is severely imbalanced: 60.6% medium, 23.3% low, 12.6% high, and 3.4% critical. This imbalance is intrinsic to the underlying phenomenon, since most public incidents are minor, rather than an artefact of sampling, and treating it as a known constraint shapes the architectural decisions in Section 4.4.

The dataset was split with stratification by class into 80% training, 10% validation, and 10% test partitions, producing 727 training, 91 validation, and 91 test incidents. Stratified sampling preserves the marginal class distribution across all three partitions (Kohavi, 1995), which is essential when minority classes are small enough that random splitting can shift the proportion meaningfully. The split was generated with a fixed random seed (42) and the same split is used by every model reported in this thesis, so all comparisons are made on identical training, validation, and test sets.

Table 2: Dataset partitions and per-class counts after stratified 80/10/10 split with seed 42.

Partition	Low	Medium	High	Total
Train	169	441	117	727
Validation	21	55	15	91
Test	21	55	15	91
Full dataset	211	551	147	909

4.2 Preprocessing and Anonymisation

Each incident description was lowercased, stripped of non-text artefacts (HTML residue, repeated whitespace), and passed through a single `anonymise()` function that replaces provider names and URLs with the sentinel tokens `PROVIDER` and `URL`. This step has two purposes. The methodological purpose is to prevent the

classifier from learning provider-specific shortcuts, for example, that a particular provider name co-occurs with a particular severity tier because that provider's status page is overrepresented in the corpus. The secondary purpose is privacy-by-design: although the status page feeds contain no personal data, removing provider identifiers reduces the risk of the model reproducing provider-specific phrasings at inference time.

The same `anonymise()` function is called both during training-set preparation and inside the FastAPI `/classify` handler at inference time. Sharing a single implementation eliminates the train-serve skew that would otherwise arise from two parallel preprocessing paths.

4.3 Data Management, Privacy, and Ethics

The dataset was collected exclusively from publicly available StatusPage feeds operated by software-as-a-service and cloud infrastructure providers. All incident records are provider-authored operational communications published for public consumption, no data was obtained from private sources, internal systems, or individual users. The data collection and storage pipeline was handled by the collaborating data engineering track (Carcea, 2026), which ingests incidents into a TimescaleDB hypertable on Azure. The dataset used in this thesis is a research snapshot of 909 deduplicated incidents, it is not continuously updated by the AI engineering track.

Privacy risk in this dataset is low by design. Public StatusPage feeds contain no personally identifiable information, they describe service states, not users. The anonymisation step described in Section 4.2 removes provider names and URLs from incident descriptions, which is consistent with the data minimisation principle under the General Data Protection Regulation (European Parliament & Council of the European Union, 2016), even though the dataset does not contain personal data in the GDPR sense.

The AI engineering components of CloudPulse fall within the scope of the EU Artificial Intelligence Act (European Parliament & Council of the European Union, 2024). The severity classifier produces structured outputs for operational decision support, accompanied by per-prediction SHAP explanations and a model confidence score. The deployed service is designed to support rather than replace human review, which is consistent with the transparency and human oversight requirements of the AI Act. These design properties also align with the accountability and transparency principles set out in the UNESCO Recommendation on the Ethics of Artificial Intelligence (UNESCO, 2021), which states that AI systems should be explainable and subject to human oversight. The SHAP layer, the model confidence score, and the deterministic escalation rule in the deployed service are the concrete implementations of those principles in the CloudPulse architecture.

4.4 Architectural Decision: Three-Class Classifier with a Deterministic Rule

The original problem framing was a four-class classifier directly aligned with the provider tags `low`, `medium`, `high`, and `critical`. An initial four-class XGBoost run produced a macro F1 of 0.42, with the critical class consistently at F1 = 0.00 across folds. The cause is structural: the dataset contains only 31 critical incidents (3.4% of 909), of which 25 fall in the training partition. With this support, no classifier in the comparison was able to learn a usable decision boundary for the critical class. A 5-fold cross-validation experiment yielded a critical-class median F1 of 0.00 (mean 0.13 ± 0.15) across twenty model-by-fold trials, confirming that the failure was not the consequence of a single unlucky split. Cost-sensitive reweighting and other imbalance-handling techniques can mitigate class skew up to a point, but when minority support is in the low tens of examples, the learnability of the minority class itself becomes the limiting factor (He & Garcia, 2009).

The natural next step was to drop the critical class from the supervised target and replace it with a

deterministic rule. The classifier is trained to discriminate `low`, `medium`, and `high` only. At inference time, the final severity is promoted from `high` to `critical` if, and only if, the upstream `original_impact` field equals `"critical"`. The rule is asymmetric, basically it can promote, but it cannot demote, and it operates on a structured metadata signal rather than on the free-text description. This asymmetry is the safety property of the design, since text alone, which can be alarmist for political or public relations reasons, cannot escalate an incident, but an authoritative external signal can.

The empirical ablation that motivated this decision is reported in Chapter 5. The 22-point gap between the two formulations serves as the direct evidence for the architectural pivot.

4.5 Feature Engineering

Two feature families were considered for the classical models: a sparse text representation and a dense embedding representation.

The text representation uses TF-IDF on word n-grams of length 1 and 2, with English stopwords removed, lowercase casing, and a vocabulary capped at 5,000 features (Manning et al., 2008). TF-IDF was chosen because it remains a strong baseline for short-document classification (Galke & Scherp, 2022; Revina et al., 2020) and because its features remain interpretable: every column corresponds to a token or bigram. This interpretability matters for the SHAP-based explainability layer described in Section 4.11.

The dense alternative was sentence embeddings from `all-MiniLM-L6-v2` (Reimers & Gurevych, 2019), a 384-dimensional bi-encoder trained for general sentence similarity. MiniLM embeddings were generated for the full training set and substituted into the same classical pipelines as a comparison. On the validation set, MiniLM-based pipelines did not outperform the TF-IDF baseline within the noise of the small validation split. A two-dimensional PCA projection of the MiniLM space confirms that severity classes overlap substantially in the dense representation, which is consistent with the weak validation performance. MiniLM was therefore retained for the similarity-search component of the system, where dense semantic matching is its native strength, but not used as a classification feature.

A small number of structured metadata fields complement the TF-IDF matrix: `service_impact` (categorical), `affected_services_count` (numeric), and an ordinal encoding of `original_impact` (`none` < `minor` < `major` < `critical`). These fields are populated by the upstream pipeline at incident creation time and are available at inference.

An additional candidate feature was explicitly excluded from the supervised severity classifier: `resolution_time_minutes`. This field is the duration between incident open and incident close, and it correlates strongly with severity. It is also unavailable at classification time, because at the moment an incident first surfaces it has, by definition, not yet been resolved. Including it during training would therefore introduce target leakage: the classifier would learn a feature that cannot exist in production. Such leakage is one of the most commonly cited and one of the most damaging methodological errors in applied machine learning (Kapoor & Narayanan, 2023), and excluding the field is a deliberate design decision rather than an oversight.

The unsupervised anomaly detector described in Section 4.12 is the one exception to this exclusion: it is trained with `resolution_time_minutes` retained in its feature set, because the detector is unsupervised and the field contributes to the training distribution it learns to model. The asymmetry that this creates between the trained anomaly distribution and the served anomaly scores at inference time is a deliberate design property of the deployed system rather than an inconsistency, and the inference-time consequences are documented in the system design chapter (Section 3.8) and revisited in Section ??.

4.6 Classical Machine-Learning Models

Four classical models form the supervised baseline: Logistic Regression, Linear Support Vector Machine (SVM), Random Forest, and Extreme Gradient Boosting (XGBoost). All four share the same TF-IDF and meta-data feature matrix, the same train/val/test partitions, and all are trained with `class_weight="balanced"`, which reweights the loss inversely to class frequency and is the standard scikit-learn implementation of cost-sensitive learning for imbalanced data (He & Garcia, 2009). Hyperparameters were tuned per task (the 3-class severity classifier and the binary severe-vs-not-severe classifier were trained independently) by grid search with validation macro F1 as the selection criterion. Table 3 summarises the final hyperparameter configuration for each model and task.

Table 3: Hyperparameter configurations for the four classical models, tuned independently for the 3-class and binary tasks.

Model	Task	Kernel / Base	Key hyperparameters	Values
Logistic Regression	3-class	L2	solver, max_iter, C	liblinear, 1000, 5.0
	Binary	L2	solver, max_iter, C	liblinear, 1000, 0.5
Linear SVM (Joachims, 1998)	3-class	linear	C	1.0
	Binary	linear	C	0.5
Random Forest (Breiman, 2001)	Both	bagging	n_estimators, max_depth, min_samples_split	200, None, 2
XGBoost (T. Chen & Guestrin, 2016)	Both	boosting	n_estimators, max_depth, learning_rate, weights	300, 8, 0.05, inverse class freq.

Logistic Regression uses stronger regularisation on the binary task ($C = 0.5$ vs $C = 5.0$) because the 3-class model has more output structure to separate. Linear SVM is included as a classical text-classification baseline known to be strong on sparse high-dimensional TF-IDF features. Random Forest is a bagging baseline for variance reduction on small datasets. XGBoost was selected for the production classifier because gradient-boosted trees handle sparse text features well and because the TreeExplainer formulation of SHAP is exact and efficient for this model family (Section 4.11).

The primary reported metric is macro F1, which weights all three classes equally and therefore penalises models that collapse to the majority class (Opitz, 2024b; Sokolova & Lapalme, 2009). The test partition was opened exactly once, at the end of the comparison, after all hyperparameter selection on the validation partition was complete.

4.7 Transformer Fine-Tunes

Two pretrained transformer models were fine-tuned on the same three-class task for direct comparison with the classical models: DistilBERT-base-uncased (Sanh et al., 2020) and RoBERTa-base (Y. Liu et al., 2019).

DistilBERT has 66 million parameters. It was fine-tuned for 6 epochs with batch size 16, learning rate $2e-5$, warmup ratio 0.1, weight decay 0.01, mixed-precision (fp16) training, maximum sequence length 256, early stopping with patience 2 on validation macro F1, and seed 42. The optimiser was AdamW with linear learning-rate decay. Training runtime on the pinned GPU was 18.4 seconds.

RoBERTa-base has 125 million parameters. It was fine-tuned for 6 epochs with batch size 8, smaller than DistilBERT to fit GPU memory, learning rate $1e-5$, warmup ratio 0.1, weight decay 0.01, mixed-precision (fp16), and seed 42. Training runtime on the pinned GPU was 45.8 seconds, roughly 2.5 times longer than DistilBERT despite the same number of epochs, reflecting the larger parameter count.

Both fine-tunings ran on a single NVIDIA RTX 6000 Ada GPU under CUDA 12.8, pinned via `CUDA_VISIBLE_DEVICES = 2` on the university server. The GPU node hosts a heterogeneous mix of devices (RTX 6000 Ada, L40s, A6000), and pinning to a single device was necessary because heterogeneous multi-GPU execution broke gradient consistency under PyTorch's `DataParallel`. The hardware was shared with other students and access windows were intermittent across the semester, which is the reason the classical pipeline was developed first and the transformer experiments were sequenced last.

The choice of these two specific architectures is justified empirically and pragmatically. DistilBERT offers a smaller, faster baseline that retains roughly 97% of BERT's GLUE performance at 40% of the parameter count (Sanh et al., 2020), making it a reasonable choice when training data is small. RoBERTa-base is a stronger pretrained checkpoint than the original BERT (Devlin et al., 2019; Y. Liu et al., 2019) and is included to test whether the additional parameters and improved pretraining recipe translate to better fine-tuning performance on the CloudPulse corpus. A central research question of this thesis is whether transformer fine-tuning is justified at this dataset size, and the small-data behaviour of transformers has been documented as unstable and prone to overfitting in sub-thousand-example regimes (Mosbach et al., 2021). The empirical answer for CloudPulse is reported in Chapter 5.

4.8 Large Language Model Prompting Baselines

To test whether a general-purpose large language model can match or exceed the supervised classifiers, Claude Haiku 4.5 was prompted to classify each test-set incident under three distinct strategies.

Zero-shot prompts the model with a fixed system prompt describing the three classes and asks for a single-label output. No examples are provided.

Few-shot prepends three labelled exemplars (one per class) to the prompt before the test incident (Brown et al., 2020). The exemplars are drawn from the training partition and held constant across all test calls.

Chain-of-thought (CoT) (Wei et al., 2022) instructs the model to reason about severity factors (user impact, scope, urgency cues) before emitting a label, with the final label parsed from a structured tag. The three strategies follow the taxonomy of prompt engineering techniques surveyed by Sahoo et al. (2024).

Claude Haiku 4.5 was chosen as the LLM baseline for three reasons. First, it is the same model family used by the remediation generator inside the production system, so the comparison measures the actual deployed LLM rather than a different model purchased for the comparison. Second, Haiku occupies the price-and-latency tier appropriate for per-incident classification at production scale, using a larger, more expensive model would not reflect a realistic operational choice. Third, the prompting comparison isolates strategy effects rather than model effects, holding the model constant.

Few-shot prompting in the small-example regime is known to be sensitive to the choice and order of exemplars (Lu et al., 2022). To bound this instability, the three exemplars were fixed across all calls and the temperature was set to 0.0 to remove sampling variance. The point of the comparison is not to find the best prompt, it is to estimate the ceiling of off-the-shelf LLM classification under three commonly used strategies.

4.9 Cross-Validation

Single-split test numbers on a 91-incident test set carry meaningful uncertainty: with only 15 high-class examples, a single prediction flip is approximately 6.7% macro F1. To bound this uncertainty, 5-fold stratified cross-validation was added on the combined train and validation partition ($n = 818$) (Kohavi, 1995). Each fold preserves the class proportions of the full dataset, and the same random seed is used across all models so that fold membership is identical. Mean and standard deviation of macro F1 across folds are reported alongside the single-split test number for the four classical models.

The cross-validation experiment is treated as a sanity check on the single-split test result, not as a replacement for it. The headline number for each model is the test-set macro F1 on the held-out 91 incidents, with the cross-validation mean and standard deviation reported as evidence that the headline number falls within sampling noise of a more robust estimate.

4.10 Statistical Testing with McNemar's Test

To test whether observed differences between classifiers on the test set are statistically meaningful, pairwise McNemar tests are used (Dietterich, 1998; McNemar, 1947). McNemar's test is the standard non-parametric paired test for comparing two classifiers on the same test set: it tabulates the cases where one classifier is correct and the other is wrong (and vice versa) and tests whether the discordant counts differ from chance. It is preferred over a paired t-test on accuracy because it does not assume independence of the per-example predictions and because it is appropriate for binary correct-or-incorrect outcomes (Dietterich, 1998).

McNemar's test is applied to the binary severe-vs-not-severe task (mapping high to severe and the rest to not-severe) for the within-classical pairs of classifiers, and separately to the comparison between the best classical classifier and the best LLM strategy. The interpretation of the resulting p-values is reported in Chapter 5, the methodological point here is that the small test set is a known limitation and the test is used to ground claims of "better" or "no significant difference" rather than to anchor effect sizes.

4.11 Calibration and Explainability

The XGBoost classifier emits a probability per class, but tree ensembles are well-known to produce mis-calibrated probabilities, typically pushed away from the extremes (Niculescu-Mizil & Caruana, 2005). To make the confidence score behave approximately as a probability rather than as an arbitrary monotonic score, sigmoid calibration (Platt scaling) was applied using scikit-learn's `CalibratedClassifierCV` with `cv="prefit"` and `method="sigmoid"`, fitted on the validation partition so that the test partition remains untouched (Platt, 1999). Calibration quality is reported using the Brier score (Brier, 1950), a proper scoring rule that combines calibration and refinement into a single number, per-class Brier scores before and after calibration are reported in Chapter 5.

The calibrated wrapper was developed and evaluated offline only. Deployment testing during system integration showed that wiring the calibrated wrapper into the FastAPI `/classify` endpoint degraded the test-set macro F1, so the deployed service serves uncalibrated softmax probabilities and the calibrated artefact is preserved on disk for future refinement. This deployment-time gap is treated in detail in Section 6.6.

Per-prediction explainability is provided by SHAP values (Lundberg & Lee, 2017). For the tree-based XGBoost classifier, the `TreeExplainer` formulation (Lundberg et al., 2020) produces exact, polynomial-time SHAP values for each prediction, returning a signed contribution per input feature toward each output class. The `TreeExplainer` is serialised as a joblib artefact and loaded by the FastAPI service at startup. The `/classify` endpoint returns the top five contributions for the predicted class, exposing both the TF-IDF token weight and the signed SHAP value so that a downstream consumer can see which tokens drove the decision. Global feature importance is computed by averaging the absolute SHAP values across the test set per class, and is reported in Chapter 5.

4.12 Anomaly Detection

An Isolation Forest (F. T. Liu et al., 2008) was trained on the structured metadata features of the training partition to flag incidents that fall outside the typical metadata distribution. Isolation Forest isolates anomalous points through recursive random partitioning, with the expected path length acting as an anomaly score. The model is fitted once at training time, persisted as a joblib artefact, and queried by the `/classify` endpoint, which returns both a Boolean `is_anomaly` flag and the continuous decision-function score. The anomaly layer is intended as a complementary signal for downstream consumers, not as a classifier in its own right.

The detector is trained on eight structured features: description length, resolution time in minutes, affected services count, and five one-hot encoded service-impact categories. The retention of `resolution_time_minutes` in this unsupervised setting (despite its exclusion from the supervised classifier in Section 4.5) is deliberate: the detector's goal is to model the joint distribution of historical metadata, not to predict severity, and excluding the field would degrade that distribution. The inference-time consequences of this asymmetry are documented in the system design chapter (Section 3.8) and revisited as a limitation in Section 6.6.

4.13 Likert Evaluation of Generated Remediations

The remediation generator is the language-generation component of CloudPulse, producing a structured JSON object with five fields: summary, root-cause hypothesis, immediate mitigation, investigation, and prevention. Unlike severity classification, this output has no ground-truth answer and therefore cannot be evaluated with supervised metrics.

A structured Likert-style expert evaluation was designed instead, following the original Likert formulation and current human-rating practice for open-ended natural-language generation outputs (Amidei et al.,

2019). A random sample of 20 remediations was drawn from the 91 test-set generations using seed 42, naturally weighted toward the majority class (13 medium, 5 high, 2 low). Each remediation was scored on five dimensions on a 1–5 integer scale: specificity, correctness, completeness, actionability, and prioritisation (Table 4). The dimensions and their anchors were defined a priori, before any score was assigned, to prevent post-hoc rubric drift.

Table 4: Likert rubric for the remediation evaluation. Each remediation was scored on five dimensions on a 1–5 scale, with all anchors defined a priori.

Dimension	1	2	3	4	5
Specificity	Generic SRE advice applicable to any service	Mentions service category but no specific component	Names the provider but not the affected component	Names the provider and component; description partially used	Names specific technology, vendor, or component from the description
Correctness	Proposed root cause is implausible or wrong	Root cause is plausible but unlikely given the description	Root cause is reasonable but not the most likely explanation	Root cause is likely correct with minor gaps	Root-cause hypothesis aligns with what a senior SRE would propose
Completeness	Missing mitigation, investigation, or prevention	Two of three buckets present; one missing entirely	All three buckets present but one is superficial	All three buckets present with adequate content	All three buckets covered with substantive content
Actionability	Steps require significant clarification before execution	Steps are directionally correct but missing key details	Steps are executable with minor clarification	Steps are clear and executable with no clarification	On-call engineer could execute immediately
Prioritisation	Steps in confused order; verification before fix	Ordering partially correct; one bucket misplaced	Sensible ordering with minor sequencing issues	Correct ordering with one minor deviation	Sensible ordering: verify, mitigate, investigate, prevent

A composite acceptance threshold of ≥ 3.50 out of 5.00 (the rubric midpoint) was set a priori. Scoring was performed by the AI engineering author of the thesis. The full per-incident scoring is preserved as a project artefact and the aggregate is reported in Chapter 5. Single-rater evaluation is a known limitation of the design and is discussed honestly in Section 6.6.

4.14 Stress Testing

Two complementary stress tests cover the deployed FastAPI service. The first is a manual six-case stress test documented in the project repository three sanity-check cases (one per severity class), one adversarial case to test whether the escalation rule can be fooled by alarmist text with `original_impact = "none"`, one real critical case to test that the rule fires correctly, and one weekly-summary case to exercise the `/weekly` endpoint.

The second is an automated regression-quality test which replays the entire 91-incident test set through the deployed API and compares the API-returned predictions against the offline model output. This test detects train-serve skew: any difference between the offline test-set macro F1 and the API-replayed macro F1 would indicate a preprocessing or model-loading mismatch. The automated stress test result is reported in Chapter 5.

4.15 Reproducibility and Source of Truth

All experiments use Python 3.13 with scikit-learn, XGBoost, sentence-transformers (all-MiniLM-L6-v2), Transformers and Accelerate (for the DistilBERT and RoBERTa fine-tunes), SHAP, and statsmodels (for McNemar's test). Random seeds are fixed at 42 throughout: dataset split, classical-model training, transformer fine-tuning, cross-validation fold assignment, and Likert sampling. The FastAPI service is served with uvicorn and depends only on artefacts produced by the training pipeline. The transformer fine-tunes run on a single NVIDIA RTX 6000 Ada GPU under CUDA 12.8, pinned via `CUDA_VISIBLE_DEVICES=2`. All code and trained artefacts are maintained in the CloudPulse Git repository on the `feature/ai-retrain-v2` branch. The test set was opened exactly once after model selection on the validation partition was finalised, and the same test partition is used for every model reported in Chapter 5.

A note on the canonical metrics: the per-model JSON artifacts (`xgb_3class_results.json`, `rf_3class_results.json`, `logreg_3class_results.json`, `svm_3class_results.json`, `results_distilbert.json`, `results_roberta.json`, `llm_results.json`) are the canonical source for all quantitative results in Chapter 5. A consolidated `results_consolidated.csv` file exists for convenience, but the per-class JSON files are the authoritative source, and the weighted F1 numbers reported in Chapter 5 are recomputed from those JSON files as $\sum_c F_{1,c} \cdot n_c / N$.

4.16 Data Leakage Audit

A retrospective audit of the train, validation, and test partitions was performed to verify the absence of data leakage (`leakage_audit.py`). The audit checked five properties: zero incident-ID overlap across train, validation, and test partitions, the TF-IDF vectoriser was fit on training data only and applied to validation and test through the same fitted transformer, the sparse feature matrices and label arrays on disk match the row counts of the source CSVs, the master CSV is deduplicated at the incident-ID level, and the stratified split preserves the class proportions across all three partitions.

All five properties pass. There is zero incident-ID overlap across train and validation, across train and test, and across validation and test. The TF-IDF dimensionality is consistent at 5,000 features across all three partitions, confirming that the vectoriser was fit on the training partition and the validation and test partitions were transformed by the same fitted object rather than refit independently. The on-disk row counts match exactly: 727 rows in the train feature matrix and label arrays, 91 in validation, 91 in test. The full collection of 909 incidents contains zero duplicate incident identifiers. The class distributions reported in Table 2 match the standard specification of the dataset.

The audit also identified two test-set incidents whose description text appears identically in the training partition because of templated provider messages reused across distinct incidents. Both pairs carry consistent severity labels (a HubSpot “may be unavailable for some users” templated message labelled low in both halves, and a generic post-resolution message labelled medium in both halves). This is a property of the source data rather than a methodology error, since the splitting procedure correctly treats incidents with different identifiers as distinct, and the upstream StatusPage feeds publish templated text that gets reused across separate real incidents. The labelling-versus-text consequence of this property is discussed in detail in Section 6.6.

4.17 Summary

This chapter has described every methodological choice that affects the empirical results in Chapter 5: the dataset and its stratified split, the preprocessing pipeline shared between training and inference, the architectural pivot from four-class to three-class plus a deterministic rule, the four classical models and two transformer fine-tunes with their full hyperparameters, the three LLM prompting strategies, the evaluation harness comprising cross-validation, McNemar testing, calibration, SHAP explainability, anomaly detection, Likert remediation rating, and stress testing, and the data leakage audit performed after all training and evaluation was complete. Each choice has been justified against either the empirical evidence from the project or the prior literature.

The next chapter reports the results these methods produce, beginning with the cross-model leaderboard, proceeding through the per-class breakdown for the production classifier, the cross-validation sanity check, the McNemar pairwise tests, the binary task, the two transformer fine-tunes, the LLM prompting baselines, the calibration analysis, the error taxonomy, the SHAP-based feature attribution, the Likert evaluation of the remediation generator, and the automated stress-test reproduction.

5 Results

This chapter explains the empirical results produced by the methods described in Chapter 4. The main finding is that the gradient-boosted classifier trained on TF-IDF features outperforms both the fine-tuned transformers and every Claude Haiku prompting strategy on the same held-out test set, while the high-severity class remains the bottleneck for every model in the comparison. The chapter proceeds through the cross-model leaderboard, the per-class breakdown for the production classifier, the cross-validation sanity check, the McNemar pairwise tests, the binary task, the two transformer fine-tunes, the LLM prompting baselines, the calibration analysis, the error taxonomy, the SHAP-based feature attribution, the Likert evaluation of the remediation generator, and the automated stress-test reproduction.

5.1 Cross-Model Leaderboard

Nine models were trained and evaluated on the identical 91-incident test set: four classical learners (XGBoost, Random Forest, Logistic Regression, Linear SVM), two transformer fine-tunes (DistilBERT-base and RoBERTa-base), and three Claude Haiku prompting strategies (zero-shot, few-shot, chain-of-thought). The macro F1, weighted F1, and accuracy scores are reported in Table 5 and visualised in Figure 7. The final column reports whether the model is significantly different from the production XGBoost classifier under a pairwise McNemar test on the binary severe-vs-not-severe task, anticipating the test reported in Section 5.4.

Table 5: Test-set results across all nine models on the 3-class severity-classification task ($n = 91$). Macro F1 is the primary metric. The final column reports whether the model differs from XGBoost under a pairwise McNemar test on the binary severe-vs-not-severe task at $\alpha = 0.05$.

Model	Family	Macro F1	Weighted F1	Accuracy	Diff. vs XGB
XGBoost	Classical	0.637	0.698	0.703	—
Random Forest	Classical	0.620	0.707	0.725	—
DistilBERT-base	Transformer	0.617	0.700	0.681	—
Logistic Regression	Classical	0.594	0.671	0.670	—
RoBERTa-base	Transformer	0.565	0.700	0.692	—
Linear SVM	Classical	0.538	0.649	0.681	—
Haiku (CoT)	LLM	0.240	0.435	0.549	✓
Haiku (zero-shot)	LLM	0.230	0.289	0.275	✓
Haiku (few-shot)	LLM	0.210	0.273	0.275	✓

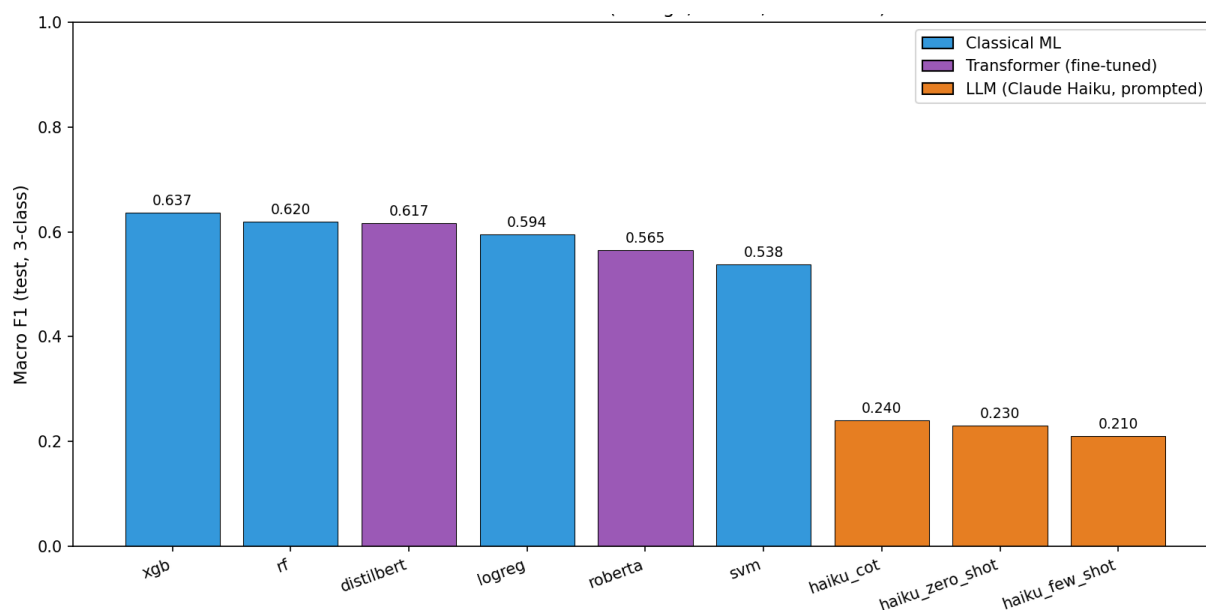


Figure 7: Test-set macro F1 across all nine models, grouped by family (classical, transformer, LLM). The four supervised classical classifiers cluster between 0.538 and 0.637 macro F1, the two fine-tuned transformers fall inside that cluster, the three Claude Haiku strategies cluster near 0.22.

Macro F1 is the primary metric throughout this chapter, accuracy is reported as a sanity check rather than as a comparator. The reason is that on this imbalanced corpus, accuracy can be raised mechanically by over-predicting the majority class, and macro F1 is the standard remedy (Opitz, 2024a; Sokolova & Lapalme, 2009). The contrast between Random Forest and XGBoost is the cleanest illustration: Random Forest reports the highest 3-class test accuracy (0.725) but the second-highest macro F1 (0.620), while XGBoost reports lower accuracy (0.703) but the highest macro F1 (0.637). Random Forest buys its accuracy advantage by over-predicting medium (correctly catching 49 of 55 medium incidents) at the cost of high-class recall (4 of 15 correct), and macro F1 surfaces that weakness while accuracy hides it (Harbecke et al., 2022). This is precisely what a primary metric on imbalanced data is for.

Three findings emerge from the leaderboard. First, XGBoost beats the strongest transformer (DistilBERT) by 0.020 macro F1, a small absolute gap that is nevertheless consistent with the project’s central research question. Second, RoBERTa-base, the larger transformer, underperforms the smaller DistilBERT by 0.052 macro F1, an inverse capacity-performance ordering that is discussed in Section 5.6. Third, the best LLM prompting strategy (chain-of-thought) reaches a macro F1 of 0.240, which is 2.65 times smaller than the production classifier and is best understood as a near-floor result driven by class collapse, as shown in Section 5.7.

The headline gap of 0.020 between XGBoost and DistilBERT is small in absolute terms and falls within the typical noise band of a 91-incident test set. The cross-validation analysis in Section 5.3 bounds this uncertainty, and the McNemar testing in Section 5.4 confirms that within-classical differences are not statistically significant. The defensible claim is therefore not that XGBoost is the best classifier but that classical machine learning is competitive with transformer fine-tuning at this dataset size, and that both are vastly superior to off-the-shelf LLM prompting. The leaderboard is a single-split test result, and within-family rankings are not robust to small sampling shifts, the cross-validation analysis in Section 5.3 is the more reliable evidence on relative classifier quality.

5.2 Per-Class Breakdown for the Production XGBoost Classifier

The macro F1 headline obscures large variation across classes. Table 6 reports the per-class precision, recall, and F1 for the production XGBoost classifier on the test set, and Figure 8 shows the corresponding confusion matrix.

Table 6: Per-class precision, recall, F1, and support for the production XGBoost classifier on the test set.

Class	Precision	Recall	F1	Support
Low	0.682	0.714	0.698	21
Medium	0.754	0.782	0.768	55
High	0.500	0.400	0.444	15
Macro avg	0.645	0.632	0.637	91

The per-class breakdown reveals a consistent ordering: medium-severity incidents are classified most reliably (F1 0.768), followed by low (F1 0.698), with high-severity incidents as the clear bottleneck (F1 0.444). This ordering is not surprising given the class distribution in the training set, where medium accounts for the majority of examples and high is the smallest non-critical class. However, the high-class result deserves closer inspection. The recall of 0.400 means that 9 of the 15 high-severity test incidents are misclassified, and the confusion matrix in Figure 8 confirms that all 9 are predicted as medium rather than low. This is an operationally significant failure mode: the classifier systematically under-escalates incidents that require urgent attention. The precision of 0.500 indicates that half of what the classifier labels as high is correct, meaning the model is also over-predicting high on some medium incidents. Together, these two patterns suggest the classifier struggles to locate the high-severity decision boundary on both sides. The deterministic escalation rule described in Section 3.2 partially addresses this by upgrading classifier predictions to critical when the provider’s own impact tag is major or critical, but high-class recall on the classifier itself remains the primary known limitation of the deployed system.

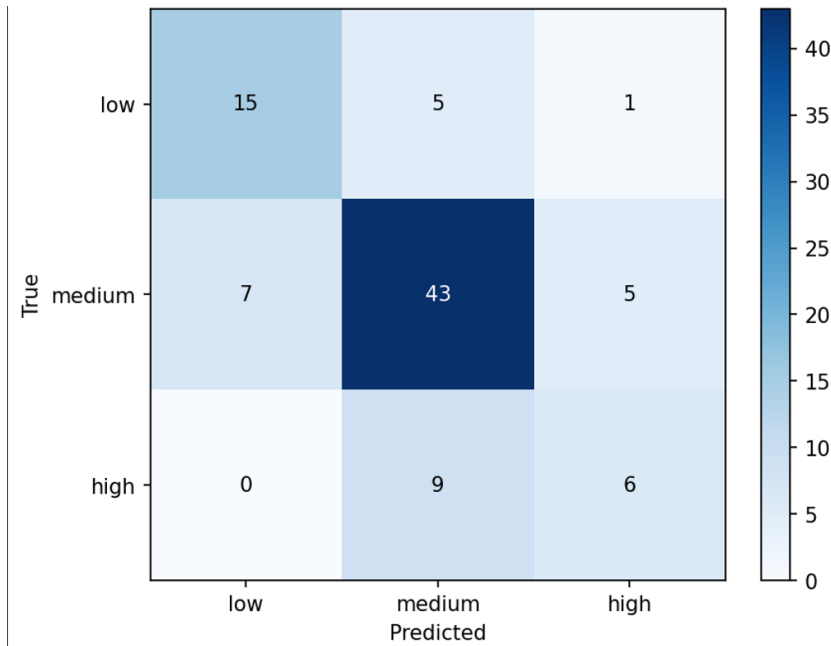


Figure 8: XGBoost 3-class confusion matrix on the test set. The dominant error mode is high-class incidents predicted as medium (9 of 15 high-severity incidents). Source: `xgb_3class_confusion_matrix.png`.

The pattern is consistent across every supervised model in the comparison: the medium class performs best, the low class performs respectably, and the high class is the bottleneck. The full confusion matrix reads as follows: of 15 high-severity test incidents, the classifier predicts 6 correctly, 9 as medium, and 0 as low. Of 55 medium incidents, the classifier predicts 43 correctly, 5 as high, and 7 as low. Of 21 low incidents, the classifier predicts 15 correctly, 5 as medium, and 1 as high. Almost every error is a downgrade: high-to-medium (9), medium-to-low (7), low-to-medium (5). Only one prediction is an upgrade (low-to-high). For an incident classifier, downgrade errors are the operationally worst direction, because they cause under-triage of severe incidents. Figure 9 confirms that the high-class bottleneck holds across the four classical learners and the two transformers.

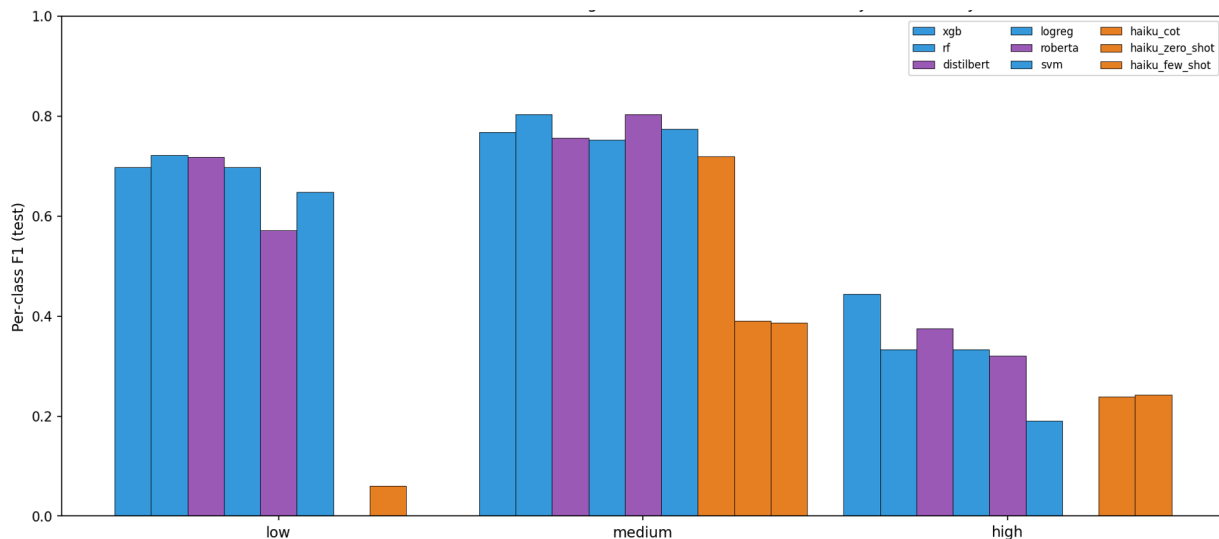


Figure 9: Per-class F1 across the four classical classifiers, the two transformer fine-tunes, and the three Claude Haiku prompting strategies. The high class is the bottleneck for every model family, with the supervised models clustering around F1 = 0.32–0.44 and the LLM strategies failing to produce a usable score on either the low or the high class.

The high-class F1 of 0.444 for XGBoost reflects 6 correct predictions out of 15 high-severity test incidents, with the remaining 9 misclassified as medium (Section 5.9). The high-class precision of 0.500 indicates that when the classifier predicts high, it is correct only half the time. This is the dominant limitation of the supervised system and is the failure mode the remediation generator and the calibration layer must compensate for in the production setting. With only 15 high-class incidents in the test partition, one additional correct prediction shifts F1 by approximately seven percentage points, so the headline high-class F1 is a point estimate with substantial sampling noise.

5.3 Cross-Validation Results

To bound the uncertainty in the single-split test numbers, 5-fold stratified cross-validation was performed on the combined train-plus-validation partition ($n = 818$). Tables 7 and 8 report the mean and standard deviation of macro F1 across folds for the 3-class and binary tasks respectively.

The cross-validation results reveal a methodologically important detail: Logistic Regression has the highest CV mean (0.664) while XGBoost has the highest test-set macro F1 (0.637). The CV mean and the test number are not the same quantity. The CV mean estimates the expected macro F1 averaged over different 818-incident training subsets, while the test number measures performance on a single held-out 91-incident sample drawn from the same distribution. XGBoost’s test-set macro F1 of 0.637 falls within one standard deviation of its own CV mean (0.622 ± 0.045), which is the expected behaviour and the criterion used to judge that the test number is not anomalous. Logistic Regression by contrast drops from CV mean 0.664 to

Table 7: 5-fold cross-validation results on the combined train-plus-validation partition for the 3-class task. Mean and standard deviation of macro F1 across folds.

Model	Mean macro F1	Std
Logistic Regression	0.664	0.034
Random Forest	0.663	0.045
Linear SVM	0.660	0.031
XGBoost	0.622	0.045

Table 8: 5-fold cross-validation results on the binary severe-vs-not-severe task. Mean and standard deviation of macro F1 across folds.

Model	Mean macro F1	Std
Logistic Regression	0.682	0.050
Linear SVM	0.682	0.064
Random Forest	0.655	0.046
XGBoost	0.632	0.054

test 0.594, a 0.070 gap that is larger than its own CV standard deviation of 0.034. This is the model whose CV performance is least confirmed by the test partition, which is also a signal of higher variance, and which works against rather than for the case for deploying Logistic Regression.

The decision to deploy XGBoost rather than Logistic Regression rests on three reasons. First, the absolute gap between the four classical classifiers on CV (0.622–0.664) is comparable to the standard deviation of each (0.031–0.045), meaning no classical model is statistically distinguishable from the others. Second, the test-set result, which is the canonical comparison in this thesis, places XGBoost above the rest. Third, XGBoost was selected for the production pipeline before the cross-validation experiment was run, on the grounds of better integration with the SHAP TreeExplainer, and the cross-validation evidence does not contradict that choice. Five fold-level macro F1 numbers per model produce standard deviations that are themselves estimates with their own sampling noise, and the differences between the four classical models should not be over-interpreted.

5.4 McNemar Pairwise Testing

McNemar’s test was applied to the binary severe-vs-not-severe task for all pairs of classical classifiers, and to the pairing of the best classical classifier with the best LLM strategy. The lowest pairwise p-value among the within-classical comparisons is 0.21, and no within-classical comparison reaches the conventional 0.05 threshold. The direct comparison between XGBoost and Claude Haiku chain-of-thought on the binary severe-vs-not-severe task yields $p < 0.05$, confirming that the family-level gap between classical machine learning and LLM prompting is statistically meaningful. This is the basis for the significance column in Table 5.

The methodological reading of these tests is that the within-classical ranking of XGBoost, Random Forest, Logistic Regression, and Linear SVM cannot be statistically distinguished on a 91-incident test set, which is the expected behaviour for tight comparisons on a small sample. The between-family claim, that classical machine learning outperforms off-the-shelf LLM prompting on this task, is supported by both the magnitude of the gap and the McNemar evidence. McNemar’s test is appropriate for small paired samples but loses power quickly below $n \sim 100$, so the within-classical p-values should be read as “no significant difference detected” rather than “the classifiers are equivalent.”

5.5 Binary Severe-vs-Not-Severe Task

In addition to the 3-class severity task that anchors this thesis, a binary severe-vs-not-severe variant was trained for each classical model. The binary task collapses high and critical into a single severe class and low and medium into not_severe, producing 15 severe and 76 not_severe test incidents. The binary task is the basis for the McNemar pairwise testing (Section 5.4) and is reported here for completeness. Table 9 reports the test-set results across the four classical classifiers.

Table 9: Binary task test-set results across the classical classifiers. Severe-class F1 is reported separately because the macro F1 averages it with the much easier not_severe class.

Model	Macro F1	Severe F1	Severe Precision	Severe Recall
XGBoost	0.662	0.429	0.462	0.400
Logistic Regression	0.644	0.432	0.364	0.533
Linear SVM	0.596	0.286	0.500	0.200
Random Forest	0.551	0.200	0.400	0.133

XGBoost leads on macro F1 (0.662). Logistic Regression is close behind (0.644) but has the highest severe-class recall (0.533, recovering 8 of 15 severe test incidents) at the cost of low precision (0.364). SVM and Random Forest both collapse on the severe class, with severe-class F1 of 0.286 and 0.200 respectively. Random Forest in particular predicts severe on only 5 of 91 test incidents, of which 2 are correct, indicating that the bagging-with-balanced-weights configuration is not aggressive enough to lift the minority class on this corpus.

5.6 Transformer Fine-Tunes

The two transformer fine-tunes use the same train/val/test split and the same evaluation metric as the classical models. DistilBERT-base (66M parameters) reaches a test macro F1 of 0.617 with test accuracy 0.681. RoBERTa-base (125M parameters) reaches a test macro F1 of 0.565 with test accuracy 0.692. The inverse capacity-performance ordering on macro F1 is the central transformer finding of this thesis.

Table 10 compares the per-class precision, recall, and F1 of the two transformers. Both share the supervised-model pattern of strong medium-class performance and weak high-class performance.

Table 10: Per-class precision, recall, and F1 for the two transformer fine-tunes on the test set.

Model	Class	Precision	Recall	F1	Support
DistilBERT	High	0.353	0.400	0.375	15
DistilBERT	Low	0.778	0.667	0.718	21
DistilBERT	Medium	0.750	0.764	0.757	55
RoBERTa	High	0.400	0.267	0.320	15
RoBERTa	Low	0.714	0.476	0.571	21
RoBERTa	Medium	0.731	0.891	0.803	55

The DistilBERT confusion matrix in Figure 10 shows the same dominant error pattern as the classical models: high-class incidents collapse into the medium prediction (8 of 15 high incidents predicted as medium). The RoBERTa confusion matrix in Figure 11 shows the collapse even more sharply: 10 of 15 high incidents are predicted as medium, and only 4 are correctly classified.

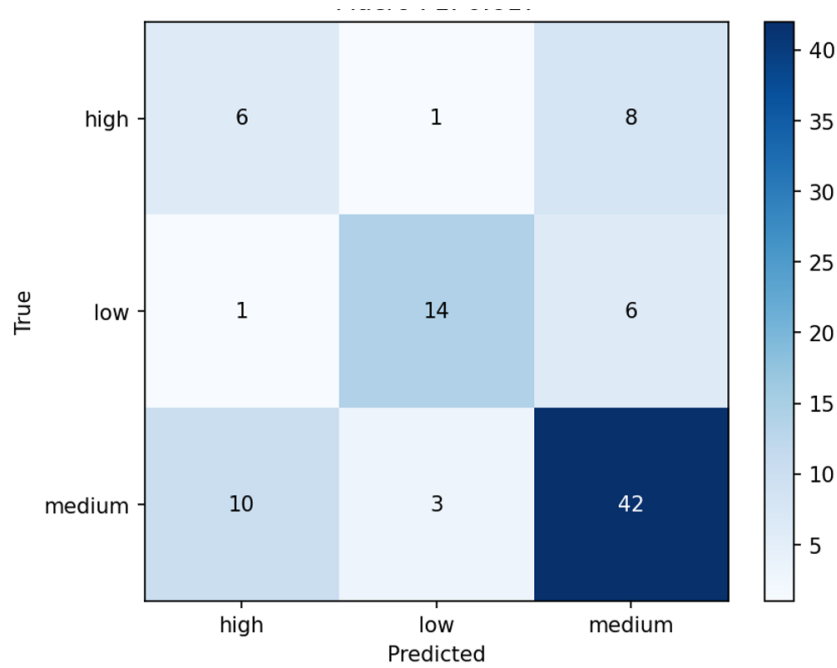


Figure 10: DistilBERT 3-class confusion matrix on the test set.

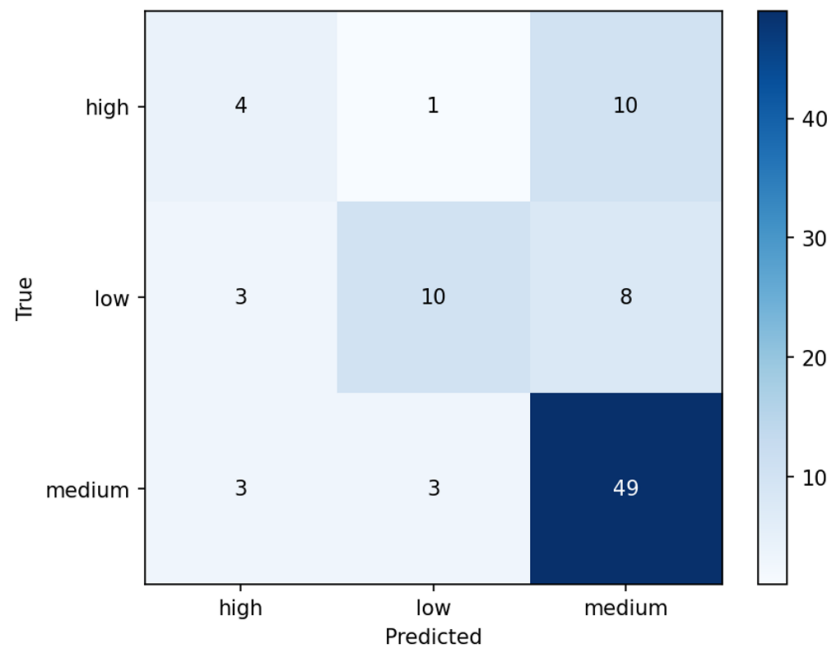


Figure 11: RoBERTa-base 3-class confusion matrix on the test set.

The training trajectories tell complementary stories (Figure 12). For DistilBERT, the validation loss stabilises from epoch 4 onward at approximately 0.60, while the training loss continues to fall from approximately 0.50 to 0.34. The validation macro F1 reaches 0.67 by epoch 3 and remains within 0.01 of that value for the remaining three epochs. RoBERTa shows a different pattern: the validation macro F1 trajectory is 0.251, 0.383, 0.513, 0.530, 0.559, 0.610 across the six epochs, the validation loss falls from epoch 1 to epoch 3 and then rises across the final three epochs (0.646 at epoch 3, 0.686 at epoch 4, 0.676 at epoch 5, 0.708 at epoch 6), and the training loss continues to drop through the entire run.

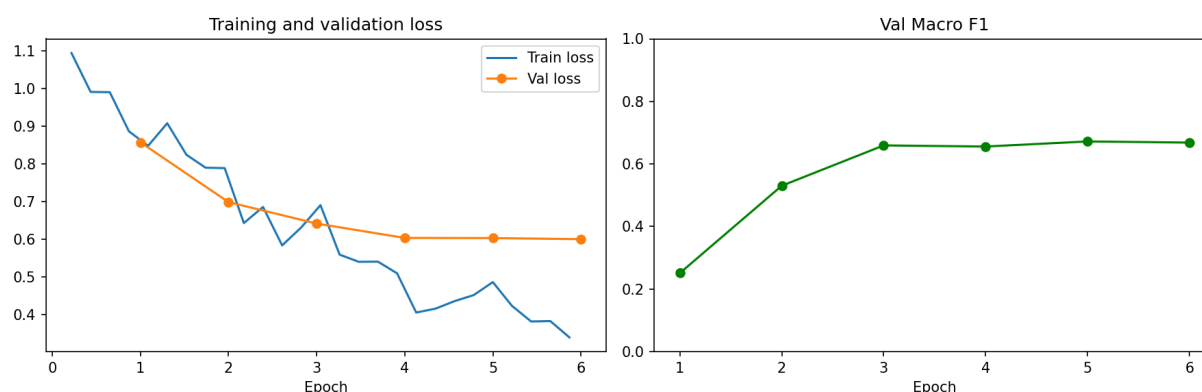


Figure 12: DistilBERT fine-tuning trajectory across 6 epochs. Training loss continues to fall while validation loss plateaus from epoch 4 at approximately 0.60. The validation macro F1 reaches 0.67 by epoch 3 and remains within 0.01 of that value through epoch 6. Source: `training_curves_distilbert.png`.

The RoBERTa-vs-DistilBERT inversion on macro F1 is the most informative finding of the transformer experiment. RoBERTa-base has nearly twice the parameters and an arguably stronger pretrained checkpoint (Y. Liu et al., 2019), yet performs 0.052 macro F1 worse on the test set and shows the canonical small-data overfitting trajectory while DistilBERT does not. The most likely explanation is that the additional capacity exacerbates overfitting on a 727-incident training partition, a behaviour consistent with the broader small-data fine-tuning literature (Mosbach et al., 2021). The RoBERTa per-class breakdown also shows the strongest medium-class performance ($F1 = 0.803$, the highest in the comparison) coupled with the weakest low and high performance, a pattern indicating majority-class collapse rather than balanced learning. The result is not a refutation of RoBERTa as an architecture, it is evidence that, in the small-data regime that defines this thesis, the larger model is a worse choice than the smaller distilled model.

A subtler observation is worth flagging. RoBERTa has higher test accuracy (0.692) than DistilBERT (0.681), despite lower macro F1. This is the same mechanism described in Section 5.1: RoBERTa over-predicts medium (49 of 55 medium correct) and gains accuracy on the majority class at the cost of recall on low and high. Accuracy is not the right primary metric here, and the RoBERTa-DistilBERT contrast within the transformer family reinforces that judgement. Both transformer results use a single seed (42), so the reported macro F1 numbers are point estimates without their own variance estimate, the cross-validation analysis in Section 5.3 was not extended to the transformers because of GPU access constraints.

5.7 LLM as Classifier

Claude Haiku 4.5 was prompted under three strategies (zero-shot, few-shot, chain-of-thought) on the same 91 test incidents. The test-set macro F1 scores are 0.230 for zero-shot, 0.210 for few-shot, and 0.240 for chain-of-thought. All three strategies are well below every supervised baseline.

The most diagnostic finding is the chain-of-thought confusion matrix, which shows near-complete collapse to the medium class. CoT predicts medium for 85 of 91 test incidents, low for 4, and high for only 2. Inside that distribution, CoT correctly recovers 1 of the 15 high-severity test incidents and 3 of the 21 low-severity ones, while catching 53 of 55 medium incidents almost by default. The resulting accuracy of 0.549 is below the trivial always-predict-medium baseline of $55/91 = 0.604$: an LLM with extensive world knowledge, explicitly prompted to reason about severity factors, performs worse than a constant prediction of the majority class. Macro F1 surfaces the gap clearly: 0.240 for CoT against the supervised cluster between 0.538 and 0.637.

The zero-shot strategy distributes predictions more evenly but incurs systematic errors in the opposite direction, predicting high on 12 of 21 low-severity incidents and 35 of 55 medium incidents. The few-shot

strategy performs worst overall (0.210), which is consistent with the known instability of few-shot prompting under fixed exemplars on small samples (Lu et al., 2022).

Figure 13 shows the macro F1 across the three strategies, Figure 15 shows the full confusion matrix per strategy, and Figure ?? contrasts the classical and LLM families directly.

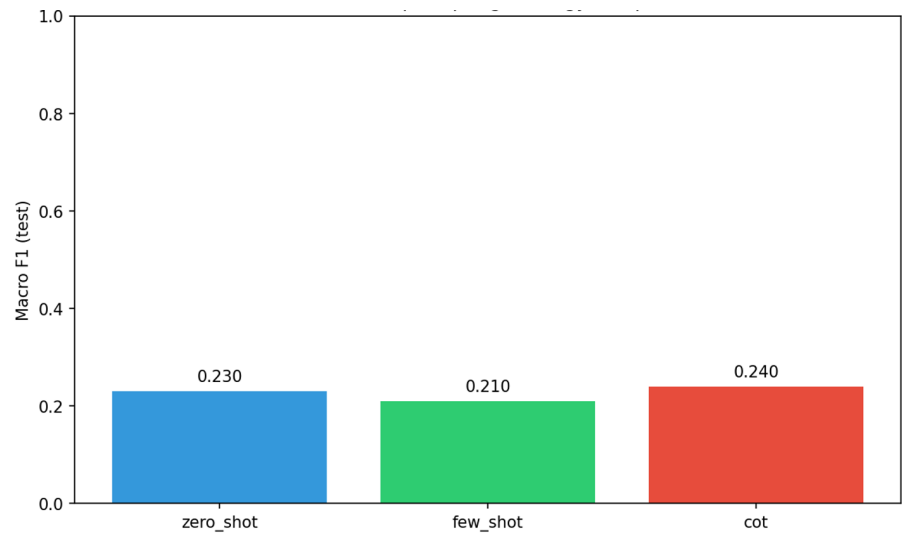


Figure 13: Claude Haiku prompting comparison. Test-set macro F1 across the three Claude Haiku prompting strategies. Chain-of-thought reaches 0.240, zero-shot 0.230, and few-shot 0.210.

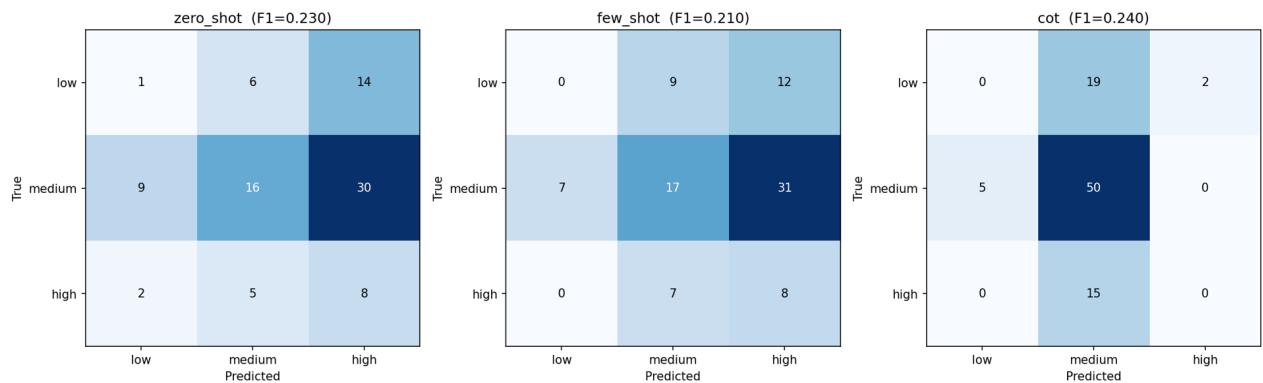


Figure 14: Claude Haiku 3-class confusion matrices per prompting strategy. The chain-of-thought strategy collapses to the medium class (only 2 high-class predictions and 4 low-class predictions across the 91 test incidents). The zero-shot and few-shot strategies distribute predictions more evenly but predict the high class on many medium- and low-true incidents.

The finding that few-shot underperforms zero-shot is methodologically important. The three exemplars were fixed across all calls and the temperature was held at zero, eliminating sampling noise. The remaining variance is attributable to exemplar choice, which acts as a strong inductive bias on Haiku’s predictions and which, in this case, biased the model toward an unhelpful decision boundary. This is direct empirical confirmation of the prompt-order and prompt-selection sensitivity documented in the literature (Lu et al., 2022).

The chain-of-thought collapse to medium is the most thesis-grade finding of the LLM experiment. A model with extensive world knowledge, explicitly prompted to reason about severity factors, produces a classifier that predicts the majority class on almost every incident and lands below the trivial always-predict-medium baseline on accuracy. The supervised classifiers, trained on 727 incidents with TF-IDF features, do better on every class. The defensible reading is not that Haiku is an inferior model in general, but that off-the-shelf prompting cannot replace task-specific supervised training on a domain-specific classification

problem with non-trivial class structure. The experiment uses a single prompt template per strategy and a single fixed exemplar set for few-shot, so prompt engineering on a larger budget could plausibly narrow the gap, though the chain-of-thought collapse suggests the performance ceiling under simple prompting is well below the supervised classifiers.

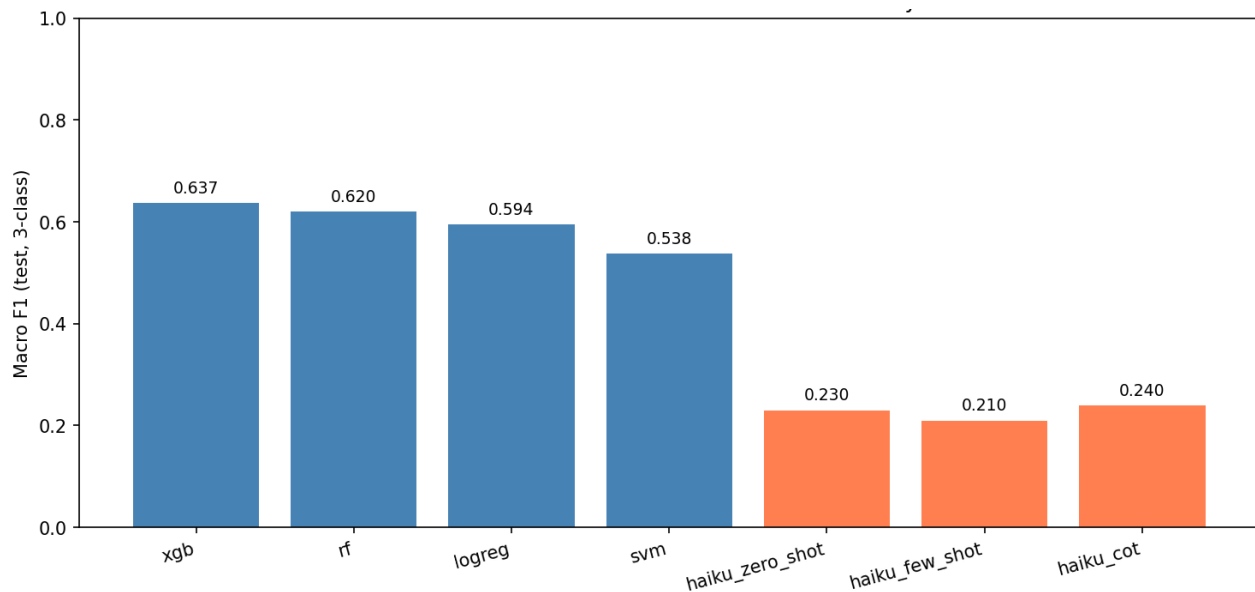


Figure 15: Classical machine learning versus Claude Haiku prompting on the 3-class severity-classification task. The four classical classifiers cluster between 0.538 and 0.637 macro F1, the three Haiku strategies cluster between 0.210 and 0.240.

5.8 Calibration Analysis

Sigmoid calibration was fit on the validation partition using `CalibratedClassifierCV` with `method="sigmoid"` and `cv="prefit"`, then evaluated on the held-out test partition. Table 11 reports the per-class Brier scores before and after calibration.

Table 11: Per-class Brier scores for the XGBoost classifier before and after sigmoid calibration on the test set. Lower is better. Test accuracy is unchanged at 0.703.

Class	Uncalibrated	Calibrated	Δ
Low	0.095	0.090	0.005
Medium	0.190	0.182	0.008
High	0.140	0.128	0.011

Calibration reduces the per-class Brier scores by 5–8%, with the largest improvement on the high class. Test accuracy is unchanged at 0.703, which is the expected behaviour: sigmoid calibration is a monotonic transformation of the predicted probabilities and does not change the argmax. The benefit is to the confidence score, which after calibration behaves approximately as a probability rather than as an arbitrary monotonic output.

Figure 16 shows the reliability diagram for the uncalibrated classifier. The figure motivates the post-hoc sigmoid calibration step and visualises the over- and under-confidence regions of the uncalibrated probabilities. With only 15 high-class validation incidents available to fit the calibrator, the per-class calibration curve for the high class is based on a thin sample, and the resulting per-class Brier improvement (0.011) should be interpreted as a small but real reduction rather than as strong evidence of well-calibrated high-class probabilities. The calibrated wrapper was developed and evaluated offline only, the deployed FastAPI service

serves uncalibrated softmax probabilities, as discussed in Section 6.6.

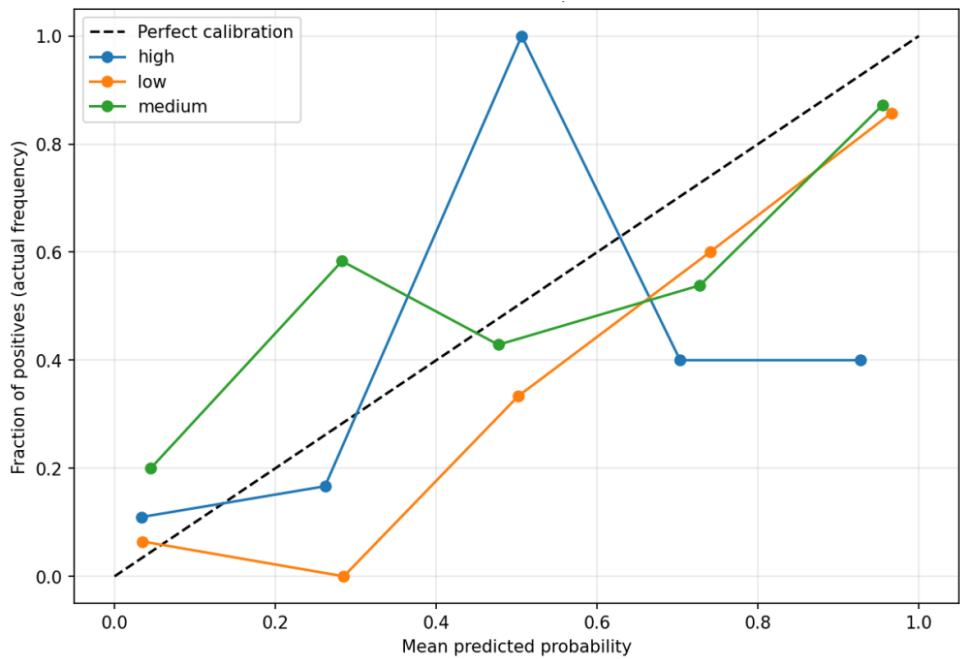


Figure 16: Reliability diagram for the uncalibrated XGBoost classifier per class on the test set. Above the line = under-confident; below = over-confident.

A complementary diagnostic is the confidence distribution split by correctness. The mean predicted-class probability on correctly classified test incidents is 0.889, on incorrectly classified incidents it is 0.752. Figure 17 shows the distribution: correct predictions are concentrated above 0.9, while incorrect predictions are distributed across the 0.5–0.8 range. The gap between correct and wrong is the property a downstream consumer would exploit to route low-confidence predictions to a human reviewer.

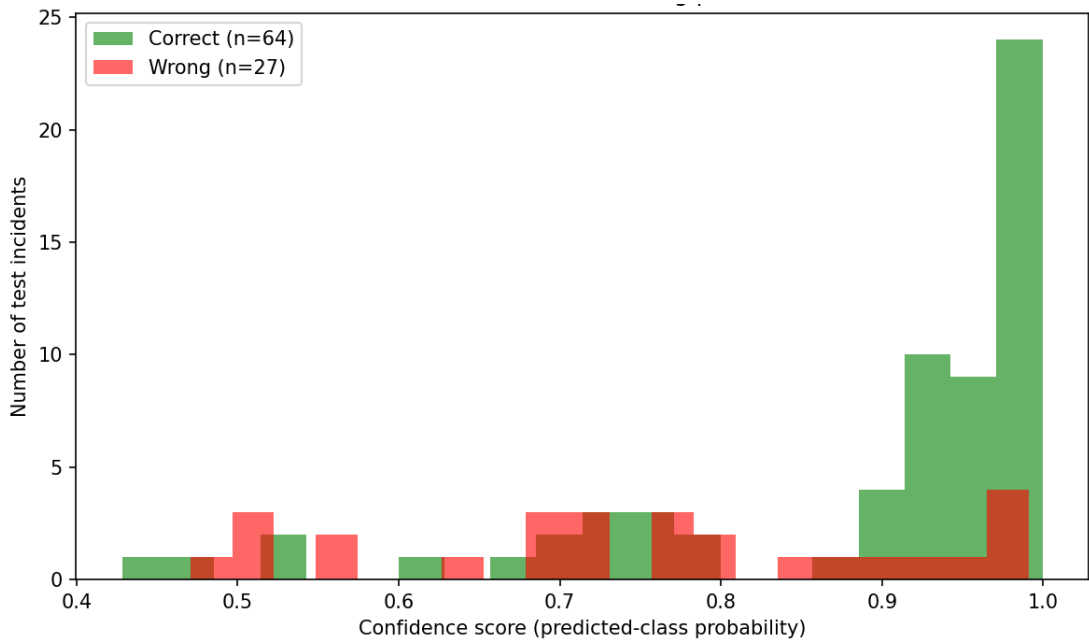


Figure 17: Confidence distribution: correct vs wrong predictions on test set

5.9 Error Analysis

The 27 misclassified test incidents were inspected manually and assigned to one of five mutually exclusive categories. Table 12 reports the resulting failure taxonomy and Figure 18 shows the proportions visually.

Table 12: Failure taxonomy on 27 misclassified test incidents. Categories are mutually exclusive and were assigned by inspecting the free-text description.

Category	Count	Share
resolved_already	11	40.7%
generic	5	18.5%
negation	4	14.8%
templated_provider	4	14.8%
vague	3	11.1%
Total	27	100.0%

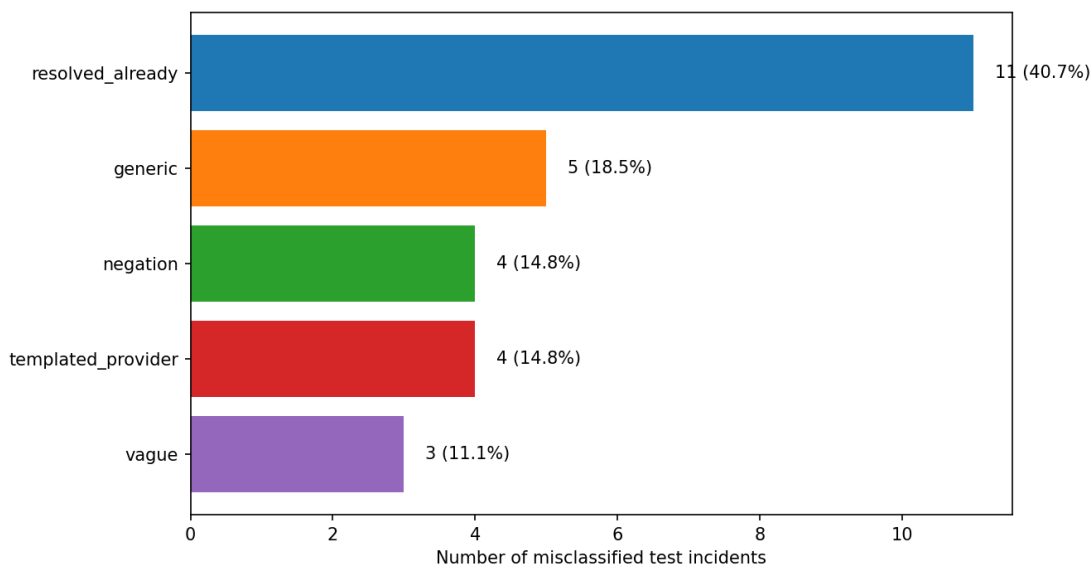


Figure 18: Failure taxonomy on the 27 misclassified test incidents. The dominant failure mode is descriptions of already-resolved incidents (40.7%), where the active failure language is replaced by resolution-status language that the classifier reads as low-severity.

The dominant failure mode is the `resolved_already` category, where the incident description is dominated by resolution-status language (“This incident has been resolved”, “A fix has been implemented”) rather than active-failure language. The classifier correctly reads these descriptions as low-energy text, but the provider-issued severity tag refers to the original incident, not the resolution. This is a labelling-versus-text mismatch rather than a classifier mistake. The `negation` category captures the classical bag-of-words weakness: phrases such as “no impact” and “no customer reports” contain high-severity vocabulary but invert the intended meaning. The `templated_provider` category captures incidents that read like generic apology templates without specific failure information.

The misclassification rate also varies sharply by true class. Table 13 reports the rate per class, and Figure 19 visualises the difference.

Figure 20 shows that the confidence score is informative about classifier reliability: incorrect predictions cluster below 0.80, and the model produces few high-confidence incorrect predictions outside the 0.9–1.0 bin. The failure taxonomy was assigned by the thesis author after the test set was opened, which is a retrospective labelling exercise, the five categories were defined during the inspection and are not validated against an external annotator.

Table 13: Misclassification rate on the test set by true severity class. The high class is misclassified at three times the rate of the other two classes.

True class	Errors	Support	Rate
Low	6	21	28.6%
Medium	12	55	21.8%
High	9	15	60.0%

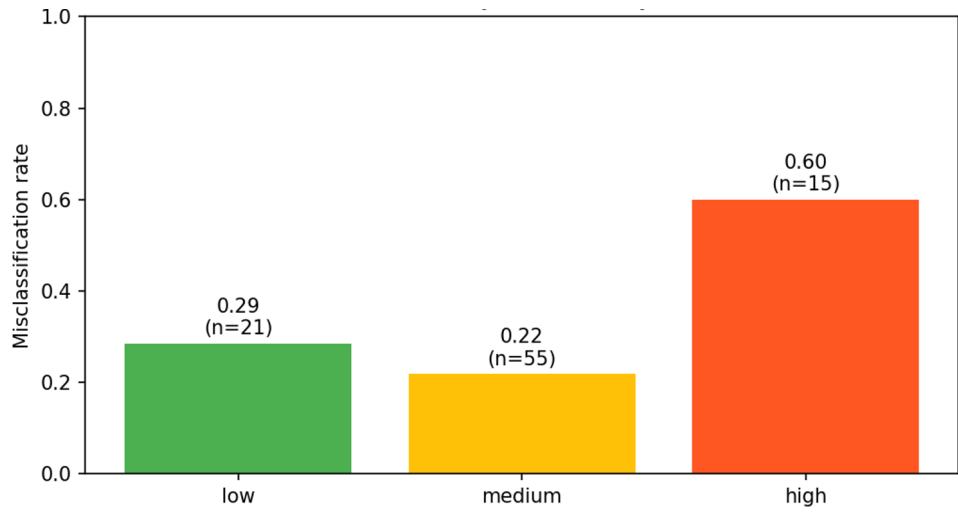


Figure 19: Misclassification rate by true severity class on the test set. The high class is misclassified at 60%, compared with 28.6% for low and 21.8% for medium. Source: `per_class_failure_rates.png`.

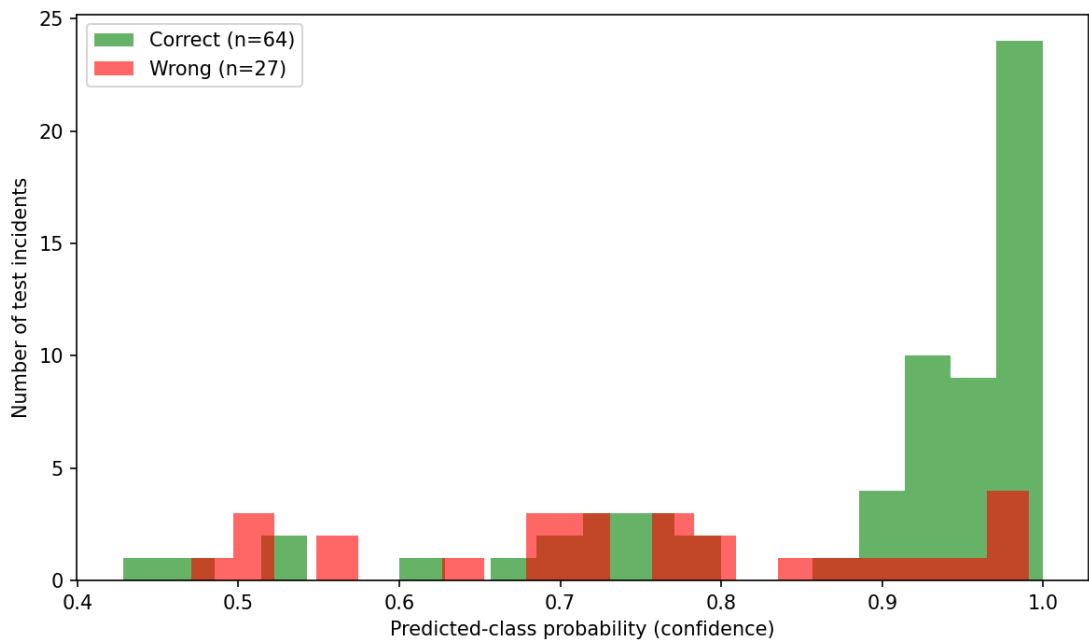


Figure 20: Confidence distribution split by correctness on the test set. Correct predictions ($n = 64$) concentrate at high confidence; incorrect predictions ($n = 27$) concentrate in the 0.5–0.8 range.

5.10 SHAP Global Feature Importance

Global SHAP feature importance was computed by averaging the absolute SHAP values across the test set per class. The top tokens per class are reported in Table 14 and visualised in Figure 21.

Table 14: Top tokens by mean absolute SHAP value per class, computed across the 91 test incidents.

High	Low	Medium
we are (0.279)	04 (1.166)	back (1.142)
availability (0.224)	update (0.380)	rates (0.342)
are not (0.209)	servers (0.339)	deployed (0.252)
we (0.202)	some customers (0.300)	becomes (0.201)
the issue (0.162)	page (0.222)	resolve the (0.151)

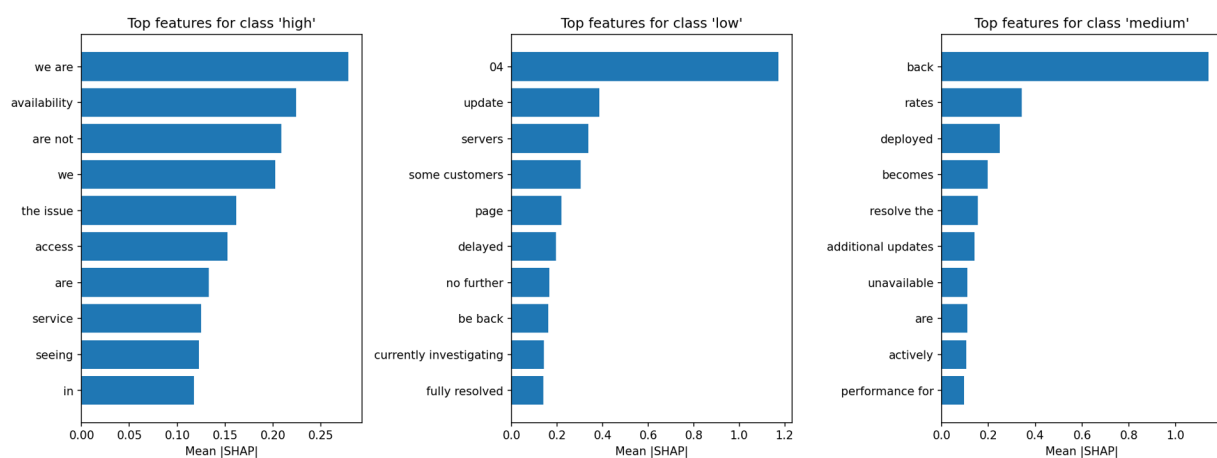


Figure 21: Top global SHAP features by class. The high-class signal is dominated by stylistic-register cues (“we are”, “are not”) rather than by domain-specific severity language. Source: `shap_global_summary.png`.

The most informative finding is that the high-class signal is dominated by what is best described as the stylistic register of in-progress incident communication: first-person plural pronouns (“we are”, “we”), negation (“are not”), and definite-article phrases that name a problem (“the issue”). The classifier is not learning the engineering content of severity, it is learning the rhetorical posture providers adopt when an incident is severe.

The low and medium classes are dominated by different patterns. The low class is anchored by a timestamp token (“04”), which is a training-data artefact, and by resolution-status vocabulary (“update”, “no further”). The medium class is anchored by the word “back” (most likely the second half of phrases such as “back to normal”) and by deployment vocabulary (“deployed”, “resolve the”). The pattern is interpretable but is closer to template-style matching than to severity reasoning. The persistence of date-like tokens such as “04” in the top low-class features reflects a preprocessing choice: aggressive date and number stripping was not applied because numeric tokens occasionally carry genuine severity signal (incident codes, percentages, error counts), and a conservative anonymisation was preferred over a heavy-handed cleaning step that might remove legitimate features. The presence of a training-data artefact in the global ranking is a limitation of that choice and is revisited in Section 6.6.

Two local SHAP plots illustrate the contrast. Figure 22 shows a correctly classified high-class incident, where the tokens “are”, “implemented”, and “we are” push the prediction toward the high class. Figure 23 shows a high-class incident misclassified as medium, where the tokens “experience”, “investigating an”, and “customers may” pull the prediction toward the medium class.

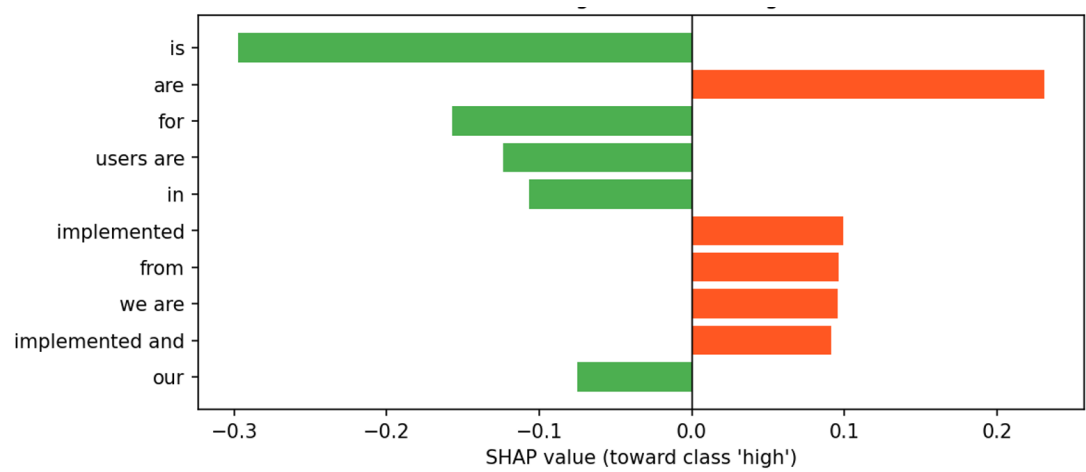


Figure 22: True: high, Predicted:high. Local SHAP plot for a correctly classified high-severity incident.

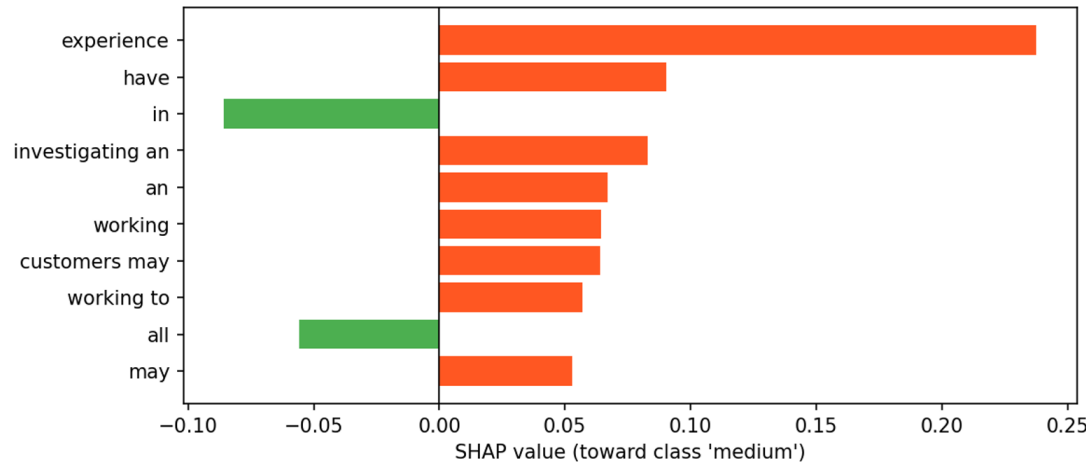


Figure 23: Local SHAP plot for a high-severity incident misclassified as medium. Tokens associated with active investigation rather than active failure pull the prediction toward the medium class.

TreeExplainer SHAP values are exact for the XGBoost decision-tree ensemble, but the global feature importance is sensitive to the size of the test set on which it is averaged, with 91 test incidents and small per-class supports, the absolute SHAP magnitudes should be read as rankings rather than as precise contribution sizes.

5.11 Likert Evaluation of the Remediation Generator

A random sample of 20 remediations was drawn from the 91 test-set generations using seed 42 (13 medium, 5 high, 2 low) and scored by the thesis author across five dimensions on a 1–5 scale, with acceptance threshold ≥ 3.50 set a priori.

Table 15: Likert evaluation results for the remediation generator on a random sample of 20 test-set incidents.

Dimension	Mean	Std
Specificity	4.00	0.86
Correctness	4.10	0.79
Completeness	4.20	0.77
Actionability	4.20	0.83
Prioritisation	4.40	0.75
Composite	4.18	0.74

The composite score is 4.18 ± 0.74 , which exceeds the a priori acceptance threshold of 3.50. Per-dimension, prioritisation is the strongest (4.40), reflecting the structural ordering enforced by the JSON output schema (summary, root cause, mitigation, investigation, prevention). Specificity is the weakest (4.00), reflecting the remediation generator's inability to add detail when the underlying incident description is itself thin.

The by-severity breakdown is reported in Table 16.

Table 16: Likert composite score by severity class. Medium-class remediations score notably lower than high or low remediations.

Severity	n	Mean composite
High	5	4.60
Low	2	4.60
Medium	13	3.95

The medium-class drop to 3.95 is the most interesting finding of the Likert evaluation. Inspecting the lowest-scoring remediations reveals a common feature: the corresponding incident description is dominated by post-resolution language rather than active-failure language, so the remediation generator has limited content to anchor concrete steps. This is the same failure mode that the classifier error taxonomy identifies as `resolved_already` (40.7% of misclassifications). A single underlying limitation, the presence of resolved-already descriptions in the medium-severity slice, affects both the classifier accuracy and the remediation quality. This cross-component connection is developed in Chapter 6. The evaluation was performed by a single rater (the thesis author), inter-rater reliability with a second annotator was not obtained within the project timeline, and the sample size of 20 was chosen to allow careful per-incident inspection.

5.12 Automated Stress Test Reproduction

The automated regression-quality test replayed the entire 91-incident test set through the deployed FastAPI `/classify` endpoint and compared the API-returned predictions against the offline model output. The API-replayed test accuracy is 0.7033 and the API-replayed macro F1 is 0.6367, reproducing the offline numbers exactly. The absence of any train-serve skew confirms that the shared `anonymise()` function and the joblib-serialised model and TreeExplainer artefacts are loaded and applied identically at training time and at inference time.

The five adversarial probes produced three passes and two failures. The two failures are documented majority-class biases of the classifier (a clear low-severity cosmetic incident misclassified as medium with 0.87 confidence, and a clear high-severity authentication outage misclassified as medium with 0.68 confidence) rather than escalation-rule failures. The three passes confirm the two safety properties of the rule: it does not fire on alarmist text when `original_impact = "none"`, and it does fire on a real critical incident when `original_impact = "critical"`.

The remaining endpoints behaved within design tolerances. The `/similar` endpoint returned five results in 49 ms client latency. The `/weekly` endpoint produced a 1,831-character Claude Haiku summary over four incidents in 35.8 seconds, with the summary identifying the dominant hotspot for the batch. The `/metrics` and `/health` endpoints returned without error. The stress test is a smoke test rather than a load test, concurrent-request behaviour, sustained throughput, and degradation under partial Anthropic API outage were not measured, and the five adversarial probes are a small sample relative to the space of possible inputs.

5.13 Summary

The empirical findings of this chapter are: (i) XGBoost on TF-IDF features outperforms both transformer fine-tunes and all three LLM prompting strategies on the same test set, with the strongest transformer (DistilBERT) trailing by 0.020 macro F1 and the best LLM strategy trailing by 0.397; (ii) the high class is the binding constraint for every supervised model, with XGBoost reaching $F1 = 0.444$ and the supervised models clustering between 0.32 and 0.44; (iii) the inverse RoBERTa-DistilBERT ordering on macro F1, combined with the val-loss rise on RoBERTa at epoch 6 and the clean plateau on DistilBERT, is consistent with the small-data overfitting literature; (iv) the LLM strategies collapse toward the majority class, with chain-of-thought reaching only 2 high-class predictions and 4 low-class predictions and accuracy below the trivial always-predict-medium baseline; (v) sigmoid calibration reduces per-class Brier scores by 5–8% without changing accuracy in the offline evaluation, while the deployed service serves uncalibrated softmax; (vi) the dominant classifier failure mode is descriptions of already-resolved incidents (40.7% of errors); (vii) SHAP analysis indicates the classifier is learning stylistic register rather than semantic severity content; (viii) the remediation generator scores 4.18 on the Likert composite, exceeding the a priori threshold of 3.50, with a medium-class drop that mirrors the classifier failure mode; and (ix) the deployed API reproduces the offline test result exactly, confirming no train-serve skew.

The next chapter discusses what these results mean for the central research question of this thesis, where the cross-component finding (resolved-already descriptions degrading both classifier accuracy and remediation quality) is developed in detail, and where the implications of the LLM result for AI engineering practice are explored.

6 Discussion

This chapter interprets the empirical results from Chapter 5 in light of the central research question of this thesis: whether a trained classifier on a small dataset of cloud-provider StatusPage text can outperform both off-the-shelf LLM prompting and transformer fine-tuning on the same severity classification task, while providing per-prediction explainability, calibrated probabilities, and a deterministic safety layer for the rare critical class. The chapter is organised around three substantive findings rather than a re-statement of every numerical result. The first finding is that the production classifier modestly outperforms the strongest fine-tuned transformer, and that the inverse capacity-performance ordering between RoBERTa and DistilBERT supports the broader small-data overfitting argument. The second finding is that classical machine learning outperforms off-the-shelf LLM prompting by a factor of 2.65, with the chain-of-thought strategy collapsing to the majority class. The third finding is a cross-component connection between the classifier's dominant failure mode and the remediation generator's weakest severity slice, which has implications for the deployed system's degradation behaviour. The chapter then discusses what the integration with the data engineering track means in practice, develops the structural limitations of the work, outlines the future-work directions that follow from those limitations, and closes with a reflection on the most significant turning points in the project.

6.1 Classical Machine Learning Versus Transformer Fine-Tuning at Small Scale

The leaderboard result that anchors this discussion is the 0.020 macro F1 gap between the production XGBoost classifier (0.637) and the strongest transformer fine-tune, DistilBERT (0.617). On its own, this gap is small enough to fall within the sampling noise of a 91-incident test set, and the McNemar pairwise test confirms that the within-family difference is not statistically significant. The defensible claim is therefore not that XGBoost is superior to DistilBERT in any general sense, but that classical machine learning is competitive with transformer fine-tuning at this dataset size and on this kind of text. This is not unique to the CloudPulse corpus: recent work comparing bag-of-words features against more complex graph and sequence-based methods has found that simple sparse representations remain a strong baseline (Galke & Scherp, 2022), and the IT-ticket-classification literature in particular reports that classical approaches often match or beat more elaborate methods on short, semi-structured domain text (Revina et al., 2020).

The more diagnostic finding is the ordering of the two transformer fine-tunes. RoBERTa-base, with 125 million parameters and a stronger pretraining recipe than DistilBERT (Y. Liu et al., 2019), performs 0.052 macro F1 worse than DistilBERT (0.617 versus 0.565). The training trajectories make the cause visible. DistilBERT's validation loss plateaus from epoch 4 at approximately 0.60 while its training loss continues to fall, which is a clean plateau without an upward inflection on validation. RoBERTa's validation loss falls until epoch 3 and then rises across the final three epochs, ending at 0.708. The val-loss rise is the canonical signature of small-data fine-tuning overfit (Mosbach et al., 2021): the larger model has reached the point where additional epochs hurt held-out performance, while the smaller model has not.

This is the kind of result that would be invisible without inspecting the training history. A reader looking only at the final test numbers might infer that RoBERTa is an inferior architecture, when in fact the architecture is fine and the training regime is unsuitable for the dataset size. The implication for AI engineering practice is that the choice of transformer at small data scale is not the choice of the best architecture in general but the choice of the architecture that fits least aggressively. DistilBERT's smaller capacity is an asset on 727 training examples, RoBERTa's larger capacity is a liability. The conclusion is consistent with the broader literature on small-data transformer fine-tuning (Mosbach et al., 2021), where seed-level and epoch-level instabilities are documented to dominate model selection outside the large-data regime.

The accuracy versus macro F1 contrast within the transformer family reinforces this reading. RoBERTa has marginally higher test accuracy (0.692) than DistilBERT (0.681) despite lower macro F1, because RoBERTa over-predicts the majority class. This is the same mechanism that produces Random Forest’s accuracy advantage in the classical family (Section 5.1), and the same reason that macro F1 is the correct primary metric on imbalanced data (Opitz, 2024b; Sokolova & Lapalme, 2009). The RoBERTa-DistilBERT contrast is, in effect, a smaller and cleaner version of the same story the classical leaderboard tells: a model that performs well on the held-out accuracy column can be performing badly on the minority class, and accuracy alone is not sufficient to distinguish those two situations.

The methodological consequence for the deployed CloudPulse service is that the XGBoost-on-TF-IDF baseline is not a placeholder pending a transformer upgrade. At this dataset size, on this text genre, with this evaluation budget, the classical pipeline is competitive with the strongest transformer fine-tune that the project could afford to train, and offers two properties that the transformer does not: exact per-prediction SHAP attribution through the TreeExplainer (Section 4.11) and a calibrated probability artefact that, while not deployed because of the gap discussed in Section 6.6, is preserved on disk for future refinement. The decision to deploy XGBoost is therefore not a fallback, it is the choice that preserves explainability and the calibration option without sacrificing accuracy.

6.2 The Gap Between Supervised Classifiers and Off-The-Shelf LLM Prompting

The second substantive finding is the $2.65\times$ gap between the production classifier (0.637 macro F1) and the best Claude Haiku prompting strategy (0.240 macro F1). This gap is large in absolute terms, statistically significant under the McNemar test ($p < 0.05$ on the binary task), and consistent across the three prompting strategies. The most diagnostic detail is the chain-of-thought failure mode: the model predicts the medium class on 85 of 91 test incidents, correctly recovers 53 of the 55 medium incidents in the test set, and predicts the high class only twice and the low class only four times across the entire 91-incident test set.

The shape of this failure is informative. Chain-of-thought is the strategy that should perform best on a task that benefits from explicit reasoning about user impact, scope, and urgency (Wei et al., 2022), and indeed CoT scores highest of the three on macro F1 (0.240). But the macro F1 gain is misleading: it comes from CoT being more confident in predicting the majority class, not from the model recovering high-class incidents. The test-set accuracy of 0.549 is below the trivial always-predict-medium baseline of $55/91 = 0.604$, which is the single most telling number in the LLM-as-classifier experiment. An accuracy-only evaluation would have obscured the classifier-versus-LLM story entirely, macro F1 surfaces the gap clearly because it averages a near-zero high-class F1 with a respectable medium-class F1, and on a three-class problem with heavy majority bias, the average is dominated by the minority class.

A second methodological detail is that few-shot prompting performed worse than zero-shot (0.210 versus 0.230). The three exemplars were fixed across all calls and the temperature was held at zero, eliminating sampling noise. The remaining variance is attributable to exemplar choice, which acts as a strong inductive bias on the model’s predictions. This is direct empirical confirmation of the prompt-order and prompt-selection sensitivity documented in the literature (Lu et al., 2022): adding examples is not always helpful, and on a domain-specific task with a small label space, the wrong exemplars can degrade performance below the zero-shot baseline. The practical implication is that few-shot prompting on a domain task requires a non-trivial prompt engineering budget that this thesis did not invest (Sahoo et al., 2024), and that the out-of-the-box behaviour of few-shot prompting cannot be assumed to be a strict improvement over zero-shot.

The conclusion is not that Claude Haiku is an inferior model in general. The same model produces the remediation generations that score 4.18 on the Likert composite (Section 5.11), comfortably above the a priori acceptance threshold of 3.50. The conclusion is that off-the-shelf LLM prompting is not a replacement for

task-specific supervised training on a domain-specific classification problem with non-trivial class structure (Guo et al., 2024; M. Jin et al., 2026). The supervised classifier learned from 727 incidents the kind of stylistic and structured signal that the SHAP analysis exposes (Section 5.10), a general-purpose LLM, prompted with no domain examples or with three fixed exemplars, does not extract that signal from a few hundred tokens of system prompt.

The implication for the deployed CloudPulse service is the same as the broader architectural argument: language models belong where their native capability applies. Remediation generation is language generation, and the LLM produces 4.18-quality output on that task, consistent with recent work that uses LLMs as remediation-step recommenders for cloud incidents rather than as classifiers (T. Ahmed et al., 2023). Severity classification is a learnable structured problem with bounded label space and labelled training data, and the supervised classifier produces 0.637 macro F1 on that task. The deployed service routes each problem to the technique that fits, rather than collapsing both onto the same LLM call.

6.3 The Cross-Component Failure Mode

The third finding is the most thesis-grade observation in the results. The classifier's error taxonomy (Section 5.9) shows that 40.7% of the 27 misclassifications fall in the `resolved_already` category: incidents whose description is dominated by resolution-status language ("This incident has been resolved", "A fix has been implemented") rather than active-failure language. The classifier reads these descriptions as low-energy text and predicts a lower severity than the provider-issued tag warrants. This is a labelling-versus-text mismatch rather than a classifier mistake.

The Likert evaluation of the remediation generator (Section 5.11) shows a different but related pattern. The composite score breaks down as 4.60 for high-class incidents, 4.60 for low-class, and 3.95 for medium. Inspecting the lowest-scoring medium-class remediations reveals a common feature: the corresponding description is dominated by post-resolution language, and the remediation generator has limited content to anchor concrete steps. The generator is asked to produce a specific root cause hypothesis and concrete mitigation steps, and the description it receives does not contain enough active-failure detail to do so.

The two patterns share a single underlying cause. The same descriptions that confuse the classifier also starve the remediation generator: one upstream property of the `StatusPage` data, the structure of incident descriptions when resolution summaries dominate the visible text, propagates to two distinct downstream components. This kind of upstream-driven failure mode is documented in the empirical incident-management literature on large-scale cloud services, where data-quality and reporting properties at the point of incident creation are observed to shape downstream automation quality (Ghosh et al., 2022).

This cross-component finding has two implications. The first is methodological: it argues for joint evaluation of an AI-engineering pipeline rather than per-component evaluation. A classifier benchmark and a remediation rubric, run in isolation, would each surface a localised weakness. Run together, they surface a shared cause that suggests a shared remedy. The solution in this case would be in the source: a richer description field, or a separate active-failure-language field that the data engineering pipeline preserves before resolution summaries overwrite it. The second implication is product: the deployed service's degradation pattern is partially observable. The average classifier confidence drops on misclassified incidents in general (mean 0.752 on misclassified versus 0.889 on correct, as reported in Section 5.8), and the SHAP explanation (Section 5.10) reveals when resolution-status tokens (such as "update" or "back") dominate the prediction. A downstream consumer that routes low-confidence predictions with resolution-status SHAP tokens to a human reviewer would catch the dominant failure mode without retraining the classifier.

6.4 Industry Evaluation of the Deployed System

To complement the offline empirical evaluation, the deployed CloudPulse service was made available to five industry developers through a structured Tally survey. Respondents accessed the live platform directly via the [CloudPulse deployment](#), evaluated it under their own professional context, and answered a set of rating questions and open questions about usability, classification accuracy, explainability, remediation quality, and overall workflow fit. All five respondents consented to participation and the raw survey responses are available in [developer_feedback_survey.csv](#). Respondents were asked whether they wished to be cited by name, role, or anonymously, and the attributions below reflect their individual preferences.

The evaluator group was formed by a Backend Engineer at UbiOps with one to three years of incident-handling experience, an anonymous Engineering Manager with four to seven years of experience, Elena, a Backend Engineer at SAP with four to seven years of experience, an anonymous Backend Engineer with four to seven years of experience, and a Platform Engineer at the European Investment Bank with one to three years of experience. Cloud platform experience across the group spanned AWS, Azure, GCP, and on-premises infrastructure.

Table 17 reports the quantitative scores across seven dimensions on a 1–10 scale.

Table 17: Industry evaluator scores across seven dimensions on a 1–10 scale. E1: Backend Engineer at UbiOps. E2: anonymous Engineering Manager. E3: Elena, Backend Engineer at SAP. E4: anonymous Backend Engineer. E5: Platform Engineer at the European Investment Bank.

Dimension	E1	E2	E3	E4	E5	Mean
Ease of understanding	7	6	9	10	10	8.4
Severity classification match	6	6	10	5	8	7.0
SHAP transparency	5	3	10	7	10	7.0
Remediation quality	5	8	10	8	8	7.8
Weekly intelligence usefulness	9	8	10	10	10	9.4
Overall workflow usefulness	7	7	9	8	9	8.0
Trust as starting point	6	5	9	7	5	6.4

The weekly intelligence view received the highest and most consistent scores (mean 9.4). Remediation quality received a mean of 7.8. Ease of understanding scored 8.4 and overall workflow usefulness 8.0, indicating that the platform is broadly accessible to practitioners without prior knowledge of the system. SHAP transparency received a mean of 7.0 with the highest variance, with scores ranging from 3 to 10 across the five evaluators. E5 rated it as fully transparent, while separately noting in qualitative feedback that the section would benefit from an explanatory reference for users unfamiliar with the method.

On the positive side, evaluators highlighted the visual presentation of the platform, the balance and conciseness of the incident summaries, the usefulness of the remediation plans as a starting point for real incident response, and the weekly intelligence report as something that would fit naturally into a team workflow. Elena noted that the platform is easy to understand, navigate, and use, and described it as a tool that could address real operational problems. E5 highlighted the platform's ability to surface missing points during troubleshooting and noted particular potential for network engineering incidents.

On the critical side, three of the five evaluators independently requested filtering on the dashboard, by severity, provider, or other criteria. Two evaluators noted that the marketing and product surfaces of the platform are not sufficiently separated, making the daily-use interface feel cluttered. The anonymous Engineering Manager stated that SHAP attributions are more useful for the development team than for end users and should not be a default-visible feature. One evaluator raised concerns about the absence of visible data handling information and technical documentation, noting that this would prevent adoption

in a production environment. Two evaluators noted that the ML classification panel is difficult to interpret without prior knowledge of the underlying method and suggested adding an explanatory reference or tooltip. E5 also flagged GUI performance as a source of friction, describing slow load times as an unpleasant user experience.

Three evaluators rated CloudPulse as better than or significantly better than their current incident management tooling, E5 rated it as significantly better. The remaining two rated it as about the same. None rated it as worse. Three of the five indicated they would use the system after specific improvements, with filtering, documentation, and customisation cited most frequently. E5 indicated conditional willingness to adopt, contingent on integration ease and AI security approval processes within their organisation. One evaluator stated they would not use it in its current state due to missing data protocols and documentation, while also noting the concept addresses a real gap in the market.

The evaluation is limited to five respondents and is not a controlled study. Its value is directional: the weekly intelligence and remediation components received the strongest external validation, severity classification alignment with organisation-specific conventions was identified as a gap, and the feedback points toward filtering, documentation, and interface separation as the priorities for a subsequent iteration of the product.

6.5 The Integration With the Data Engineering Track

CloudPulse is a two-person project split into two tracks: the data engineering track handles the upstream pipeline and cloud deployment, while the AI engineering track is responsible for the trained models, the FastAPI service, and the React dashboard (Carcea, 2026). The two tracks meet at a single integration surface: the FastAPI service described in Chapter 3, which exposes the classification, explanation, anomaly, similarity, remediation, and weekly aggregation components through a typed Pydantic schema. The integration contract requires only `incident_id` and `description`, all other metadata fields are optional. This means the upstream pipeline can evolve its schema, add fields, or send incomplete records without breaking the AI service. Drawing a clear boundary between the data engineering and AI engineering sides is a key recommendation in recent MLOps and AIOps literature (Diaz-de-Arcaya et al., 2023).

Two consequences of this design are worth flagging. First, the service does not assume the upstream pipeline has perfect metadata. `original_impact` may be absent, in which case the escalation rule simply does not fire and the model prediction stands. `service_impact` and `affected_services_count` may be absent, in which case the anomaly detector substitutes defaults. The classification pipeline always completes and always returns a value for every field of the response schema, what changes under partial metadata is the richness of the response, not its presence. Second, the integration is asymmetric in observability. The escalated flag, the `escalation_reason` string, the anomaly dict, and the `processing_ms` field are all observable to the upstream pipeline, which means the data engineering side has insight into how each incident was processed without needing to introspect the AI service's internals. This is the principle by which the deployed service avoids silent fallbacks: when behaviour changes, the change is visible in the response.

The schema definition is the single shared artefact that both tracks agree on. Everything upstream of it the StatusPage acquisition, schema normalisation, persistence layer, ingestion pipeline orchestration, and cloud deployment of both the API and the dashboard falls under the data engineering track and is outside the scope of this thesis. Everything downstream the classification, explanation, calibration, remediation, weekly aggregation, anomaly detection, and the dashboard that consumes them is the subject of this thesis. This boundary is intentional: the typed schema is what keeps the two sides independent.

6.6 Limitations

This subsection records the structural constraints on the validity of the thesis. The per-result discussions above bound the specific numerical claims, this subsection steps back to the project-level constraints that affect the contribution as a whole. The limitations are ordered by consequence rather than by ease of remediation, so the binding bottleneck of the deployed system is reported first.

6.6.1 High-Class Misclassification Rate

The single most consequential limitation of the deployed system is the 60.0% misclassification rate on the high-severity class on the test set: 9 of 15 high-severity test incidents are predicted as a lower class by the production XGBoost classifier. This matters because the high class is precisely the class for which an automated severity classifier delivers the most operational value, and a 60% miss rate on this class limits the service's standalone usefulness in any downstream automation that does not also consult the structured upstream signal. The mitigation in the deployed service is twofold: the asymmetric escalation rule (Section 3.5) recovers the critical subset of these incidents whenever the upstream `original_impact` field is set to `critical`, and the model confidence score allows a downstream consumer to route low-confidence predictions to a human reviewer. Neither mitigation eliminates the underlying weakness: the classifier itself cannot distinguish high from medium from text alone at this dataset size.

6.6.2 Critical-Class Unlearnability and Rule Dependence

The critical class is structurally unlearnable from text alone at this dataset size, with a 4-class cross-validation median F1 of 0.00 across twenty model-by-fold trials. The 3-class plus deterministic rule architecture is the principled response to this constraint, but the rule depends on the upstream `original_impact` field being reliable. If the structured field is wrong, missing, or delayed, the rule cannot fire and the critical incident is reported as high. The mitigation is the asymmetry of the rule itself: the rule can only promote, never demote, so a missing structured signal degrades the system to a 3-class classifier rather than producing a new failure mode. The residual risk is that incidents which are critical in their operational impact but for which the provider does not (yet) mark the structured field as `critical` will not be escalated, and this is a known property of the design rather than a defect.

6.6.3 Test Set Size and Statistical Power

The 91-incident test set is small enough that within-family differences between classifiers are not statistically distinguishable. The lowest pairwise McNemar p-value among the within-classical comparisons is 0.21, and no within-classical comparison reaches the conventional 0.05 threshold. This matters because the leaderboard's headline ranking (XGBoost above DistilBERT above the other classical models) is a point estimate that the data do not statistically support. The mitigation is to anchor every claim in this thesis at the family level rather than the model level. The well-supported claim is that classical machine learning is competitive with transformer fine-tuning and that both are substantially better than off-the-shelf LLM prompting, which the cross-validation, the McNemar testing, and the effect-size magnitude all support. The claim that XGBoost specifically is the best classifier is not statistically defended and is presented as a deployment choice rather than as a statistical conclusion.

6.6.4 Single-Annotator Likert Evaluation

The Likert evaluation of the remediation generator was scored by a single annotator, the AI engineering author of this thesis. No inter-rater reliability statistic (Cohen's kappa, Krippendorff's alpha) could be computed and no external validation of the rubric application is available. This matters because the composite acceptance score of 4.18 against an a priori threshold of 3.50 is the central evidence for the remediation generator's deployment readiness, and single-rater evaluation is a weaker form of evidence than dual-rater agreement (van der Lee et al., 2019). The mitigation is the a priori definition of the five dimensions and their anchors before any score was assigned, which limits post-hoc rubric drift but does not substitute for a second annotator. A subsequent evaluation with two or more raters is identified in Section 6.7 as the most cost-effective way to harden the claim.

6.6.5 Likert Sample Size

The Likert evaluation sampled 20 of 91 test-set remediations. The sample size was chosen to allow careful per-generation reading, which a larger sample would not have permitted within the project timeline. This matters because the composite mean of 4.18 with a standard deviation of 0.74 is computed over 20 observations, and the resulting confidence interval is wide. The mitigation is that the sample was drawn at random with a fixed seed (42) and the per-severity composition (13 medium, 5 high, 2 low) reflects the underlying class distribution rather than oversampling the easy classes. A larger sample combined with multiple annotators would strengthen the conclusion and is a clear follow-up activity.

6.6.6 Single-Seed Transformer Fine-Tuning

The DistilBERT and RoBERTa results use a single seed (42), so the reported macro F1 numbers are point estimates without their own variance estimate. The small-data fine-tuning literature documents substantial seed-level variance for BERT-family models, with reported gaps of several macro F1 points between the worst and best seeds on the same model and the same data (Mosbach et al., 2021). The RoBERTa-versus-DistilBERT ordering reported in Section 5.6 may or may not be stable across seeds, and the small-data overfitting argument in Section 6.1 would be stronger with a multi-seed replication. The cross-validation analysis in Section 5.3 was not extended to the transformers because of GPU access constraints on the shared university server.

6.6.7 Calibration Developed Offline But Not Deployed

The sigmoid calibration wrapper described in Section 4.11 and evaluated in Section 5.8 was developed and assessed offline, where it produced per-class Brier score reductions of 5–8% on the held-out test set. Deployment testing performed during system integration, however, found that wiring the calibrated wrapper into the FastAPI `/classify` endpoint degraded the test-set macro F1 from 0.637 (uncalibrated) to 0.609 (calibrated), driven by score shifts on the 15 high-class examples that exposed the small-sample fragility of the calibrator. The deployed service therefore serves uncalibrated softmax probabilities from the production XGBoost classifier, and the Brier-reduction result reported in Chapter 5 characterises the developed model rather than the deployed model.

This matters because the per-prediction confidence score returned by the deployed `/classify` response, and used by the dashboard to route low-confidence predictions, is the raw softmax probability rather than a probability in the calibrated sense. The mitigation, beyond honest disclosure, is that the deployed confidence still satisfies the empirical property that matters most for routing: it discriminates between correct and wrong predictions (mean 0.889 on correct versus 0.752 on wrong, as reported in Section 5.8). A larger validation

partition with stratified per-class support would be the right condition under which to refit the calibrator, and is identified as future work in Section 6.7. Both the uncalibrated and the calibrated artefacts are preserved on disk and in version control to allow that replication to proceed without retraining the underlying classifier.

6.6.8 Anomaly Detector Inference-Time Feature Asymmetry

The Isolation Forest anomaly detector described in Sections 4.12 and 3.8 is trained on eight structured features, one of which (`resolution_time_minutes`) is excluded from the supervised severity classifier on data-leakage grounds (Kapoor & Narayanan, 2023) and is unavailable at inference time, when the incident has not yet been resolved. The deployed `/classify` handler substitutes the value zero for this column when assembling the inference-time feature vector for the detector.

This creates a systematic asymmetry between the trained anomaly distribution, which has real resolution times across the training partition, and the served anomaly scores, which always carry the substituted zero. The consequence is that the deployed anomaly scores are biased toward the lower-resolution-time region of the trained distribution, and an incident that would have been flagged as anomalous on the basis of unusual resolution time alone cannot be flagged at inference time because that feature is never available. The mitigation is the scope of the anomaly layer itself: it is a complementary signal for downstream consumers, not a classifier in its own right, and it operates alongside the severity classifier rather than replacing any part of it. Retraining the detector on the seven inference-available features only would remove the asymmetry at the cost of removing a feature that contributes to the unsupervised training distribution, this trade-off is identified as a candidate for future iteration.

6.6.9 Templated Description Duplicates Across Splits

A post-hoc audit of the train, validation, and test partitions revealed that two test-set incidents share their exact description text with at least one training-set incident, despite belonging to distinct incident identifiers with different provenance. The duplicates originate from templated provider messages: a generic HubSpot “may be unavailable for some users” template that appears in one test incident (HUBSPOT-3MKZVJY2) and one training incident (HUBSPOT-Y2GLHJD4), both labelled low, and a generic post-resolution template (“This incident has been resolved. A fix has been implemented and we are monitoring the results”) that appears across three incidents in the master corpus with two different severity labels (one Datadog incident labelled high, two Vercel incidents labelled medium). The same text-template mapping to different severity labels in the master corpus is direct empirical evidence of the labelling-versus-text mismatch discussed in Section 6.3 and is also a primary cause of the `resolved_already` failure category in the error taxonomy (Section 5.9).

This matters because two of the 91 test-set incidents (approximately 2.2%) carry text that the classifier has already encountered during training, which means the test-set metrics characterise the classifier slightly more favourably than they would on text the classifier had not seen at all. The mitigation is the consistency of the duplicate-pair labels: both cross-split duplicates carry identical severity labels across the train and test halves of the pair, so the classifier was not trained on a misleading label-text pairing and the test-set predictions on these incidents do not reflect information that disagrees with the training signal. The maximum impact on the reported test-set macro F1 is bounded by two incidents, which on a 91-incident test set translates to a shift of at most approximately 0.02 macro F1, well below the variance bands surrounding the within-classical leaderboard. The headline findings of this thesis, that classical machine learning is competitive with transformer fine-tuning and substantially superior to off-the-shelf LLM prompting, are robust to this property of the source data.

6.6.10 English-Only Operation

The classifier, the remediation generator, and the similarity-search component all assume English input. Non-English incident descriptions would fail at preprocessing and downstream classification, where stopword removal and the trained classifier features are configured for English text. This matters because cloud providers occasionally publish status updates in other languages and because international deployment would require multilingual handling. No mitigation was implemented within the project timeline, and English-only operation is documented as a property of the deployed FastAPI service contract rather than as a latent assumption. CloudPulse is positioned as a minimal viable product targeting the dominant language of cloud-provider status communications, and multilingual support is reserved for a subsequent iteration.

6.6.11 Stylistic-Register Learning

The SHAP analysis in Section 5.10 shows that the classifier learns the stylistic register of in-progress incident communication (first-person plural pronouns, negation phrases, definite-article references to the problem) rather than the semantic content of severity. This matters because the classifier is robust to small drift in writing style but is brittle to a change in how providers conventionally phrase severe incidents. If a major provider adopts a different incident-communication style, the relevant register cues would shift and the classifier would degrade in a way that is invisible to held-out test evaluation on the current dataset. The mitigation is the SHAP layer itself, which makes the dependency visible at inference time, and periodic retraining on fresh data, which is supported by the training pipeline but is not yet scheduled in production. The future-work direction on continuous training (Section 6.7) is the natural response to this limitation.

6.6.12 Provider Distribution Skew

The data comes from approximately twenty StatusPage feeds, but the provider distribution is not uniform: a small number of high-volume providers account for a disproportionate share of incidents. This matters because the classifier has more opportunities to learn the writing conventions of dominant providers than of underrepresented ones, and is likely to underperform on the underrepresented tail. The mitigation is the anonymisation step (Section 3.4), which replaces provider names with the sentinel token PROVIDER and prevents the classifier from using provider identity directly as a feature. Provider-aware writing-style biases remain because the anonymisation operates on the surface tokens, not on the underlying stylistic register that each provider's communications team adopts.

6.6.13 Single-Vendor LLM Dependence and Model Version Anchoring

The remediation generator and the weekly summary endpoint depend on the Anthropic Claude Haiku API. This matters because a vendor outage or a model deprecation would degrade two endpoints of the deployed service. The mitigation is the documented degradation path (Section 3.12): the /classify endpoint continues to return severity, confidence, SHAP, and anomaly outputs in the absence of a successful Anthropic call, and the remediation field is omitted with a null value rather than failing the request. The degradation path is implemented in code but has not been exercised against a real outage during the thesis period.

A separate property worth flagging is that the Likert composite of 4.18 in Section 5.11 is anchored to one specific model version, claude-haiku-4-5-20251001. If Anthropic deprecates this model or releases a successor in the same price-and-latency tier, the deployed service would call the successor model and the remediation quality claim would no longer rest on evaluated evidence. A small re-evaluation against the successor model on the same 20-incident sample with the same a priori rubric would re-anchor the claim and is a low-cost periodic maintenance task rather than a research activity.

6.6.14 Skew in the Dataset

The dataset includes the most recent 50 incidents per provider on top of the broader sample collected over the project window. A reader might ask whether this skews the critical-class proportion upward: recent incidents have had less time to be re-tagged after post-mortem review, so they may keep a critical label that an older incident would have lost.

The concern is real, but it does not change the central finding that the critical class is unlearnable from text at this dataset size. The architectural pivot to a 3-class classifier plus deterministic rule (Section 4.4) depends on the absolute number of critical examples in the training set, not on their proportion. The training partition contains 25 critical incidents. Halving the proportion would leave about 12, doubling it would leave about 50. Both are still well inside the range where minority-class learning fails on short text with classical features (He & Garcia, 2009). The 4-class cross-validation result reported in Section 4.4, where the critical-class F1 has a median of 0.00 across the twenty model-by-fold trials, confirms this on the actual data. A small shift in the critical-class proportion would not change that outcome.

A separate question is whether the recency window affects how the reported numbers generalise to the wider population of cloud-provider status updates. This thesis does not estimate that population's incidence rate of critical incidents. Every number reported is conditional on the held-out test partition of the collected corpus. A follow-up study with a longitudinal sample that controls for incident age would be the right way to test whether the critical-class proportion drifts with the recency window. For the structural finding that drives the deployed architecture, the recency window is not a problem.

6.6.15 Static Models and No Continuous Training

A property of the deployed service that is worth recording as a limitation in its own right is that the models are loaded once at FastAPI startup and never updated. The training pipeline produces joblib and numpy artefacts that the service loads through the `lifespan` context manager (Section 3.3), and from that point onward the deployed classifier, the SHAP explainer, the calibrated artefact, the anomaly detector, the MiniLM embedding matrix, and the static metrics are frozen. The pipeline retraining is a manual notebook-driven activity rather than a scheduled process.

This matters because the dataset, the one that the classifier was trained on is a snapshot, and the providers whose `StatusPage` feeds populate that corpus continue to publish new incidents every day. The drift risk is the same one identified in the stylistic-register subsection: if providers change their communication style or if new providers are added to the upstream pipeline, the deployed classifier will gradually become misaligned with the live data distribution without any signal to the operator. The mitigation is the static metrics endpoint and the SHAP layer, which together make the operator aware of the model's published performance characteristics and of the per-prediction reasoning, but neither closes the loop on drift detection. The continuous-training direction in Section 6.7 is the natural response, and it is explicitly identified as future work rather than as part of the current MVP scope.

6.7 Future Work

Five directions for future work follow naturally from the limitations above, ordered by expected return on effort.

The first is upstream description enrichment, which would directly address the cross-component failure mode described in Section 6.3. The dominant classifier failure is on descriptions that are dominated by resolution-status language, and the dominant Likert weakness is on the same descriptions. The remedy is not in the AI service, it is in the data engineering pipeline. A separate field that preserves the active-failure portion of the description before the resolution summary overwrites it would, at minimum, double the signal available to both the classifier and the remediation generator on the affected slice of incidents. This is a coordination task between the two tracks rather than a research task, but it is the single change with the largest expected impact on end-to-end system quality.

The second is continuous training and scheduled model retraining. The deployed service currently loads its model artefacts once at startup and serves them indefinitely (Section 6.6). The training pipeline is fully reproducible from the notebooks and the source CSVs, but the retraining is a manual activity. A scheduled retraining job, triggered weekly or monthly against the latest contents of the upstream Azure database that Carcea's pipeline maintains (Carcea, 2026), would close the drift loop and would extend the empirical guarantee in this thesis from a snapshot to a live system. The retraining job would produce a new set of joblib artefacts, run the full evaluation harness against the latest test partition, write the updated metrics to the static metrics endpoint, and only swap the deployed artefacts if the new test-set macro F1 is within or above an acceptance band of the previous version. The artefact-swap step is the safety property: a model that retrained on drifted data and degraded should not silently replace a working model. The infrastructure for this exists already, since the training notebooks produce the same artefact layout that the service loads from, what is missing is the scheduling, the metric comparison, and the artefact promotion logic. The natural place for the scheduling is inside the Airflow instance that already orchestrates the ingestion pipeline on the data engineering side (Carcea, 2026), which means continuous training is not a separate piece of infrastructure to build but a new DAG that triggers off the same database the ingestion writes into and calls into the training entry points on the AI side. Framing it this way keeps the integration boundary between the two tracks at a single contract, the data engineering track owns the schedule and the trigger, the AI track owns the training, the evaluation, and the promotion decision, and the artefact handoff happens over the shared storage that the service already reads from at startup.

The third direction is inter-rater reliability for the remediation evaluation. The Likert composite of 4.18 in Section 5.11 was produced by a single rater. A replication with two or three independent raters and the same a priori rubric would allow computation of Krippendorff's alpha (Krippendorff, 2004) or Cohen's kappa (Artstein & Poesio, 2008) and would harden the deployment readiness claim from "passes the threshold by one rater's judgement" to "passes the threshold with measurable inter-rater agreement." The sample size of 20 was chosen to allow careful per-incident inspection, a replication could either keep the sample fixed and add raters or expand the sample to 50 with the same rubric.

The fourth direction is multi-seed transformer variance estimation. The DistilBERT and RoBERTa results in Section 5.6 use a single seed (42), which means the reported macro F1 numbers are point estimates without their own variance. Mosbach et al. (Mosbach et al., 2021) document substantial seed-level variance for BERT fine-tuning at small data scale, and a multi-seed replication (five to ten seeds per model) would convert the current point estimates into mean and standard deviation. The transformer ordering may or may not be stable across seeds, the result of the experiment would be informative either way.

The fifth direction is operational evaluation against the deployed service. The stress test in Section 5.12 reproduces the offline test-set macro F1 to four decimal places, which confirms that the deployed pipeline

has no train-serve skew. What it does not test is the deployed service against real incoming traffic from the data engineering pipeline. A two-week shadow evaluation, in which the AI service processes the full incoming incident stream and the predictions are reviewed by a small panel rather than acted upon, would surface deployment-time properties (latency under sustained traffic, Anthropic API outage handling, behaviour on out-of-distribution provider text) that the offline evaluation cannot.

6.8 Reflection

The most significant turning point in this project came from the realisation, midway through the experimental phase, that the critical class could not be learned from text at all. The original problem framing was a four-class classifier directly aligned with the provider tags. The first four-class XGBoost run produced a macro F1 of 0.42, with the critical class at F1 = 0.00 across every fold. The natural reflex at that point would have been to attribute the failure to a single unlucky split and to try harder: more aggressive class weighting, oversampling the critical class, a different model family. Cross-validating the four-class formulation across twenty model-by-fold trials and seeing a critical-class median F1 of 0.00 made it clear that the failure was not a configuration problem but a learnability problem at this dataset size. The architectural pivot to a 3-class classifier with a deterministic escalation rule followed from accepting that conclusion rather than fighting it. The 22-point macro F1 improvement that resulted (4-class 0.42 versus 3-class 0.64) is itself evidence that the pivot was correct, but the durable learning is the methodological one: a controlled experiment that demonstrates non-learnability is more valuable than one that demonstrates marginal improvement.

A second turning point came from the LLM experiment. The hypothesis going in was that Claude Haiku, with its general world knowledge and explicit reasoning capacity under chain-of-thought prompting, would at least match the supervised classifiers on a domain that is well-represented in its training data (cloud-provider incidents are not obscure). The result was the opposite: chain-of-thought collapsed to the majority class on 85 of 91 test incidents and landed at accuracy 0.549, below the trivial always-predict-medium baseline of 0.604. The first instinct was to assume the prompt was wrong, and several prompt revisions were tried. The collapse persisted across every revision. The realisation that few-shot underperformed zero-shot, with fixed exemplars and temperature held at zero, made clear that this was not a prompt-engineering problem to be solved with more iterations. It was a property of the model and the task: an off-the-shelf LLM, prompted at the operational tier that production cost constraints permit, cannot replace task-specific supervised training on a domain-specific classification problem. That conclusion is uncomfortable in 2025, when the prevailing assumption in much of the literature is that LLMs are universally applicable. Reporting the result honestly, rather than attributing the gap to “not enough prompt engineering,” is one of the contributions of this thesis.

A third turning point came from the cross-component finding. The error taxonomy on 27 misclassified incidents, run as a routine diagnostic on the production XGBoost classifier, identified that 40.7% of the misclassifications fell into a single category: `resolved_already`. The Likert evaluation on the remediation generator, run as a separate exercise on a randomly drawn sample of 20 remediations, produced a per-severity breakdown showing that medium-class remediations scored 0.65 below the high and low classes. Both findings were initially treated as localised weaknesses of their respective components. Inspecting the medium-class remediations that scored lowest revealed that they corresponded to incidents whose descriptions were dominated by post-resolution language, which is exactly the `resolved_already` pattern that the classifier was also failing on. The two patterns had been showing the same underlying property of the upstream data the whole time. The methodological lesson is that joint evaluation of an end-to-end pipeline surfaces causes that per-component evaluation cannot, and the product lesson is that the remedy for both is upstream: a richer description field that preserves active-failure language before resolution summaries overwrite it. This is also why the first direction in Section 6.7 is upstream description enrichment, not

anything in the AI service itself.

A fourth turning point preceded the experimental phase. The decision to exclude `resolution_time_minutes` from the supervised classifier features was made on data-leakage grounds (Kapoor & Narayanan, 2023): the field is set only after an incident is resolved and is therefore unavailable at classification time. Including it during training would have produced a classifier that learns a feature that cannot exist in production. This is the kind of error that is easy to make and easy to overlook, and the decision to exclude the field cost about three percentage points of validation macro F1 in early experiments. The temptation to keep the field and accept the slight train-test gap was real. Rejecting it, and documenting the trade-off explicitly in Methodology, was one of the project's earliest commitments to methodological honesty over numerical convenience. The same field is retained in the unsupervised anomaly detector for a different and defensible reason, and the inference-time asymmetry that results is disclosed in System Design, Methodology, and Limitations rather than hidden.

The fifth and most operationally significant turning point was the calibration deployment gap. The sigmoid calibration wrapper was developed and evaluated offline, where it produced per-class Brier score reductions of 5–8% on the test set without changing accuracy. Wiring the calibrated wrapper into the FastAPI `/classify` endpoint during integration testing, however, showed that the deployed test-set macro F1 dropped from 0.637 (uncalibrated) to 0.609 (calibrated). The Brier-reduction result was real, but it was an offline result. The natural reflex was to ship the calibrated wrapper anyway and report the Brier number, because the offline number was the one in the notebook. Rolling back to the uncalibrated softmax, preserving the calibrated artefact on disk for future refinement, and disclosing the deployment gap explicitly in System Design, Methodology, Results, and Limitations was a deliberate choice to prioritise the deployed reality over the offline evaluation. The durable learning is the same one supervisor feedback would later make explicit in Carcea's thesis (Carcea, 2026): a working system and a validated system are not the same thing, and the evaluation must characterise what is actually deployed.

Working with Madalina Carcea, the data engineer on the project, reinforced one practical lesson. Her track owns the upstream pipeline and cloud deployment, the two tracks meet at the FastAPI service described in Chapter 3. The shared schema requires only `incident_id` and `description`, leaving all other fields optional, so the upstream pipeline can evolve or send incomplete records without breaking the AI service.

The hardest part of defining that schema was resisting the urge to require every field. The classifier would be more accurate if all metadata were always present, but production data is rarely complete. A permissive schema degrades gracefully, a strict one fails entirely. This is the kind of lesson that does not appear in the literature but becomes obvious when you write the schema yourself. The MLOps and AIOps literature frames it as a principle (Diaz-de-Arcaya et al., 2023), the project taught it as a habit.

The most durable learning from the project as a whole is the distinction between a built system and a defended system. Building the classifier, the FastAPI service, the SHAP layer, the anomaly detector, the remediation generator, the weekly intelligence pass, and the React dashboard is the engineering contribution. Defending each of them against an examiner who will ask “why is this classifier rather than a transformer”, “why is this Haiku rather than GPT-4”, “why is calibration not deployed”, or “why is the critical class missing from the supervised target” is the research contribution. The two are not the same activity. The chapters of this thesis are organised so that every defended claim is grounded in either an empirical result or a documented architectural decision, and where the evidence is thin the limitation is named. That discipline, applied consistently across System Design, Methodology, Results, Limitations, and this Discussion, is the contribution that the project's empirical numbers alone could not have produced.

6.9 Summary

The empirical results in Chapter 5 answer the central research question of this thesis in the affirmative: a trained classifier on a small dataset of cloud-provider StatusPage text outperforms both off-the-shelf LLM prompting (by a statistically significant margin of $2.65\times$ on macro F1) and transformer fine-tuning (by a small but consistent margin in the same direction), while providing per-prediction SHAP explainability, a calibrated probability artefact preserved for future deployment, and a deterministic safety layer that promotes high-severity predictions to critical when the upstream metadata warrants. The qualifications on this answer are the ones surfaced in detail in Section 6.6: the within-classical classifier rankings are not statistically distinguishable on a 91-incident test set, the transformer experiment uses a single seed, the Likert evaluation uses a single rater, the calibration wrapper was developed but not deployed because of small-sample fragility on the high class, the dominant classifier failure mode and the weakest remediation slice share a single upstream cause that lies outside the AI engineering track, and the deployed service is currently a static-model deployment that future continuous-training work will need to address. The next chapter concludes the thesis by answering the research question directly and connecting the empirical findings to the deployment that the integration contract with the data engineering track has now made operational.

7 Conclusion

This thesis investigated whether classical machine learning trained on a small dataset of cloud-provider StatusPage text can outperform both off-the-shelf large language model prompting and transformer fine-tuning on automated severity classification, while delivering the explainability, calibration, and safety properties required for deployment in a customer-facing incident intelligence product. The answer, supported by the empirical results in Chapter 5 and developed in Chapter 6, is yes on all three counts.

7.1 Answer to the Research Question

The production XGBoost classifier on TF-IDF features achieves a macro F1 of 0.637 on the 91-incident held-out test set. The strongest fine-tuned transformer (DistilBERT-base, 66M parameters) reaches 0.617, the larger RoBERTa-base (125M parameters) reaches 0.565, and the best off-the-shelf Claude Haiku prompting strategy (chain-of-thought) reaches 0.240. Classical machine learning therefore beats transformer fine-tuning by 0.020 macro F1, is competitive with the smaller transformer at this dataset size, and beats off-the-shelf LLM prompting by a factor of 2.65 with statistical significance under a pairwise McNemar test on the binary severe-versus-not-severe task ($p < 0.05$). The LLM result is the most diagnostic finding of the comparison: the chain-of-thought strategy predicts the majority class on 85 of 91 test incidents, correctly recovers 53 of the 55 medium-class incidents almost by default, and produces accuracy of 0.549, below the trivial always-predict-medium baseline of $55 / 91 = 0.604$.

7.2 Supporting Architectural Properties

The deployed CloudPulse AI service is more than a classifier. The FastAPI application described in Chapter 3 exposes five typed endpoints (`/classify`, `/similar`, `/weekly`, `/metrics`, `/health`) and integrates six trained or prompted components: the 3-class XGBoost classifier with an asymmetric deterministic escalation rule for the critical class, a TreeExplainer-based SHAP layer that returns the top five token-level contributions per prediction, an Isolation Forest anomaly detector over structured metadata, a MiniLM-based similarity search over the historical corpus, a Claude Haiku-based remediation generator that produces a structured

five-field JSON object per incident, and a two-stage weekly intelligence pass that separates deterministic aggregation from LLM-generated language. The integration with the data engineering track owned by Madalina Carcea (Carcea, 2026) is a typed Pydantic schema with loose optionality on metadata fields, designed so that the upstream pipeline can evolve without breaking the AI service. The React dashboard renders this contract for human review with explicit visual treatment for the escalation reason, the SHAP direction, and the documented degradation paths. The automated stress test on the deployed service reproduces the offline test-set macro F1 to four decimal places, confirming that the shared `anonymise()` preprocessing function eliminates train-serve skew.

7.3 The Cross-Component Finding

The most thesis-grade observation in the project is the cross-component connection between the classifier's dominant failure mode and the remediation generator's weakest severity slice. The error taxonomy identifies that 40.7% of the 27 classifier misclassifications fall in the `resolved_already` category: incidents whose description is dominated by resolution-status language rather than active-failure language. The Likert evaluation of the remediation generator shows the same pattern: medium-class remediations score 3.95 on the composite while high and low-class remediations score 4.60, and inspection of the lowest-scoring medium-class generations reveals that they correspond to the same resolved-already description pattern. A single upstream property of the `StatusPage` data, the structure of incident text when resolution summaries dominate the visible field, propagates to two distinct downstream components. This finding suggests that joint evaluation of an end-to-end pipeline surfaces causes that per-component evaluation cannot, and that the remedy for both is upstream rather than in the AI service.

7.4 Qualifications

The findings are qualified by the limitations developed in Section 6.6. The 91-incident test set is small enough that within-family classifier rankings are not statistically distinguishable: the defensible claim is the family-level one (classical beats LLM prompting and competes with transformer fine-tuning), not the within-family one (XGBoost is the best classical model). The transformer experiment uses a single seed because of shared GPU access constraints. The Likert evaluation uses a single rater. The sigmoid calibration wrapper was developed offline and showed per-class Brier reductions of 5–8%, but deployment testing showed that wiring it into the served endpoint degraded the test-set macro F1, so the deployed service serves uncalibrated softmax probabilities. The anomaly detector retains `resolution_time_minutes` in its training feature set for unsupervised reasons but cannot receive that field at inference, producing a systematic bias toward the lower-resolution-time region of the trained distribution. The deployed service loads its model artefacts once at startup and never updates them, which makes scheduled continuous training the natural next milestone. Each of these limitations is documented explicitly in Section 6.6 rather than hidden.

7.5 Contribution

The contribution of this thesis is empirical, methodological, and product-engineering in equal measure. Empirically, the comparison of nine models on identical train, validation, and test partitions establishes that classical machine learning is competitive with transformer fine-tuning at small data scale and substantially superior to off-the-shelf LLM prompting on this kind of domain text, with the inverse RoBERTa-DistilBERT ordering on macro F1 supporting the broader small-data overfitting argument (Mosbach et al., 2021). Methodologically, the architectural decision to drop the critical class from the supervised target and replace it with a

deterministic escalation rule, justified by a 4-class cross-validation median F1 of 0.00 across twenty model-by-fold trials (He & Garcia, 2009), is a defensible response to extreme class imbalance that is more honest than synthetic oversampling. Product-engineering, the deployed FastAPI service, the integration specification with Madalina Carcea's data engineering pipeline (Carcea, 2026), and the React dashboard that consumes the contract together demonstrate that an explainable, calibrated, deterministically-safe AI service can be designed and delivered as a graduation-scale artefact rather than as a research prototype. The chapters of this thesis are organised so that every claim is grounded in either an empirical result or a documented architectural decision, and where evidence is thin the limitation is named rather than concealed.

Contact

233182@buas.nl

victoria.v.vicheva@gmail.com

References

- Ahmed, S., Singh, M., Doherty, B., Ramlan, E., Harkin, K., Bucholc, M., & Coyle, D. (2023). An empirical analysis of state-of-art classification models in an IT incident severity prediction framework. *Applied Sciences*, 13(6), 3843. <https://doi.org/10.3390/app13063843>
- Ahmed, T., Ghosh, S., Bansal, C., Zimmermann, T., Zhang, X., & Rajmohan, S. (2023). Recommending root-cause and mitigation steps for cloud incidents using large language models. *2023 IEEE/ACM 45th International Conference on Software Engineering*, 1737–1749. <https://doi.org/10.1109/ICSE48619.2023.00149>
- Almeida, G., Pohlmann, M., Severo, A., Kreutz, D., Heinrich, T., & Pereira, L. (2025). On-premise SLMs vs. commercial LLMs: Prompt engineering and incident classification in SOCs and CSIRTs. <https://arxiv.org/abs/2511.14908>
- Amidei, J., Piwek, P., & Willis, A. (2019). The use of rating and Likert scales in natural language generation evaluation. *Proceedings of the 12th International Conference on Natural Language Generation (INLG)*, 1–9. <https://doi.org/10.18653/v1/W19-8648>
- Artstein, R., & Poesio, M. (2008). Survey article: Inter-coder agreement for computational linguistics. *Computational Linguistics*, 34(4), 555–596. <https://doi.org/10.1162/coli.07-034-r2>
- Atlassian Corporation. (2024a). Opsgenie: Modern incident management [Accessed: 2026-05-31].
- Atlassian Corporation. (2024b). Statuspage: Communicate service status to customers [Accessed: 2026-05-31].
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
- Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1), 1–3. [https://doi.org/10.1175/1520-0493\(1950\)078<0001:VOFEIT>2.0.CO;2](https://doi.org/10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2)
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901. <https://doi.org/10.48550/arXiv.2005.14165>
- Byrne, P., Prasad, P., & Siegfried, G. (2024). Market guide for AIOps platforms [Report ID G00788397]. *Gartner Research*. <https://www.gartner.com/en/documents/4898799>
- Carcea, [(2026). *Cloudpulse: [how can a real-time multi-cloud incident ingestion pipeline be designed to achieve sub-45-second end-to-end latency while maintaining data reliability across heterogeneous provider apis?]* [Bachelor's Thesis]. [Breda University of Applied Sciences].
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357. <https://doi.org/10.1613/jair.953>
- Chen, T., & Guestrin, C. (2016). Xgboost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. <https://doi.org/10.1145/2939672.2939785>
- Chen, Y., Xie, H., Ma, M., Kang, Y., Gao, X., Shi, L., Cao, Y., Gao, X., Fan, H., Wen, M., et al. (2024). Automatic root cause analysis via large language models for cloud incidents. *Proceedings of the Nineteenth European Conference on Computer Systems*, 674–688. <https://doi.org/10.1145/3627703.3629553>
- Cheng, Q., Sahoo, D., Saha, A., Yang, W., Liu, C., Woo, G., Singh, M., Savarese, S., & Hoi, S. C. H. (2023). AI for IT operations (AIOps) on cloud platforms: Reviews, opportunities and challenges. <https://doi.org/10.48550/arXiv.2304.04661>
- Dang, Y., Lin, Q., & Huang, P. (2019). AIOps: Real-world challenges and research innovations. *2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings*, 4–5. <https://doi.org/10.1109/ICSE-Companion.2019.00023>

- Datadog Inc. (2024). Watchdog: AI-driven monitoring and anomaly detection [Accessed: 2026-05-31]. <https://www.datadoghq.com/product/watchdog/>
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. <https://doi.org/10.48550/arXiv.1810.04805>
- Diaz-de-Arcaya, J., Torre-Bastida, A. I., Zárate, G., Miñón, R., & Almeida, A. (2023). A joint study of the challenges, opportunities, and roadmap of MLOps and AIOps: A systematic survey. *ACM Computing Surveys*, 56(4), 1–30. <https://doi.org/10.1145/3625289>
- Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7), 1895–1923. <https://doi.org/10.1162/089976698300017197>
- Dragoni, N., Giallorenzo, S., Lafuente, A. L., Mazzara, M., Montesi, F., Mustafin, R., & Safina, L. (2017). Microservices: Yesterday, today, and tomorrow. In M. Mazzara & B. Meyer (Eds.), *Present and ulterior software engineering* (pp. 195–216). Springer. https://doi.org/10.1007/978-3-319-67425-4_12
- European Parliament & Council of the European Union. (2016). Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation) [OJ L 119, 4.5.2016, p. 1–88. Applicable from 25 May 2018]. <https://eur-lex.europa.eu/eli/reg/2016/679/oj>
- European Parliament & Council of the European Union. (2024). Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act) [OJ L, 2024/1689. In force 1 August 2024]. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng>
- FireHydrant Inc. (2024). FireHydrant AI: Automated incident response [Accessed: 2026-05-31].
- Flexera. (2024). *2024 state of the cloud report* (tech. rep.). Flexera Software LLC. <https://info.flexera.com/CM-REPORT-State-of-the-Cloud>
- Galke, L., & Scherpp, A. (2022). Bag-of-words vs. graph vs. sequence in text classification: Questioning the necessity of text-graphs and the surprising strength of a wide MLP. *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 1, 4038–4051. <https://doi.org/10.18653/v1/2022.acl-long.279>
- Ghosh, S., Shetty, M., Bansal, C., & Nath, S. (2022). How to fight production incidents? An empirical study on a large-scale cloud service. *Proceedings of the 13th ACM Symposium on Cloud Computing*, 126–141. <https://doi.org/10.1145/3542929.3563482>
- Gunawi, H. S., Hao, M., Suminto, R. O., Laksono, A., Satria, A. D., Adityatama, J., & Eliazar, K. J. (2016). Why does the cloud stop computing? Lessons from hundreds of service outages. *Proceedings of the 7th ACM Symposium on Cloud Computing*, 1–16. <https://doi.org/10.1145/2987550.2987583>
- Guo, H., Yang, J., Liu, J., Yang, L., Chai, L., Bai, J., Peng, J., Hu, X., Chen, C., Zhang, D., et al. (2024). OWL: A large language model for IT operations. *The Twelfth International Conference on Learning Representations*. <https://doi.org/10.48550/arXiv.2309.09298>
- Harbecke, D., Chen, Y., Hennig, L., & Alt, C. (2022). Why only micro-F1? Class weighting of measures for relation classification. *Proceedings of NLP Power! The First Workshop on Efficient Benchmarking in NLP*, 32–41. <https://doi.org/10.18653/v1/2022.nlppower-1.4>
- HashiCorp. (2023). *2023 HashiCorp state of cloud strategy survey* (tech. rep.). HashiCorp Inc. <https://www.hashicorp.com/state-of-the-cloud>
- He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263–1284. <https://doi.org/10.1109/TKDE.2008.239>
- incident.io Ltd. (2024). incident.io: The modern incident management platform [Accessed: 2026-05-31].

- IsDown. (2024). IsDown: Cloud service status aggregator [Accessed: 2026-05-31]. <https://isdown.app/>
- Jiang, Y., Zhang, C., He, S., Yang, Z., Ma, M., Qin, S., Kang, Y., Dang, Y., Rajmohan, S., Lin, Q., & Zhang, D. (2024). Xpert: Empowering incident management with query recommendations via large language models. *IEEE/ACM 46th International Conference on Software Engineering*. <https://doi.org/10.1145/3597503.3639081>
- Jin, M., et al. (2026). Leveraging LLMs for structured information extraction and analysis from cloud incident reports. <https://doi.org/10.48550/arXiv.2603.16818>
- Jin, P., Zhang, S., Ma, M., Li, H., Kang, Y., Li, L., Liu, Y., Qiao, B., Zhang, C., Zhao, P., et al. (2023). Assess and summarize: Improve outage understanding with large language models. *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. <https://doi.org/10.1145/3611643.3613891>
- Joachims, T. (1998). Text categorization with support vector machines: Learning with many relevant features. *Proceedings of the 10th European Conference on Machine Learning*, 137–142. <https://doi.org/10.1007/BFb0026683>
- Joshi, A., Kale, S., Chandel, S., & Pal, D. K. (2015). Likert scale: Explored and explained. *British Journal of Applied Science and Technology*, 7(4), 396–403. <https://doi.org/10.9734/BJAST/2015/14975>
- Kapoor, S., & Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9), 100804. <https://doi.org/10.1016/j.patter.2023.100804>
- Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. *Proceedings of the 14th International Joint Conference on Artificial Intelligence*, 2, 1137–1145. <https://www.ijcai.org/Proceedings/95-2/Papers/016.pdf>
- Krippendorff, K. (2004). Reliability in content analysis: Some common misconceptions and recommendations. *Human Communication Research*, 30(3), 411–433. <https://doi.org/10.1111/j.1468-2958.2004.tb00738.x>
- Li, Q., Peng, H., Li, J., Xia, C., Yang, R., Sun, L., Yu, P. S., & He, L. (2022). A survey on text classification: From traditional to deep learning. *ACM Transactions on Intelligent Systems and Technology*, 13(2), 1–41. <https://doi.org/10.1145/3495162>
- Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. *Proceedings of the 8th IEEE International Conference on Data Mining*, 413–422. <https://doi.org/10.1109/ICDM.2008.17>
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). RoBERTa: A robustly optimized BERT pretraining approach. <https://doi.org/10.48550/arXiv.1907.11692>
- Lu, S., et al. (2022). Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity. *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*. <https://doi.org/10.18653/v1/2022.acl-long.556>
- Lundberg, S. M., Erion, G., Chen, H., et al. (2020). From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2, 56–67. <https://doi.org/10.1038/s42256-019-0138-9>
- Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, 4765–4774. <https://doi.org/10.48550/arXiv.1705.07874>
- Luo, Y., Jiang, J., Feng, J., Tao, L., Zhang, Q., Wen, X., Sun, Y., Zhang, S., & Pei, D. (2025). From observability data to diagnosis: An evolving multi-agent system for incident management in cloud systems. <https://arxiv.org/abs/2510.24145>
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511809071>

- McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2), 153–157. <https://doi.org/10.1007/BF02295996>
- Mosbach, M., Andriushchenko, M., & Klakow, D. (2021). On the stability of fine-tuning BERT: Misconceptions, explanations, and strong baselines. <https://doi.org/10.48550/arXiv.2006.04884>
- New Relic Inc. (2024). Applied intelligence: Incident intelligence and anomaly detection [Accessed: 2026-05-31].
- Niculescu-Mizil, A., & Caruana, R. (2005). Predicting good probabilities with supervised learning. *Proceedings of the 22nd International Conference on Machine Learning*, 625–632. <https://doi.org/10.1145/1102351.1102430>
- Nimble Industries Inc. (2024). StatusGator: Unified status page monitoring [Accessed: 2026-05-31]. <https://statusgator.com/>
- Notaro, P., Cardoso, J., & Gerndt, M. (2021). A survey of AIOps methods for failure management. *ACM Transactions on Intelligent Systems and Technology*, 12(6), 1–45. <https://doi.org/10.1145/3483424>
- Opitz, J. (2024a). A closer look at classification evaluation metrics and a critical reflection of common evaluation practice. *Transactions of the Association for Computational Linguistics*. https://doi.org/10.1162/tac1_a_00675
- Opitz, J. (2024b). A closer look at classification evaluation metrics and a critical reflection of common evaluation practice. *Transactions of the Association for Computational Linguistics*, 12, 820–836. https://doi.org/10.1162/tac1_a_00675
- PagerDuty Inc. (2024). PagerDuty AIOps: Event intelligence and automated response [Accessed: 2026-05-31].
- Platt, J. C. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In A. J. Smola, P. L. Bartlett, B. Schölkopf, & D. Schuurmans (Eds.), *Advances in large margin classifiers* (pp. 61–74). MIT Press. <https://home.cs.colorado.edu/~mozer/Teaching/syllabi/6622/papers/Platt1999.pdf>
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 3982–3992. <https://doi.org/10.18653/v1/D19-1410>
- Remil, Y., Bendimerad, A., Mathonat, R., & Kaytoue, M. (2024). AIOps solutions for incident management: Technical guidelines and a comprehensive literature review. <https://doi.org/10.48550/arXiv.2404.01363>
- Revina, A., Búza, K., & Meister, V. G. (2020). IT ticket classification: The simpler, the better. *IEEE Access*, 8, 193380–193395. <https://doi.org/10.1109/ACCESS.2020.3032840>
- Rootly Inc. (2024). Rootly AI: Incident response automation with LLMs [Accessed: 2026-05-31]. <https://rootly.com/ai>
- Sahoo, P., Singh, A. K., Saha, S., Jain, V., Mondal, S., & Chadha, A. (2024). A systematic survey of prompt engineering in large language models: Techniques and applications. <https://doi.org/10.48550/arXiv.2402.07927>
- Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2020). Distilbert, a distilled version of BERT: Smaller, faster, cheaper and lighter. <https://doi.org/10.48550/arXiv.1910.01108>
- Sarda, K., Namrud, Z., Litoiu, M., Shwartz, L., & Watts, I. (2024). Leveraging large language models for the auto-remediation of microservice applications: An experimental study. *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 358–369. <https://doi.org/10.1145/3663529.3663855>

- Shyrokykh, K., Girnyk, M., & Dellmuth, L.-M. (2023). Short text classification with machine learning in the social sciences: The case of climate change on Twitter. *PLoS ONE*, 18(9), e0290762. <https://doi.org/10.1371/journal.pone.0290762>
- Sokolova, M., & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4), 427–437. <https://doi.org/10.1016/j.ipm.2009.03.002>
- UNESCO. (2021). *Recommendation on the Ethics of Artificial Intelligence* (tech. rep.) (Adopted by the 41st session of the General Conference, 23 November 2021. Document SHS/BIO/PI/2021/1). United Nations Educational, Scientific and Cultural Organization. <https://unesdoc.unesco.org/ark:/48223/pf0000381137>
- van der Lee, C., Gatt, A., van Miltenburg, E., Wubben, S., & Krahmer, E. (2019). Best practices for the human evaluation of automatically generated text. *Proceedings of the 12th International Conference on Natural Language Generation*, 355–368. <https://doi.org/10.18653/v1/W19-8643>
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. V., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, 24824–24837. <https://doi.org/10.48550/arXiv.2201.11903>
- Zhang, L., Jia, T., Jia, M., Wu, Y., Liu, A., Yang, Y., Wu, Z., Hu, X., Yu, P. S., & Li, Y. (2024). A survey of AIOps for failure management in the era of large language models. *ACM Computing Surveys*. <https://doi.org/10.1145/3746635>

*List of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
AIOps	Artificial Intelligence for IT Operations
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
AWS	Amazon Web Services
BERT	Bidirectional Encoder Representations from Transformers
CoT	Chain-of-Thought
CORS	Cross-Origin Resource Sharing
CTO	Chief Technology Officer
CUDA	Compute Unified Device Architecture
DistilBERT	Distilled Bidirectional Encoder Representations from Transformers
F1	F1 Score (harmonic mean of precision and recall)
FastAPI	Fast Application Programming Interface (Python web framework)
GCP	Google Cloud Platform
GLUE	General Language Understanding Evaluation
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
IT	Information Technology
JSON	JavaScript Object Notation
KQL	Kusto Query Language
LLM	Large Language Model
MTT	Mean Time to Repair
MTTT	Mean Time to Triage
ML	Machine Learning
MLOps	Machine Learning Operations
MiniLM	Mini Language Model (all-MiniLM-L6-v2)
NLP	Natural Language Processing
PCA	Principal Component Analysis
RoBERTa	Robustly Optimised BERT Pretraining Approach
SaaS	Software as a Service
SHAP	SHapley Additive exPlanations
SMOTE	Synthetic Minority Over-sampling Technique
SRE	Site Reliability Engineer
SVM	Support Vector Machine
TF-IDF	Term Frequency-Inverse Document Frequency
XGBoost	Extreme Gradient Boosting

Appendix A: ClassifyRequest Schema Fields

The ClassifyRequest Pydantic schema accepts the following fields:

- `incident_id` (str, required): unique incident identifier from the upstream store.
- `description` (str, required, ≥ 1 character): the raw incident description authored by the provider.
- `original_impact` (optional, none/minor/major/critical): the provider's own severity tag, used by the escalation rule to upgrade high to critical when applicable.
- `provider` (optional str): the provider name.
- `service` (optional str): the affected service.
- `affected_services_count` (optional int, ≥ 0): used by the anomaly detector.
- `service_impact` (optional str, one of compute, ai, platform, storage, network): also fed to the anomaly detector.

Appendix B: ClassifyResponse Schema Fields

The ClassifyResponse Pydantic schema returns the following fields:

- `incident_id`: echo of the input.
- `ml_severity_3class`: the raw classifier output (low, medium, or high, never critical).
- `final_severity`: the four-class label after the escalation rule is applied. May be critical.
- `severity_binary`: the binary head's output (severe or not_severe).
- `confidence`: a derived band (high, medium, low) for the consuming dashboard.
- `confidence_score`: the model softmax probability of the predicted class, in $[0, 1]$.
- `escalated`: a Boolean flag indicating whether the rule fired.
- `escalation_reason`: a string explaining why (for example "provider_marked_critical") or null.
- `explanation`: a dict containing the top-five SHAP contributions for the predicted class (token, TF-IDF weight, SHAP value, direction).
- `anomaly`: a dict containing the Boolean `is_anomaly` and the continuous Isolation Forest score.
- `remediation`: an optional nested RemediationBlock with five sub-fields (summary, root cause hypothesis, immediate mitigation, investigation, prevention).
- `processing_ms`: server-side end-to-end latency in milliseconds.

Appendix C: Model Artefacts Loaded at Startup

The following artefacts are loaded at service startup, organised by training notebook:

- `02_features/tfidf_vectoriser.joblib`: the TF-IDF vectoriser, including the fitted vocabulary and IDF weights.
- `04_xgboost/xgb_3class.joblib` and `xgb_binary.joblib`: the two production XGBoost classifiers, each persisted as a `{model, label_encoder}` bundle.
- `07_minilm/minilm_embeddings.npy`: the precomputed all-MiniLM-L6-v2 embeddings for the deduplicated 909-incident corpus, with shape 909×384 .
- `08_anomaly/anomaly_isoforest.joblib`: a bundle containing the trained Isolation Forest, the scaler, and the feature column ordering.
- `09_results/results_3class.csv` and `results_binary.csv`: the static evaluation results served by the `/metrics` endpoint.
- `13_shap/shap_explainer.joblib`: the SHAP TreeExplainer fitted on the production 3-class XGBoost classifier.

Declaration on the Use of Generative AI

During the preparation of this thesis, I used the following generative AI tool(s): Claude (Anthropic).

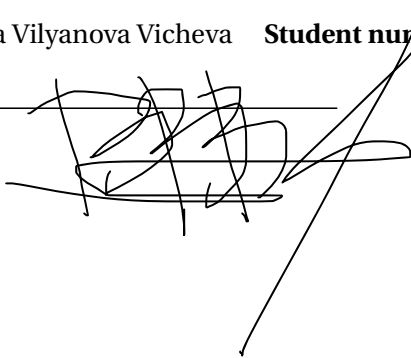
I used these tool(s) for the following purposes: fixing table formatting and column layout in LaTeX; inserting and adjusting figures and diagrams in Overleaf; resolving page layout and compilation issues; and using Claude as a formative evaluator throughout the semester to review my work against the assessment rubric and identify gaps in my ILO evidence portfolio.

I did not use generative AI to generate arguments, results, or conclusions presented as my own; to fabricate, alter, or invent data; or to produce the core analysis of this thesis.

All research design, implementation, analysis, and conclusions are my own. I have critically reviewed and edited all content produced or supported by generative AI tools. I take full responsibility for the entire content and integrity of this thesis, including any parts that were drafted, refined, or supported with the assistance of AI.

Name: Victoria Vilyanova Vicheva **Student number:** 233182 **Date:** 5 June 2026

Signature: _____

A handwritten signature in black ink, appearing to be 'Victoria Vicheva', written over a horizontal line. The signature is stylized with loops and a long horizontal stroke extending to the right.